six_port_cac

April 11, 2026

1 SPARX: An Automated, Programmatically Generated Frequency-Scalable Six-Port Receiver in 130-nm CMOS

Authors: David Kellerer-Pirklbauer, Simon Dorrer, and Harald Pretl

Institute: Institute for Integrated Circuits and Quantum Computing, Johannes Kepler University (JKU), Linz, Austria

Email: {david.kellerer-pirklbauer, simon.dorrer, harald.pretl}@jku.at

2 Project Description

In this notebook, a tapeout-ready frequency-scalable mmWave six-port receiver is programmatically generated in Python using GDSFactory and the 130-nm IHP-Open-PDK with CMOS-only devices. **SPARX** stands for **S**ix-**P**ort **A**utomated Receiver (**RX**).

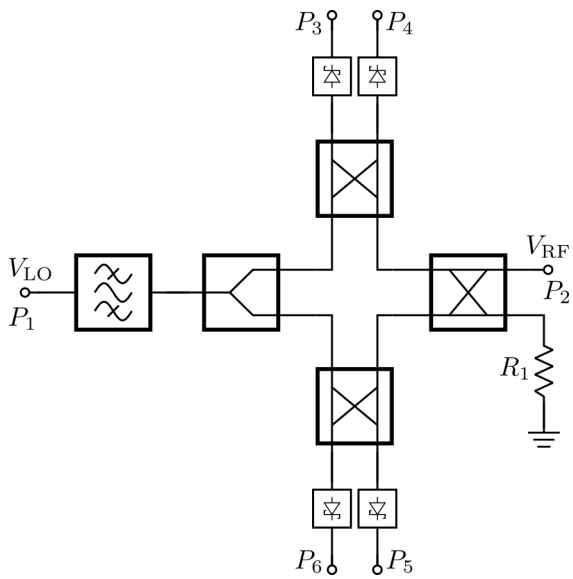
The goal of this project was to integrate the IHP-Open-PDK and self-made RF devices into GDSFactory to test its limits with an actual tapeout. During development, we pushed the design toward full frequency scalability, so changing the target frequency in the notebook automatically resizes the full RF layout.

The receiver consists of three branch-line couplers, a hairpin coupled-line bandpass filter, a Wilkinson power divider, and a power detector utilizing Schottky barrier diodes (SBDs).

3 Block Diagram

The following block diagram visualizes the mmWave six-port receiver design.

[12]:
```
from IPython.display import Image, display
display(Image(filename="assets/sparx_blockdiagram.png", width=576))
```



4 Video Demonstration

In the following video, the generation of six-port receivers from 60 GHz to 300 GHz in under one minute is demonstrated.

[13]:
```
from IPython.display import Video, display
display(Video("assets/sparx_gen_60ghz_to_300ghz.mp4", embed=True,␣
↪mimetype="video/mp4", width=1024))
```

```
<IPython.core.display.Video object>
```

5 References

To understand the principle of six-port receivers and their architectures, it is recommended to read the following references: - A. Koelpin, G. Vinci, B. Laemmle, D. Kissinger and R. Weigel, "The Six-Port in Modern Society," in IEEE Microwave Magazine, vol. 11, no. 7, pp. 35-43, Dec. 2010, doi: 10.1109/MMM.2010.938584: https://ieeexplore.ieee.org/document/5590352 - T. Hentschel, "The six-port as a communications receiver," in IEEE Transactions on Microwave Theory and Techniques, vol. 53, no. 3, pp. 1039-1047, March 2005, doi: 10.1109/TMTT.2005.843507: https://ieeexplore.ieee.org/document/1406309 - M. Mailand, "System Analysis of Six-Port-Based RF-Receivers," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 65, no. 1, pp. 319-330, Jan. 2018, doi: 10.1109/TCSI.2017.2734922: https://ieeexplore.ieee.org/document/8011483

6 Requirements

To build this six-port receiver, the following tools and their respective dependencies are required: - GDSFactory: https://github.com/gdsfactory/gdsfactory - Updated IHP-Open-PDK GDSFac-

tory version: https://github.com/iic-jku/IHP - IHP-Open-PDK: https://github.com/iic-jku/IHP-Open-PDK

The updated IHP-Open-PDK GDSFactory version contains all self-made RF devices and wraps existing PCells provided by the IHP-Open-PDK, allowing them to be used directly within the GDSFactory framework. We choose this approach because it requires very little maintenance. If IHP changes the layout of a cell, no wrapper update is necessary. Only interface changes to a PCell function require updates on our side.

> **Note:** This notebook only demonstrates the layout generation of the six-port receiver. For the full design flow, refer to the official SPARX GitHub repository, which includes S-parameter simulation of the passive RF structures with AWS Palace, a complete LVS, DRC, and RCX verification flow using KLayout, Magic, and Netgen, and SBD-based power detector design in Xschem with ngspice and VACASK simulation. The entire flow is controlled by a Makefile. Simply install the IIC-OSIC-TOOLS container from JKU, clone the repository and run `make all` to build and verify the six-port receiver at any target frequency.

Installation of IHP-Open-PDK GDSFactory version:

First, our GDSFactory integration of the IHP-Open-PDK is cloned and installed. This IHP-Open-PDK version contains all self-made RF devices and wraps existing PCells provided by the IHP-Open-PDK, allowing them to be used directly within the GDSFactory framework. We choose this approach because it requires very little maintenance. If IHP changes the layout of a cell, no wrapper update is necessary. Only interface changes to a PCell function require updates on our side.

Running `pip install .` installs the required Python dependencies automatically.

[ ]:
```
! rm -rf IHP/
! git clone -b IHP-TO https://github.com/iic-jku/IHP.git
! cd IHP && pip install .
```

```
'rm' is not recognized as an internal or external command,
operable program or batch file.

Cloning into 'IHP'…

Processing c:\users\si150\eda\designs\sg13g2_sparx\sscs-ose-code-a-chip\ihp
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Getting requirements to build wheel: started
  Getting requirements to build wheel: finished with status 'done'
  Preparing metadata (pyproject.toml): started
  Preparing metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: gdsfactory~=9.18.1 in
c:\users\si150\anaconda3\lib\site-packages (from ihp==0.0.6) (9.18.1)
Requirement already satisfied: gplugins~=1.4.2 in
c:\users\si150\anaconda3\lib\site-packages (from
gplugins[sax]~=1.4.2->ihp==0.0.6) (1.4.2)
Requirement already satisfied: doroutes>=0.2.0 in
c:\users\si150\anaconda3\lib\site-packages (from ihp==0.0.6) (0.5.0)
Requirement already satisfied: jinja2<4 in c:\users\si150\anaconda3\lib\site-
```

```
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (3.1.6)
Requirement already satisfied: loguru<1 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (0.7.3)
Requirement already satisfied: matplotlib<4 in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(3.10.6)
Requirement already satisfied: numpy in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.2.0)
Requirement already satisfied: orjson<4 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (3.11.8)
Requirement already satisfied: pandas in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.3.3)
Requirement already satisfied: pydantic>=2 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.11.10)
Requirement already satisfied: pydantic-settings<3 in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(2.12.0)
Requirement already satisfied: pydantic-extra-types<3 in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(2.11.2)
Requirement already satisfied: pyyaml in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (6.0.3)
Requirement already satisfied: qrcode in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (8.2)
Requirement already satisfied: rectpack<1 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (0.2.2)
Requirement already satisfied: rich<15 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (14.2.0)
Requirement already satisfied: scipy<2 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (1.16.3)
Requirement already satisfied: shapely<3 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.1.2)
Requirement already satisfied: toolz<2 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (1.0.0)
Requirement already satisfied: types-PyYAML in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(6.0.12.20260408)
Requirement already satisfied: typer<1 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (0.17.4)
Requirement already satisfied: kfactory<1.15,>=1.14.4 in
c:\users\si150\anaconda3\lib\site-packages (from
kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (1.14.4)
Requirement already satisfied: watchdog<7 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (4.0.2)
Requirement already satisfied: freetype-py in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.5.1)
Requirement already satisfied: mapbox_earcut in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
```

```
(2.0.0)
Requirement already satisfied: networkx in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (3.5)
Requirement already satisfied: scikit-image in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(0.25.2)
Requirement already satisfied: trimesh>=4.4.1 in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(4.11.5)
Requirement already satisfied: ipykernel in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (6.31.0)
Requirement already satisfied: attrs in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (25.4.0)
Requirement already satisfied: graphviz in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (0.21)
Requirement already satisfied: pyglet<3 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (2.1.14)
Requirement already satisfied: typing-extensions in
c:\users\si150\anaconda3\lib\site-packages (from gdsfactory~=9.18.1->ihp==0.0.6)
(4.15.0)
Requirement already satisfied: pygit2 in c:\users\si150\anaconda3\lib\site-
packages (from gdsfactory~=9.18.1->ihp==0.0.6) (1.19.2)
Requirement already satisfied: pint in c:\users\si150\anaconda3\lib\site-
packages (from gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.25.3)
Requirement already satisfied: tqdm in c:\users\si150\anaconda3\lib\site-
packages (from gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (4.67.1)
Requirement already satisfied: xarray==2025.7.1 in
c:\users\si150\anaconda3\lib\site-packages (from
gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (2025.7.1)
Requirement already satisfied: packaging>=24.1 in
c:\users\si150\anaconda3\lib\site-packages (from
xarray==2025.7.1->gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (25.0)
Requirement already satisfied: sax<0.16.0,>=0.15.6 in
c:\users\si150\anaconda3\lib\site-packages (from
gplugins[sax]~=1.4.2->ihp==0.0.6) (0.15.20)
Requirement already satisfied: MarkupSafe>=2.0 in
c:\users\si150\anaconda3\lib\site-packages (from
jinja2<4->gdsfactory~=9.18.1->ihp==0.0.6) (3.0.2)
Requirement already satisfied: aenum<4,>=3.1.15 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (3.1.17)
Requirement already satisfied: cachetools>=5.2.0 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (5.5.1)
Requirement already satisfied: klayout<0.31.0,>=0.30 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.30.7)
Requirement already satisfied: rapidfuzz<4,>=3.13.0 in
```

```
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (3.14.5)
Requirement already satisfied: rectangle-packer<3,>=2.0.2 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (2.1.0)
Requirement already satisfied: requests<3,>=2.23.3 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (2.32.5)
Requirement already satisfied: ruamel.yaml.string<0.2,>=0.1.1 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.1.1)
Requirement already satisfied: semver<4,>=3.0.4 in
c:\users\si150\anaconda3\lib\site-packages (from kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (3.0.4)
Requirement already satisfied: ipyevents>=2.0.2 in
c:\users\si150\anaconda3\lib\site-packages (from
kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (2.0.4)
Requirement already satisfied: ipython>=9.0.2 in
c:\users\si150\anaconda3\lib\site-packages (from
kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (9.7.0)
Requirement already satisfied: ipytree>=0.2.2 in
c:\users\si150\anaconda3\lib\site-packages (from
kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.2.2)
Requirement already satisfied: ipywidgets>=8.1.5 in
c:\users\si150\anaconda3\lib\site-packages (from
kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (8.1.7)
Requirement already satisfied: colorama>=0.3.4 in
c:\users\si150\anaconda3\lib\site-packages (from
loguru<1->gdsfactory~=9.18.1->ihp==0.0.6) (0.4.6)
Requirement already satisfied: win32-setctime>=1.0.0 in
c:\users\si150\anaconda3\lib\site-packages (from
loguru<1->gdsfactory~=9.18.1->ihp==0.0.6) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in
c:\users\si150\anaconda3\lib\site-packages (from
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
c:\users\si150\anaconda3\lib\site-packages (from
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in
c:\users\si150\anaconda3\lib\site-packages (from
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
c:\users\si150\anaconda3\lib\site-packages (from
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (1.4.9)
Requirement already satisfied: pillow>=8 in c:\users\si150\anaconda3\lib\site-
packages (from matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in
c:\users\si150\anaconda3\lib\site-packages (from
```

```
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
c:\users\si150\anaconda3\lib\site-packages (from
matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (2.9.0.post0)
Requirement already satisfied: annotated-types>=0.6.0 in
c:\users\si150\anaconda3\lib\site-packages (from
pydantic>=2->gdsfactory~=9.18.1->ihp==0.0.6) (0.6.0)
Requirement already satisfied: pydantic-core==2.33.2 in
c:\users\si150\anaconda3\lib\site-packages (from
pydantic>=2->gdsfactory~=9.18.1->ihp==0.0.6) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in
c:\users\si150\anaconda3\lib\site-packages (from
pydantic>=2->gdsfactory~=9.18.1->ihp==0.0.6) (0.4.2)
Requirement already satisfied: python-dotenv>=0.21.0 in
c:\users\si150\anaconda3\lib\site-packages (from pydantic-
settings<3->gdsfactory~=9.18.1->ihp==0.0.6) (1.1.0)
Requirement already satisfied: cffi>=2.0 in c:\users\si150\anaconda3\lib\site-
packages (from pygit2->gdsfactory~=9.18.1->ihp==0.0.6) (2.0.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
c:\users\si150\anaconda3\lib\site-packages (from requests<3,>=2.23.3-
>kfactory<1.15,>=1.14.4->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1-
>ihp==0.0.6) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
c:\users\si150\anaconda3\lib\site-packages (from requests<3,>=2.23.3-
>kfactory<1.15,>=1.14.4->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1-
>ihp==0.0.6) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
c:\users\si150\anaconda3\lib\site-packages (from requests<3,>=2.23.3-
>kfactory<1.15,>=1.14.4->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1-
>ihp==0.0.6) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
c:\users\si150\anaconda3\lib\site-packages (from requests<3,>=2.23.3-
>kfactory<1.15,>=1.14.4->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1-
>ihp==0.0.6) (2026.1.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
c:\users\si150\anaconda3\lib\site-packages (from
rich<15->gdsfactory~=9.18.1->ihp==0.0.6) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
c:\users\si150\anaconda3\lib\site-packages (from
rich<15->gdsfactory~=9.18.1->ihp==0.0.6) (2.19.2)
Requirement already satisfied: ruamel.yaml>=0.17.17 in
c:\users\si150\anaconda3\lib\site-packages (from ruamel.yaml.string<0.2,>=0.1.1-
>kfactory<1.15,>=1.14.4->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1-
>ihp==0.0.6) (0.18.16)
Requirement already satisfied: jax>=0.6.0 in c:\users\si150\anaconda3\lib\site-
packages (from sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.9.2)
Requirement already satisfied: jaxtyping>=0.2.38 in
c:\users\si150\anaconda3\lib\site-packages (from
```

```
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.3.9)
Requirement already satisfied: klujax<0.5.0,>=0.4.1 in
c:\users\si150\anaconda3\lib\site-packages (from
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.4.8)
Requirement already satisfied: lark>=1.2.2 in c:\users\si150\anaconda3\lib\site-
packages (from sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (1.3.1)
Requirement already satisfied: natsort>=8.0.0 in
c:\users\si150\anaconda3\lib\site-packages (from
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (8.4.0)
Requirement already satisfied: optax>=0.2.8 in
c:\users\si150\anaconda3\lib\site-packages (from
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.2.8)
Requirement already satisfied: scikit-rf>=1.8.0 in
c:\users\si150\anaconda3\lib\site-packages (from
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (1.11.0)
Requirement already satisfied: sympy>=1.14.0 in
c:\users\si150\anaconda3\lib\site-packages (from
sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (1.14.0)
Requirement already satisfied: jaxlib>=0.5.0 in
c:\users\si150\anaconda3\lib\site-packages (from
klujax<0.5.0,>=0.4.1->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6)
(0.9.2)
Requirement already satisfied: click>=8.0.0 in
c:\users\si150\anaconda3\lib\site-packages (from
typer<1->gdsfactory~=9.18.1->ihp==0.0.6) (8.2.1)
Requirement already satisfied: shellingham>=1.3.0 in
c:\users\si150\anaconda3\lib\site-packages (from
typer<1->gdsfactory~=9.18.1->ihp==0.0.6) (1.5.4)
Requirement already satisfied: pycparser in c:\users\si150\anaconda3\lib\site-
packages (from cffi>=2.0->pygit2->gdsfactory~=9.18.1->ihp==0.0.6) (2.23)
Requirement already satisfied: decorator>=4.3.2 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(5.2.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(1.1.1)
Requirement already satisfied: jedi>=0.18.1 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(0.19.2)
Requirement already satisfied: matplotlib-inline>=0.1.5 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(0.2.1)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
c:\users\si150\anaconda3\lib\site-packages (from
```

```
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(3.0.52)
Requirement already satisfied: stack_data>=0.6.0 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in
c:\users\si150\anaconda3\lib\site-packages (from
ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(5.14.3)
Requirement already satisfied: wcwidth in c:\users\si150\anaconda3\lib\site-
packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=9.0.2-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.2.13)
Requirement already satisfied: comm>=0.1.3 in c:\users\si150\anaconda3\lib\site-
packages (from
ipywidgets>=8.1.5->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(0.2.3)
Requirement already satisfied: widgetsnbextension~=4.0.14 in
c:\users\si150\anaconda3\lib\site-packages (from
ipywidgets>=8.1.5->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(4.0.14)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in
c:\users\si150\anaconda3\lib\site-packages (from
ipywidgets>=8.1.5->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(3.0.15)
Requirement already satisfied: ml_dtypes>=0.5.0 in
c:\users\si150\anaconda3\lib\site-packages (from
jax>=0.6.0->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.5.4)
Requirement already satisfied: opt_einsum in c:\users\si150\anaconda3\lib\site-
packages (from
jax>=0.6.0->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (3.4.0)
Requirement already satisfied: wadler-lindig>=0.1.3 in
c:\users\si150\anaconda3\lib\site-packages (from
jaxtyping>=0.2.38->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6)
(0.1.7)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
c:\users\si150\anaconda3\lib\site-packages (from jedi>=0.18.1->ipython>=9.0.2-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.8.5)
Requirement already satisfied: mdurl~=0.1 in c:\users\si150\anaconda3\lib\site-
packages (from markdown-it-py>=2.2.0->rich<15->gdsfactory~=9.18.1->ihp==0.0.6)
(0.1.2)
Requirement already satisfied: absl-py>=0.7.1 in
c:\users\si150\anaconda3\lib\site-packages (from
optax>=0.2.8->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (2.4.0)
Requirement already satisfied: pytz>=2020.1 in
c:\users\si150\anaconda3\lib\site-packages (from
pandas->gdsfactory~=9.18.1->ihp==0.0.6) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
```

```
c:\users\si150\anaconda3\lib\site-packages (from
pandas->gdsfactory~=9.18.1->ihp==0.0.6) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\si150\anaconda3\lib\site-
packages (from python-
dateutil>=2.7->matplotlib<4->gdsfactory~=9.18.1->ihp==0.0.6) (1.17.0)
Requirement already satisfied: ruamel.yaml.clib>=0.2.7 in
c:\users\si150\anaconda3\lib\site-packages (from ruamel.yaml>=0.17.17-
>ruamel.yaml.string<0.2,>=0.1.1->kfactory<1.15,>=1.14.4-
>kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6) (0.2.14)
Requirement already satisfied: executing>=1.2.0 in
c:\users\si150\anaconda3\lib\site-packages (from stack_data>=0.6.0-
>ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in
c:\users\si150\anaconda3\lib\site-packages (from stack_data>=0.6.0-
>ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4->gdsfactory~=9.18.1->ihp==0.0.6)
(3.0.0)
Requirement already satisfied: pure_eval in c:\users\si150\anaconda3\lib\site-
packages (from stack_data>=0.6.0->ipython>=9.0.2->kfactory[ipy]<1.15,>=1.14.4-
>gdsfactory~=9.18.1->ihp==0.0.6) (0.2.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
c:\users\si150\anaconda3\lib\site-packages (from
sympy>=1.14.0->sax<0.16.0,>=0.15.6->gplugins[sax]~=1.4.2->ihp==0.0.6) (1.3.0)
Requirement already satisfied: debugpy>=1.6.5 in
c:\users\si150\anaconda3\lib\site-packages (from
ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (1.8.16)
Requirement already satisfied: jupyter-client>=8.0.0 in
c:\users\si150\anaconda3\lib\site-packages (from
ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (8.6.3)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
c:\users\si150\anaconda3\lib\site-packages (from
ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (5.8.1)
Requirement already satisfied: nest-asyncio>=1.4 in
c:\users\si150\anaconda3\lib\site-packages (from
ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (1.6.0)
Requirement already satisfied: psutil>=5.7 in c:\users\si150\anaconda3\lib\site-
packages (from ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (7.0.0)
Requirement already satisfied: pyzmq>=25 in c:\users\si150\anaconda3\lib\site-
packages (from ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (27.1.0)
Requirement already satisfied: tornado>=6.2 in
c:\users\si150\anaconda3\lib\site-packages (from
ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (6.5.1)
Requirement already satisfied: platformdirs>=2.5 in
c:\users\si150\anaconda3\lib\site-packages (from jupyter-
core!=5.0.*,>=4.12->ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (4.5.0)
Requirement already satisfied: pywin32>=300 in
c:\users\si150\anaconda3\lib\site-packages (from jupyter-
core!=5.0.*,>=4.12->ipykernel->gdsfactory~=9.18.1->ihp==0.0.6) (311)
```

```
Requirement already satisfied: flexcache>=0.3 in
c:\users\si150\anaconda3\lib\site-packages (from
pint->gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.3)
Requirement already satisfied: flexparser>=0.4 in
c:\users\si150\anaconda3\lib\site-packages (from
pint->gplugins~=1.4.2->gplugins[sax]~=1.4.2->ihp==0.0.6) (0.4)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in
c:\users\si150\anaconda3\lib\site-packages (from scikit-
image->gdsfactory~=9.18.1->ihp==0.0.6) (2.37.2)
Requirement already satisfied: tifffile>=2022.8.12 in
c:\users\si150\anaconda3\lib\site-packages (from scikit-
image->gdsfactory~=9.18.1->ihp==0.0.6) (2025.10.4)
Requirement already satisfied: lazy-loader>=0.4 in
c:\users\si150\anaconda3\lib\site-packages (from scikit-
image->gdsfactory~=9.18.1->ihp==0.0.6) (0.4)
Building wheels for collected packages: ihp
  Building wheel for ihp (pyproject.toml): started
  Building wheel for ihp (pyproject.toml): finished with status 'done'
  Created wheel for ihp: filename=ihp-0.0.6-py3-none-any.whl size=340230
sha256=28f516961f261c3a85c589a8088ffe2da96a4cfba1c5cfc0041a5a8ed4cac3d5
  Stored in directory: C:\Users\si150\AppData\Local\Temp\pip-ephem-wheel-cache-
c1keiwrd\wheels\a9\a5\1d\15175b7fbcd5e7f3bc73d45491d51dc54d4d7243ccbed466e5
Successfully built ihp
Installing collected packages: ihp
  Attempting uninstall: ihp
    Found existing installation: ihp 0.0.6
    Uninstalling ihp-0.0.6:
      Successfully uninstalled ihp-0.0.6
Successfully installed ihp-0.0.6
```

Installation of IHP-Open-PDK:

Next, the IHP-Open-PDK itself is cloned and placed in the correct folder. The IHP-Open-PDK provides the technology files required for layout generation.

[ ]:
```
! rm -rf pdk/
! rm -rf /foss/pdks/
! mkdir -p /foss/pdks/
! git clone https://github.com/iic-jku/IHP-Open-PDK.git pdk
! cd pdk && git checkout dev && git submodule update --init --recursive
! cd pdk && mv ihp-sg13g2 /foss/pdks
```

```
Cloning into 'pdk'…
remote: Enumerating objects: 20656, done.
remote: Counting objects: 100% (2588/2588), done.
remote: Compressing objects: 100% (255/255), done.
remote: Total 20656 (delta 2414), reused 2342 (delta 2333), pack-reused 18068
(from 2)
Receiving objects: 100% (20656/20656), 409.68 MiB | 12.54 MiB/s, done.
```

```
Resolving deltas: 100% (13475/13475), done.
Updating files: 100% (4948/4948), done.
Already on 'dev'
Your branch is up to date with 'origin/dev'.
Submodule 'ihp-sg13g2/libs.tech/digital'
(https://github.com/hneemann/IHP130.git) registered for path 'ihp-
sg13g2/libs.tech/digital'
Submodule 'ihp-sg13g2/libs.tech/klayout/python/pycell4klayout-api'
(https://github.com/IHP-GmbH/pycell4klayout-api) registered for path 'ihp-
sg13g2/libs.tech/klayout/python/pycell4klayout-api'
Submodule 'ihp-sg13g2/libs.tech/klayout/python/pypreprocessor'
(https://github.com/IHP-GmbH/pypreprocessor) registered for path 'ihp-
sg13g2/libs.tech/klayout/python/pypreprocessor'
Submodule 'ihp-sg13g2/libs.tech/openems/openems_ihp_sg13g2'
(https://github.com/VolkerMuehlhaus/openems_ihp_sg13g2) registered for path
'ihp-sg13g2/libs.tech/openems/openems_ihp_sg13g2'
Submodule 'ihp-sg13g2/libs.tech/palace'
(https://github.com/VolkerMuehlhaus/gds2palace_ihp_sg13g2.git) registered for
path 'ihp-sg13g2/libs.tech/palace'
Cloning into '/foss/designs/designs/SG13G2_SPARX160/sscs-ose-code-a-
chip/pdk/ihp-sg13g2/libs.tech/digital'…
Cloning into '/foss/designs/designs/SG13G2_SPARX160/sscs-ose-code-a-
chip/pdk/ihp-sg13g2/libs.tech/klayout/python/pycell4klayout-api'…
Cloning into '/foss/designs/designs/SG13G2_SPARX160/sscs-ose-code-a-
chip/pdk/ihp-sg13g2/libs.tech/klayout/python/pypreprocessor'…
Cloning into '/foss/designs/designs/SG13G2_SPARX160/sscs-ose-code-a-
chip/pdk/ihp-sg13g2/libs.tech/openems/openems_ihp_sg13g2'…
Cloning into '/foss/designs/designs/SG13G2_SPARX160/sscs-ose-code-a-
chip/pdk/ihp-sg13g2/libs.tech/palace'…
Submodule path 'ihp-sg13g2/libs.tech/digital': checked out
'9a6e7c9d404f9f4334513a24788aa8454918d400'
Submodule path 'ihp-sg13g2/libs.tech/klayout/python/pycell4klayout-api': checked
out '4c463c43991fb0967a824906518b66047bfb33c4'
Submodule path 'ihp-sg13g2/libs.tech/klayout/python/pypreprocessor': checked out
'cf1ff9bad0fb5338cf1c5b990b2b816b1ea01a64'
Submodule path 'ihp-sg13g2/libs.tech/openems/openems_ihp_sg13g2': checked out
'e29417aa1e5fb97b4ebd27dd1e4fcf043f97ef63'
Submodule path 'ihp-sg13g2/libs.tech/palace': checked out
'bb0edb0fb73b242db3f3bb6a552a47f4c32890d4'
```

Import of Python Modules:

[15]:
```python
import gdsfactory as gf
import ihp
from pathlib import Path

import scipy
from math import sqrt
```

```python
import klayout.db as kdb

ihp.PDK.activate()
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[15], line 2
      1 import gdsfactory as gf
----> 2 import ihp
      3 from pathlib import Path
      5 import scipy

File c:\Users\si150\anaconda3\Lib\site-packages\ihp\__init__.py:11
      6 from gdsfactory.pdk import Pdk
      7 from gdsfactory.typings import (
      8     ConnectivitySpec,
      9 )
---> 11 from ihp import cells, tech
     12 from ihp.config import PATH
     14 # from ihp.models import get_models

File c:\Users\si150\anaconda3\Lib\site-packages\ihp\cells\__init__.py:3
      1 """Cells."""
----> 3 from .bondpads import *
      4 from .capacitors import *
      5 from .containers import *

File c:\Users\si150\anaconda3\Lib\site-packages\ihp\cells\bondpads.py:7
      4 sys.path.append("/foss/pdks/ihp-sg13g2/libs.tech/klayout/python")
      5 sys.path.append("/foss/pdks/ihp-sg13g2/libs.tech/klayout/python/
 ↪pycell4klayout-api/source/python/")
----> 7 from sg13g2_pycell_lib.ihp.bondpad_code import bondpad as bondpadIHP
     10 import gdsfactory as gf
     11 from typing import Literal

ModuleNotFoundError: No module named 'sg13g2_pycell_lib'
```

7 Power Detector Design based on Schottky Barrier Diodes

Circuit Operation: - **R1** provides a 50 Ω input termination. - **C1** ac-couples the RF + LO signal to the demodulating SBD **D1** and is implemented as a MIM capacitor. - **M1**–**M2** and **R2** form a transimpedance amplifier (TIA) that delivers a buffered voltage output. - **M3**–**M4**, **D2**, and **R3**–**R5** implement a replica bias. With this replica circuit, it is now possible to measure the demodulated output signal differentially. - **C2-C3** are MIM capacitor arrays which provide proper supply-rail decoupling.

Layout Considerations: - Minimize RF input parasitics (R, L, C): - Route TopMetal2 directly down to the top plate (TopMetal1) of C1with a VIA. - Use a VIA-stack from the bottom plate (Metal5) of C1 directly down to the SBD. - Apply **Metal5 no-fill regions** to reduce parasitic capacitance. - Match components of the main and replica paths. - **C2–C3** values may be adapted to layout requirements, provided they remain large enough for sufficient supply-rail decoupling.

> In the following code block, the SBD-based power detector (PD) is defined as a standalone `@gf.cell` function. This modular approach enables the same detector design to be reused across all four output channels.

[ ]:
```
@gf.cell
def powdet_sbd() -> gf.Component:
    c = gf.Component("powdet_sbd")

    # via from rfin on TM2 to C1 on TM1
    via_TM1_TM2 = ihp.cells.via_stack(
        top_layer="TopMetal2",
        bottom_layer="TopMetal1",
        vt2_columns=4,
        vt2_rows=3,
    )
    via_TM1_TM2.ports["bottom"].orientation = 0
    via_TM1_TM2_ref = c.add_ref(via_TM1_TM2)

    # reference cmim capacitor and connect to via
    cmim = ihp.cells.cmim(width=10, length=10)
    C1_ref = c.add_ref(cmim)
    C1_ref.connect("T", via_TM1_TM2_ref.ports["bottom"],␣
↪allow_width_mismatch=True)
    c.add_label(text="rfin", layer=ihp.tech.LAYER.TopMetal2text,␣
↪position=via_TM1_TM2_ref.ports["top"].center)

    # via cell from Metal2 to Metal5
    via_M2_M5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal2",
        vn_columns=6,
        vn_rows=3,
    )

    # via from CMIM C1 to XQ1
    via_M2_M5.ports["top"].orientation = 0
    via_M2_M5_ref = c.add_ref(via_M2_M5)
    via_M2_M5_ref.connect("top", C1_ref.ports["B"], allow_width_mismatch=True)

    # create D1 and connect to rfin via C1 and the vias
    schottky = ihp.cells.schottky()
    schottky.locked = False
```

```
    schottky.add_label(text="schottky_nbl1", layer=ihp.tech.LAYER.TEXTdrawing)

    D1_ref = c.add_ref(schottky)
    D1_ref.connect("e4", via_M2_M5_ref.ports["bottom"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    # c.add_ref(schottky_text).center = D1_ref.center

    # create D2 and place nearby D1 for better matching
    D2_ref = c.add_ref(schottky)
    D2_ref.ymin = D1_ref.ymin
    D2_ref.xmin = D1_ref.xmax # min spacing according to design rules 5.10
    # c.add_ref(schottky_text).center = D2_ref.center

    # create capasitor array for cap 2
    C2 = gf.Component("C2")
    cmim = ihp.cells.cmim(width=10, length=10)
    cap_connection_thickness = 4 # thickness of the metal connection between␣
↪caps, adjust as needed
    # create 3 * 4 array of cmim capacitors with column pitch of 0.52 + 11.2 to␣
↪fit the required spacing for TM1 and the size of the caps
    # with one missing cap to fit C1 in there
    spacing_between_caps = 0.52
    top_row = C2.add_ref(gf.components.array(component=cmim, columns=6, rows=1,␣
↪column_pitch= spacing_between_caps + 11.2))

    middle_row = C2.add_ref(gf.components.array(component=cmim, columns=6,␣
↪rows=1, column_pitch= spacing_between_caps + 11.2))
    bottom_row = C2.add_ref(gf.components.array(component=cmim, columns=6,␣
↪rows=3, column_pitch= spacing_between_caps + 11.2, row_pitch=␣
↪-(spacing_between_caps + 11.2)))
    top_row.ymin = middle_row.ymax + spacing_between_caps # min spacing␣
↪according to design rules 5.17
    bottom_row.ymax = middle_row.ymin - spacing_between_caps # min spacing␣
↪according to design rules 5.17
    C2.add_ports(top_row.ports, prefix="TR_")
    C2.add_ports(middle_row.ports, prefix="MR_")
    C2.add_ports(bottom_row.ports, prefix="BR_")

    C2.add_label(text="vss", layer=ihp.tech.LAYER.Metal5text, position=C2.
↪ports["BR_T_1_1"].center)
    # orient the outer most port of top and bottom row to look inward to make a␣
↪horizontal connection between each cap

    C2.ports["TR_T_1_1"].orientation = 0
    C2.ports["TR_B_1_1"].orientation = 0
    C2.ports["MR_T_1_1"].orientation = 0
```

```
C2.ports["MR_B_1_1"].orientation = 0
C2.ports["BR_T_1_1"].orientation = 0
C2.ports["BR_B_1_1"].orientation = 0
C2.ports["BR_T_2_1"].orientation = 0
C2.ports["BR_B_2_1"].orientation = 0
C2.ports["BR_T_3_1"].orientation = 0
C2.ports["BR_B_3_1"].orientation = 0

C2.ports["TR_T_1_6"].orientation = 180
C2.ports["TR_B_1_6"].orientation = 180
C2.ports["MR_T_1_6"].orientation = 180
C2.ports["MR_B_1_6"].orientation = 180
C2.ports["BR_T_1_6"].orientation = 180
C2.ports["BR_B_1_6"].orientation = 180
C2.ports["BR_T_2_6"].orientation = 180
C2.ports["BR_B_2_6"].orientation = 180
C2.ports["BR_T_3_6"].orientation = 180
C2.ports["BR_B_3_6"].orientation = 180

C2_connection_T = gf.routing.route_bundle_electrical(
    component=C2,
    ports1=[C2.ports["TR_T_1_1"],C2.ports["BR_T_2_1"],C2.ports["BR_T_3_1"],␣
↪C2.ports["MR_T_1_1"], C2.ports["BR_T_1_1"]],
    ports2=[C2.ports["TR_T_1_6"],C2.ports["BR_T_2_6"],C2.ports["BR_T_3_6"],␣
↪C2.ports["MR_T_1_6"], C2.ports["BR_T_1_6"]],
    route_width=cap_connection_thickness,
    layer=ihp.tech.LAYER.TopMetal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

C2_connection_B = gf.routing.route_bundle_electrical(
    component=C2,
    ports1=[C2.ports["TR_B_1_1"],C2.ports["BR_B_2_1"],C2.ports["BR_B_3_1"],␣
↪C2.ports["MR_B_1_1"], C2.ports["BR_B_1_1"]],
    ports2=[C2.ports["TR_B_1_6"],C2.ports["BR_B_2_6"],C2.ports["BR_B_3_6"],␣
↪C2.ports["MR_B_1_6"], C2.ports["BR_B_1_6"]],
    route_width=cap_connection_thickness,    # min width for metal1
    layer=ihp.tech.LAYER.Metal5drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
```

```
    # orient top row ports to face downwards to connect to bottom row ports
    ihp.cells.utils.change_port_orientation(C2, ["TR_T_1_1", "TR_T_1_2",␣
↪"TR_T_1_3", "TR_T_1_4", "TR_T_1_5", "TR_T_1_6"], 270)
    ihp.cells.utils.change_port_orientation(C2, ["TR_B_1_1", "TR_B_1_2",␣
↪"TR_B_1_3", "TR_B_1_4", "TR_B_1_5", "TR_B_1_6"], 270)

    # orient bottom row ports to face downwards to connect to top row ports
    ihp.cells.utils.change_port_orientation(C2, ["BR_T_3_1", "BR_T_3_2",␣
↪"BR_T_3_3", "BR_T_3_4", "BR_T_3_5", "BR_T_3_6"], 90)
    ihp.cells.utils.change_port_orientation(C2, ["BR_B_3_1", "BR_B_3_2",␣
↪"BR_B_3_3", "BR_B_3_4", "BR_B_3_5", "BR_B_3_6"], 90)

    C2_connection_top = gf.routing.route_bundle_electrical(
        component=C2,
        ports1=[C2.ports["BR_T_3_1"], C2.ports["BR_T_3_2"], C2.
↪ports["BR_T_3_3"], C2.ports["BR_T_3_4"], C2.ports["BR_T_3_5"], C2.
↪ports["BR_T_3_6"]],
        ports2=[C2.ports["TR_T_1_1"], C2.ports["TR_T_1_2"], C2.
↪ports["TR_T_1_3"], C2.ports["TR_T_1_4"], C2.ports["TR_T_1_5"], C2.
↪ports["TR_T_1_6"]],
        route_width=cap_connection_thickness,    # adjust for thicker␣
↪connections
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    C2_connection_bottom = gf.routing.route_bundle_electrical(
        component=C2,
        ports1=[C2.ports["BR_B_3_1"], C2.ports["BR_B_3_2"], C2.
↪ports["BR_B_3_3"], C2.ports["BR_B_3_4"], C2.ports["BR_B_3_5"], C2.
↪ports["BR_B_3_6"]],
        ports2=[C2.ports["TR_B_1_1"], C2.ports["TR_B_1_2"], C2.
↪ports["TR_B_1_3"], C2.ports["TR_B_1_4"], C2.ports["TR_B_1_5"], C2.
↪ports["TR_B_1_6"]],
        route_width=cap_connection_thickness,    # adjust for thicker␣
↪connections
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```
    # orient top row ports to face upwards to connect the ground plates to the␣
↪other caps
    ihp.cells.utils.change_port_orientation(C2, ["TR_B_1_1", "TR_B_1_2",␣
↪"TR_B_1_3", "TR_B_1_6"], 90)
    ihp.cells.utils.change_port_orientation(C2, ["TR_T_1_6"], 0) # for␣
↪connection to bias1
    ihp.cells.utils.change_port_orientation(C2, ["BR_B_3_4", "BR_B_3_5"], 270)

    C2_ref = c.add_ref(C2)
    C2_ref.rotate(-90)
    C2_ref.center = C1_ref.center
    C2_ref.ymin = C1_ref.ymax + 10
    C2_ref.xmax = C1_ref.xmax + C1_ref.xsize + 0.52
    ## end of C2 generation

    r_sil = ihp.cells.rsil(
        width=0.5,
        length=2.5)

    # let the ports of XR1 and XR5 face each other for easier routing
    r_sil.ports["e2"].orientation = 0
    XR1_ref = c.add_ref(r_sil.copy())

    r_sil.ports["e2"].orientation = 180
    XR5 = r_sil.copy()
    XR5_ref = c.add_ref(XR5)

    # spacing between D1 and XR1/XR5
    spacing = 1
    XR1_ref.xmax = D1_ref.xmax - 1.88
    XR1_ref.ymax = D1_ref.ymin - spacing

    XR5_ref.xmin = D2_ref.xmin + 1.88
    XR5_ref.ymin = XR1_ref.ymin

    route_XR1_XR5 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[XR1_ref.ports["e2"]],
        ports2=[XR5_ref.ports["e2"]],
        route_width=0.26,    # min width for metal1
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```python
# via for connection D1 to XR1 and XR5 to D2
via_M1_M2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=6,
    vn_rows=2,
)

# via D1 -> XR1
via_M1_M2.ports["top"].orientation = 90
via_M1_M2.ports["bottom"].orientation = 0
via_M1_M2_ref = c.add_ref(via_M1_M2.copy())
via_M1_M2_ref.center = D1_ref.center
via_M1_M2_ref.ymax = D1_ref.ymin

# route from D1 to via
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[D1_ref.ports["e4"]],
    ports2=[via_M1_M2_ref.ports["top"]],
    route_width=max(via_M1_M2_ref.xsize, via_M1_M2_ref.ysize),
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# route from via to XR1
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[via_M1_M2_ref.ports["bottom"]],
    ports2=[XR1_ref.ports["e1"]],
    route_width=min(via_M1_M2_ref.xsize, via_M1_M2_ref.ysize),
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# connect second diode to input resistors
# via D2 -> XR5
via_M1_M2.ports["top"].orientation = 90
via_M1_M2.ports["bottom"].orientation = 180
via_M1_M2_ref = c.add_ref(via_M1_M2.copy())
via_M1_M2_ref.center = D2_ref.center
```

```
via_M1_M2_ref.ymax = D2_ref.ymin

# route from D2 to via
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[D2_ref.ports["e4"]],
    ports2=[via_M1_M2_ref.ports["top"]],
    route_width=max(via_M1_M2_ref.xsize, via_M1_M2_ref.ysize),
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# route from via to XR5
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[via_M1_M2_ref.ports["bottom"]],
    ports2=[XR5_ref.ports["e1"]],
    route_width=min(via_M1_M2_ref.xsize, via_M1_M2_ref.ysize),
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

## build transimpedance amplifier with the schottky diode as input device
# M1 nmos and M2 pmos each have 10 gates, M3 and M4 have one gate each, all␣
↪nmos are placed together and so are the pmos

ng_mos = 13 # 10 + 1 = 11, plus 2 dummy transistors for better matching
ng_mos = 24 # 20 + 2 + 2 = 24, M1 + M3 + 2 dummy for nmos, M2 + M4 + 2␣
↪dummy for pmos
width_nmos = 60 # um
width_pmos = 120

c_output = gf.Component("output_stage")

M1_3 = ihp.cells.nmos(
    w=width_nmos,
    l=0.13,
    ng=ng_mos,
    guardRingType="psub",
    guardRingDistance=2,
)
```

```
via_gat_M1 = ihp.cells.via_stack(
        top_layer="Metal1",
        bottom_layer="GatPoly",
        vn_columns=1,
        vn_rows=2,
    )

via_gat_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="GatPoly",
        vn_columns=1,
        vn_rows=3,
    )

via_gat_M3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="GatPoly",
        vn_columns=1,
        vn_rows=3,
    )

M1_3.locked=False
via_gat_M1.ports["bottom"].orientation = 90
via_gat_M2.ports["bottom"].orientation = 90
via_gat_M3.ports["bottom"].orientation = 90
via_gat_M3.ports["top"].orientation = 270 # turn top port for later␣
↪connection
gate_ports = M1_3.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
gate_straight = ihp.cells.straight(
    length= 2.15,
    cross_section="gatpoly_routing",
    width=0.13 # manual meassure of the heigth of the via
)
gate_via_refs_nmos = []
for i, p in enumerate(gate_ports):
    if p.name in ["G_1", "G_24"]: # these gates will be connected to metal1
        gate_via_refs_nmos.append(M1_3.add_ref(via_gat_M2))
    elif p.name in ["G_12", "G_13"]: # these gates will be connected to␣
↪metal2
        gate_via_refs_nmos.append(M1_3.add_ref(via_gat_M2))
    else: # all others to metal3
        gate_via_refs_nmos.append(M1_3.add_ref(via_gat_M3))
    gate_straight_ref = M1_3.add_ref(gate_straight)
    gate_straight_ref.connect("e1", M1_3.ports[p.name],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    via_gat_M2_ref = gate_via_refs_nmos[-1]
```

```
        via_gat_M2_ref.connect("bottom", gate_straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect gates 1 and 24 with metal1 to ground
    via_gat_M2.ports["top"].orientation = 0
    start_point = gate_via_refs_nmos[0].ports["top"].center[0]
    end_point = M1_3.ports["DS_29"].center[0]
    straight_ref = M1_3.add_ref(ihp.cells.straight(
        length=abs(end_point-start_point),
        cross_section="metal2_routing",
        width=1.11 # manual meassure of the heigth of the via
        ))
    straight_ref.connect("e1", gate_via_refs_nmos[0].ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=1,
    )
    via_M1_M2_ref = M1_3.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("top", straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    via_gat_M2.ports["top"].orientation = 180
    straight_ref = M1_3.add_ref(ihp.cells.straight(
        length=abs(end_point-start_point),
        cross_section="metal2_routing",
        width=1.11 # manual meassure of the heigth of the via
        ))
    straight_ref.connect("e1", gate_via_refs_nmos[23].ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    via_M1_M2_ref = M1_3.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("top", straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect Gates 2-11 and 14-23 (all 20 gates) together with metal3
    start_point = M1_3.ports["G_2"].center
    end_point = M1_3.ports["G_23"].center
    straight_ref = M1_3.add_ref(ihp.cells.straight(
        length=abs(end_point[0]-start_point[0]),
        cross_section="metal3_routing",
        width=max(gate_via_refs_nmos[1].xsize, gate_via_refs_nmos[1].ysize) #␣
↪manual meassure of the heigth of the via
        ))
```

```
    straight_ref.center = ((start_point[0]+end_point[0])/2,␣
↪gate_via_refs_nmos[1].center[1]) # align with the via of the first M1 gate

    # connect Gates 12-13 (2 gates) together with metal2
    start_point = M1_3.ports["G_12"].center
    end_point = M1_3.ports["G_13"].center
    straight_ref = M1_3.add_ref(ihp.cells.straight(
        length=abs(end_point[0]-start_point[0]),
        cross_section="metal2_routing",
        width=0.6 # manual meassure of the heigth of the via
        ))
    straight_ref.center = ((start_point[0]+end_point[0])/2,␣
↪gate_via_refs_nmos[11].center[1]) # align with the via of the first M3 gate

    # connect Gates 14-23 (10 gates) together with metal1
    # start_point = M1_3.ports["G_14"].center
    # end_point = M1_3.ports["G_23"].center
    # straight_ref = M1_3.add_ref(ihp.cells.straight(
    #     length=abs(end_point[0]-start_point[0]),
    #     cross_section="metal2_routing",
    #     width=0.6 # manual meassure of the heigth of the via
    #     ))
    # straight_ref.center = ((start_point[0]+end_point[0])/2,␣
↪gate_via_refs_nmos[13].center[1]) # align with the via of the 13th M1 gate

    # connect source to GND
    M1_3_ds_ports = M1_3.get_ports_list(layer=ihp.tech.LAYER.Metal1drawing)
    M1_3_source_ports = []
    M1_3_drain_ports = []
    for i in range(3, len(M1_3_ds_ports)-1): # filter out unwanted ports
        # print(M1_3_ds_ports[i].name)
        if i == 3 or i == len(M1_3_ds_ports)-2 or i % 2 == 0:
            M1_3_source_ports.append(M1_3_ds_ports[i])
        else:
            M1_3_drain_ports.append(M1_3_ds_ports[i])

    # source ground connection
    source_straight = ihp.cells.straight(
        length= abs(M1_3.ports["DS_1"].center[1] - M1_3.ports["DS_27"].
↪center[1]), # any source port to the the center of the top guardring segment
        cross_section="metal2_routing",
        width=0.26 # manual meassure of the heigth of the via
    )
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=7,
```

```
        vn_rows=1,
    )
    via_M1_M2.ports["bottom"].orientation = 0
    via_M1_M2.ports["top"].orientation = 180
    for p in M1_3_source_ports:
        p.orientation = 90 # orient ports to face the top
        via_M1_M2_ref = M1_3.add_ref(via_M1_M2)
        via_M1_M2_ref.connect("bottom", p, allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
        source_straight_ref = M1_3.add_ref(source_straight)
        source_straight_ref.connect("e1", via_M1_M2_ref.ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect drains to M3
    via_M1_M4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=7,
    )

    drain_via_refs_nmos = []
    for p in M1_3_drain_ports:
        if p.name in ["DS_13"]:
            via_M1_M4_short = ihp.cells.via_stack(
                top_layer="Metal4",
                bottom_layer="Metal1",
                vn_columns=1,
                vn_rows=3,
            )
            via_M1_M4_short.ports["top"].orientation = 270
            drain_via_refs_nmos.append(M1_3.add_ref(via_M1_M4_short))
            drain_via_refs_nmos[-1].center = p.center
            drain_via_refs_nmos[-1].ymax = p.center[1] -0.245 # manual␣
↪meassurement to align with metal1 below
        else:
            via_M1_M4.ports["bottom"].orientation = 90
            via_M1_M4.ports["top"].orientation = 90
            via_M1_M4_ref = M1_3.add_ref(via_M1_M4)
            via_M1_M4_ref.connect("bottom", p, allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
            drain_via_refs_nmos.append(via_M1_M4_ref)

    # horizontal drain connection for nmos
    straight_drain_connection = ihp.cells.straight(
        length= abs(drain_via_refs_nmos[0].center[0] - drain_via_refs_nmos[-1].
↪center[0]), # any drain via to the the center of the top guardring segment
```

```
        cross_section="metal4_routing",
        width=2 # manual meassure of the heigth of the via
    )
    straight_drain_connection_ref = M1_3.add_ref(straight_drain_connection)
    straight_drain_connection_ref.xmin = drain_via_refs_nmos[-1].center[0]
    straight_drain_connection_ref.ymin = drain_via_refs_nmos[-1].center[1]

    M1_3.add_ports(gate_via_refs_nmos[17].ports, prefix="gate_M1_")
    M1_3.add_port(name="drain_connection_M1", port=drain_via_refs_nmos[2].
↪ports["top"])
    M1_3.add_port(name="drain_connection_M3", port=drain_via_refs_nmos[5].
↪ports["top"])
    M1_3.add_port(name="gate_connection_M3", center=((gate_via_refs_nmos[11].
↪center[0] + gate_via_refs_nmos[12].center[0]) /2, gate_via_refs_nmos[11].
↪center[1]), cross_section="metal2_routing", orientation=270,␣
↪port_type="electrical")
    M1_3_ref = c_output.add_ref(M1_3)
    M1_3_ref.move((0,30))
    # c_output.add(gate_via_refs_nmos)
    # c_output.add_port(name="drain_connection_M1", port=drain_via_refs_nmos[2].
↪ports["top"])

    M2_4 = ihp.cells.pmos(
        w=width_pmos,
        l=0.13,
        ng=ng_mos,
        guardRingType="nwell",
        guardRingDistance=2,
    )

    # connect gates of M2 and M4 together
    M2_4.locked=False
    gate_ports = M2_4.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
    gate_straight = ihp.cells.straight(
        length= 3.55,
        cross_section="gatpoly_routing",
        width=0.13 # manual meassure of the heigth of the via
    )
    gate_via_refs_pmos = []
    for i, p in enumerate(gate_ports):
        p.orientation += 180 # orient gates to face downwards for easier␣
↪connection to nmos gates
        if p.name in ["G_1", "G_25"]: # these gates will be connected to metal1
            gate_via_refs_pmos.append(M2_4.add_ref(via_gat_M2))
        elif p.name in ["G_13", "G_14"]: # these gates will be connected to␣
↪metal2
```

```
            gate_via_refs_pmos.append(M2_4.add_ref(via_gat_M2))
        else: # all others to metal3
            gate_via_refs_pmos.append(M2_4.add_ref(via_gat_M3))
        gate_straight_ref = M2_4.add_ref(gate_straight)
        gate_straight_ref.connect("e1", M2_4.ports[p.name],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
        gate_via_refs_pmos[-1].connect("bottom", gate_straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)


    ## Gate numbers are shifted by 1 after G1, e.g. G1 G3 G4 G5 .. dont know␣
↪why... similar for DS, DS1 DS3 DS5 DS6 DS7 ... maybe guardrings are␣
↪intereering even though they are disabled
    # connect gates 1 and 24 with metal1 to ground
    via_gat_M2.ports["top"].orientation = 180
    start_point = gate_via_refs_pmos[0].ports["top"].center[0]
    end_point = M2_4.ports["DS_31"].center[0]
    straight_ref = M2_4.add_ref(ihp.cells.straight(
        length=abs(end_point-start_point),
        cross_section="metal2_routing",
        width=1.11 # manual meassure of the heigth of the via
        ))

    straight_ref.connect("e1", gate_via_refs_pmos[0].ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=1,
    )
    via_M1_M2_ref = M2_4.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("top", straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_ref.connect("e2", via_M1_M2_ref.ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    via_gat_M2.ports["bottom"].orientation = 0
    straight_ref.connect("e1", gate_via_refs_pmos[0].ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    straight_ref = M2_4.add_ref(ihp.cells.straight(
        length=abs(end_point-start_point),
        cross_section="metal2_routing",
        width=1.11 # manual meassure of the heigth of the via
        ))
```

```
    straight_ref.connect("e1", gate_via_refs_pmos[23].ports["bottom"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    via_M1_M2_ref = M2_4.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("top", straight_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect Gates 2-11 and 14-23 (20 gates) together with metal3
    start_point = M2_4.ports["G_3"].center
    end_point = M2_4.ports["G_24"].center
    straight_ref = M2_4.add_ref(ihp.cells.straight(
        length=abs(end_point[0]-start_point[0]),
        cross_section="metal3_routing",
        width=max(gate_via_refs_pmos[1].xsize, gate_via_refs_pmos[1].ysize) #␣
↪manual meassure of the heigth of the via
        ))
    straight_ref.center = ((start_point[0]+end_point[0])/2,␣
↪gate_via_refs_pmos[1].center[1]) # align with the via of the first M1 gate

    # connect Gates 12-13 (2 gates) together with metal2
    start_point = M2_4.ports["G_13"].center
    end_point = M2_4.ports["G_14"].center
    straight_ref = M2_4.add_ref(ihp.cells.straight(
        length=abs(end_point[0]-start_point[0]),
        cross_section="metal2_routing",
        width=0.6 # manual meassure of the heigth of the via
        ))
    straight_ref.center = ((start_point[0]+end_point[0])/2,␣
↪gate_via_refs_pmos[11].center[1]) # align with the via of the first M3 gate

    # connect source to vdd
    M2_4_ds_ports = M2_4.get_ports_list(layer=ihp.tech.LAYER.Metal1drawing)
    M2_4_source_ports = []
    M2_4_drain_ports = []
    for i in range(3, len(M2_4_ds_ports)-1): # filter out unwanted ports
        # print(M2_4_ds_ports[i].name)
        if i == 3 or i == len(M2_4_ds_ports)-2 or i % 2 == 0:
            M2_4_source_ports.append(M2_4_ds_ports[i])
        else:
            M2_4_drain_ports.append(M2_4_ds_ports[i])

    # source vdd connection
    source_straight = ihp.cells.straight(
        length= abs(M2_4.ports["DS_1"].center[1] - M2_4.ports["DS_29"].
↪center[1]), # any source port to the the center of the top guardring segment
        cross_section="metal2_routing",
```

```
        width=0.26 # manual meassure of the heigth of the via
    )
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=13,
        vn_rows=1,
    )
    via_M1_M2.ports["bottom"].orientation = 180
    via_M1_M2.ports["top"].orientation = 0
    for p in M2_4_source_ports:
        p.orientation = 270 # orient ports to face the top
        via_M1_M2_ref = M2_4.add_ref(via_M1_M2)
        via_M1_M2_ref.connect("bottom", p, allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
        source_straight_ref = M2_4.add_ref(source_straight)
        source_straight_ref.connect("e1", via_M1_M2_ref.ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect drains to metal4
    via_M1_M4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=13,
    )
    drain_via_refs_pmos = []
    for p in M2_4_drain_ports:
        if p.name in ["DS_15"]:
            via_M1_M4_short = ihp.cells.via_stack(
                top_layer="Metal4",
                bottom_layer="Metal1",
                vn_columns=1,
                vn_rows=6,
            )
            via_M1_M4_short.ports["top"].orientation = 90
            drain_via_refs_pmos.append(M2_4.add_ref(via_M1_M4_short))
            drain_via_refs_pmos[-1].center = p.center
            drain_via_refs_pmos[-1].ymin = p.center[1] + 0.245 # manual␣
↪meassurement to align with metal1 below
        else:
            via_M1_M4.ports["bottom"].orientation = 90
            via_M1_M4.ports["top"].orientation = 270
            via_M1_M4_ref = M2_4.add_ref(via_M1_M4)
            via_M1_M4_ref.connect("bottom", p, allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
            drain_via_refs_pmos.append(via_M1_M4_ref)
```

```
    # horizontal drain connection for nmos
    straight_drain_connection = ihp.cells.straight(
        length= abs(drain_via_refs_pmos[0].center[0] - drain_via_refs_pmos[-1].
↪center[0]), # any drain via to the the center of the top guardring segment
        cross_section="metal4_routing",
        width=2.61 #  manual measurement
    )
    straight_drain_connection_ref = M2_4.add_ref(straight_drain_connection)
    straight_drain_connection_ref.xmin = drain_via_refs_pmos[-1].center[0]
    straight_drain_connection_ref.ymin = drain_via_refs_pmos[-1].ymin

    M2_4.add_ports(gate_via_refs_pmos[17].ports, prefix="gate_M2_")
    M2_4.add_port(name="drain_connection_M2", port=drain_via_refs_pmos[2].
↪ports["top"])
    M2_4.add_port(name="drain_connection_M4", port=drain_via_refs_pmos[5].
↪ports["top"])
    M2_4.add_port(name="gate_connection_M4", center=((gate_via_refs_pmos[11].
↪center[0] + gate_via_refs_pmos[12].center[0]) /2, gate_via_refs_pmos[11].
↪center[1]), cross_section="metal2_routing", orientation=90,␣
↪port_type="electrical")

    M2_4_ref = c_output.add_ref(M2_4)

    spacing_between_nmos_pmos = 15 # manual meassurement of the spacing between␣
↪the drain vias of M1 and M2, used to set the length of the connection␣
↪between M1 and M2 drains
    drain_connection_M1_M2 = c_output.add_ref(ihp.cells.straight(
        length= spacing_between_nmos_pmos, # manual measurement, used to set␣
↪the distance between M1 and M2
        cross_section="metal4_routing",
        width=4.28 #  manual measurement
    ))

    drain_connection_M1_M2.connect("e1", M1_3_ref.ports["drain_connection_M1"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    M2_4_ref.connect("drain_connection_M2", drain_connection_M1_M2.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # drain connection for replica circuit
    routing_drain_M1_M3 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[M1_3_ref.ports["drain_connection_M3"]],
        ports2=[M2_4_ref.ports["drain_connection_M4"]],
        route_width=0.61,
        layer=ihp.tech.LAYER.Metal4drawing,
```

```
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # vias for resistor XR2 and XR4 connections
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
    )
    via_M1_M2.ports["top"].orientation = 90
    via_M1_M3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
    )
    via_M1_M4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
    )

    XR4 = ihp.cells.rppd(width=0.5, length=1.5)
    XR4_ref = c_output.add_ref(XR4).rotate(90)
    XR4_ref.center = M1_3_ref.ports["DS_15"].center
    XR4_ref.movex(0.01) # manual meassurement to align with the gate connection
    XR4_ref.ymax =  M1_3_ref.ymin - 2 # manual meassurement to set the spacing␣
↪between the nmos and the resistors

    via_M1_M4_ref = c_output.add_ref(via_M1_M4)
    via_M1_M4_ref.connect("bottom", XR4_ref.ports["e1"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    c_output.add_label(text="vref", layer=ihp.tech.LAYER.Metal4text,␣
↪position=via_M1_M4_ref.ports["top"].center)
    c_output.add_port(name="vref", port=via_M1_M4_ref.ports["top"])

    via_M1_M2.ports["top"].orientation = 180
    via_M1_M2_ref = c_output.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("bottom", XR4_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    c_output.add_ports(via_M1_M2_ref.ports, prefix="XR4_")
```

```
    XR2 = ihp.cells.rppd(width=0.5, length=1.5)
    XR2_ref = c_output.add_ref(XR2).rotate(90)
    XR2_ref.ymax = XR4_ref.ymin -1 # manual meassurement to set the spacing␣
↪between the two resistors to 0
    XR2_ref.xmax = XR4_ref.xmax

    via_M1_M4_ref = c_output.add_ref(via_M1_M4)
    via_M1_M4_ref.connect("bottom", XR2_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    c_output.add_label(text="vout", layer=ihp.tech.LAYER.Metal4text,␣
↪position=via_M1_M4_ref.ports["top"].center)
    c_output.add_port(name="vout", port=via_M1_M4_ref.ports["top"])
    via_M1_M3_ref = c_output.add_ref(via_M1_M3)
    via_M1_M3_ref.connect("bottom", XR2_ref.ports["e1"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # bias2
    route_gates_M3_M4 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[M1_3_ref.ports["gate_connection_M3"]],
        ports2=[via_M1_M2_ref.ports["top"]],
        route_width=0.5,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
        steps=[
            {"dy": -2.46, "dx": 0.5},
            {"dx": 1},
        ]
    )

    # node bb_int routing
    route_gates_M1_M2 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[M1_3_ref.ports["gate_M1_top"]],
        ports2=[M2_4_ref.ports["gate_M2_top"]],
        route_width=0.6,
        layer=ihp.tech.LAYER.Metal3drawing,
        start_straight_length=0.5,
        end_straight_length=0.5,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
```

```
    )
    route_gates_M1_M2 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[M1_3_ref.ports["gate_M1_top"]],
        ports2=[XR2_ref.ports["e1"]],
        route_width=0.6,
        layer=ihp.tech.LAYER.Metal3drawing,
        start_straight_length=0.5,
        end_straight_length=0.5,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c_output.add_ports(M1_3_ref.ports, prefix="M1_")
    c_output.add_ports(M2_4_ref.ports, prefix="M2_")
    output_stage_ref = c.add_ref(c_output.rotate(90))
    output_stage_ref.center = C2_ref.center
    output_stage_ref.ymin = C2_ref.ymin
    output_stage_ref.move((17.99 + 1.64, 9.02 + 5.23))

    # create capasitor array for cap 3
    C3 = gf.Component("C3")
    # create 3 * 4 array of cmim capacitors with column pitch of 0.52 + 11.2 to␣
↪fit the required spacing for TM1 and the size of the caps
    # with one missing cap to fit C1 in there
    spacing_between_caps = 0.52
    left_col = C3.add_ref(gf.components.array(component=cmim, columns=3,␣
↪rows=5, column_pitch= spacing_between_caps + 11.2, row_pitch=␣
↪-(spacing_between_caps + 11.2)))
    middle_col = C3.add_ref(gf.components.array(component=cmim, columns=2,␣
↪rows=4, column_pitch= spacing_between_caps + 11.2, row_pitch=␣
↪-(spacing_between_caps + 11.2)))
    right_col = C3.add_ref(gf.components.array(component=cmim, columns=1,␣
↪rows=5, column_pitch= spacing_between_caps + 11.2, row_pitch=␣
↪-(spacing_between_caps + 11.2)))
    left_col.xmax = middle_col.xmin - spacing_between_caps # min spacing␣
↪according to design rules 5.17
    right_col.xmin = middle_col.xmax + spacing_between_caps # min spacing␣
↪according to design rules 5.17
    C3.add_ports(left_col.ports, prefix="LC_")
    C3.add_ports(middle_col.ports, prefix="MC_")
    C3.add_ports(right_col.ports, prefix="RC_")

    # orient the outer most port of for horizontal connection
```

```
    ihp.cells.utils.change_port_orientation(C3, ["LC_T_1_1", "LC_T_2_1",␣
↪"LC_T_3_1", "LC_T_4_1", "LC_T_5_1"], 0)
    ihp.cells.utils.change_port_orientation(C3, ["LC_B_1_1", "LC_B_2_1",␣
↪"LC_B_3_1", "LC_B_4_1", "LC_B_5_1"], 0)

    ihp.cells.utils.change_port_orientation(C3, ["RC_T_1_1", "RC_T_2_1",␣
↪"RC_T_3_1", "RC_T_4_1", "LC_T_5_3"], 180)
    ihp.cells.utils.change_port_orientation(C3, ["RC_B_1_1", "RC_B_2_1",␣
↪"RC_B_3_1", "RC_B_4_1", "LC_B_5_3"], 180)

    C3_connection_T = gf.routing.route_bundle_electrical(
        component=C3,
        ports1=[C3.ports["LC_T_1_1"], C3.ports["LC_T_2_1"], C3.
↪ports["LC_T_3_1"], C3.ports["LC_T_4_1"], C3.ports["LC_T_5_1"]],
        ports2=[C3.ports["RC_T_1_1"], C3.ports["RC_T_2_1"], C3.
↪ports["RC_T_3_1"], C3.ports["RC_T_4_1"], C3.ports["LC_T_5_3"]],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    C3_connection_B = gf.routing.route_bundle_electrical(
        component=C3,
        ports1=[C3.ports["LC_B_1_1"], C3.ports["LC_B_2_1"], C3.
↪ports["LC_B_3_1"], C3.ports["LC_B_4_1"], C3.ports["LC_B_5_1"]],
        ports2=[C3.ports["RC_B_1_1"], C3.ports["RC_B_2_1"], C3.
↪ports["RC_B_3_1"], C3.ports["RC_B_4_1"], C3.ports["LC_B_5_3"]],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to bottom row ports
    ihp.cells.utils.change_port_orientation(C3, ["LC_T_1_1", "LC_T_1_2",␣
↪"LC_T_1_3", "MC_T_1_1", "MC_T_1_2", "RC_T_1_1"], 270)
    ihp.cells.utils.change_port_orientation(C3, ["LC_B_1_1", "LC_B_1_2",␣
↪"LC_B_1_3", "MC_B_1_1", "MC_B_1_2", "RC_B_1_1"], 270)

    # orient bottom row ports to face downwards to connect to top row ports
```

```
    ihp.cells.utils.change_port_orientation(C3, ["LC_T_5_1", "LC_T_5_2",␣
↪"LC_T_5_3", "MC_T_4_1", "MC_T_4_2", "RC_T_5_1"], 90)
    ihp.cells.utils.change_port_orientation(C3, ["LC_B_5_1", "LC_B_5_2",␣
↪"LC_B_5_3", "MC_B_4_1", "MC_B_4_2", "RC_B_5_1"], 90)

    C3_connection_top = gf.routing.route_bundle_electrical(
        component=C3,
        ports1=[C3.ports["LC_T_1_1"], C3.ports["LC_T_1_2"], C3.
↪ports["LC_T_1_3"], C3.ports["MC_T_1_1"], C3.ports["MC_T_1_2"], C3.
↪ports["RC_T_1_1"]],
        ports2=[C3.ports["LC_T_5_1"], C3.ports["LC_T_5_2"], C3.
↪ports["LC_T_5_3"], C3.ports["MC_T_4_1"], C3.ports["MC_T_4_2"], C3.
↪ports["RC_T_5_1"]],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    C3_connection_bottom = gf.routing.route_bundle_electrical(
        component=C3,
        ports1=[C3.ports["LC_B_1_1"], C3.ports["LC_B_1_2"], C3.
↪ports["LC_B_1_3"], C3.ports["MC_B_1_1"], C3.ports["MC_B_1_2"], C3.
↪ports["RC_B_1_1"]],
        ports2=[C3.ports["LC_B_5_1"], C3.ports["LC_B_5_2"], C3.
↪ports["LC_B_5_3"], C3.ports["MC_B_4_1"], C3.ports["MC_B_4_2"], C3.
↪ports["RC_B_5_1"]],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to face the middle for␣
↪connection purposes
    ihp.cells.utils.change_port_orientation(C3, ["MC_T_4_1", "MC_T_4_2",␣
↪"LC_T_5_1", "LC_T_5_2", "LC_T_5_3"], 270)
    # orient top row ports to face downwards to connect to face C2 for␣
↪connection purposes
    ihp.cells.utils.change_port_orientation(C3, ["MC_B_4_1", "MC_B_4_2",␣
↪"LC_B_5_1", "LC_B_5_2", "LC_B_5_3", "RC_B_5_1"], 270)
```

```
    C3.add_label(text="vdd", layer=ihp.tech.LAYER.TopMetal1text, position=C3.
↪ports["LC_T_1_1"].center)
    C3_ref = c.add_ref(C3)

    C3_ref.rotate(-90)
    C3_ref.xmin = C2_ref.xmax + 0.52 # manual meassurement to set the spacing␣
↪between C2 and C3
    C3_ref.ymin = C2_ref.ymin

    C_fill = ihp.cells.cmim(width=6.8, length=9)
    C_fill.ports["B"].orientation = 0
    C_fill_1_ref = c.add_ref(C_fill)
    C_fill_2_ref = c.add_ref(C_fill)

    straight_M5 = ihp.cells.straight(
        length=10.12,
        cross_section="metal5_routing",
        width=cap_connection_thickness, # manual meassurement to align with the␣
↪via
    )
    straight_M5_ref = c.add_ref(straight_M5)
    straight_M5_ref.connect("e1", C3_ref.ports["MC_B_4_1"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    C_fill_1_ref.connect("B", straight_M5_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_M5_ref = c.add_ref(straight_M5)
    straight_M5_ref.connect("e1", C3_ref.ports["MC_B_4_2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    C_fill_2_ref.connect("B", straight_M5_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    route_C2_C3_VSS = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[C2_ref.ports["TR_B_1_1"], C2_ref.ports["TR_B_1_2"], C2_ref.
↪ports["TR_B_1_3"], C2_ref.ports["TR_B_1_6"]],
        ports2=[C3_ref.ports["LC_B_5_1"], C3_ref.ports["LC_B_5_2"], C3_ref.
↪ports["LC_B_5_3"], C3_ref.ports["RC_B_5_1"]],
        route_width=cap_connection_thickness,    # adjust for thicker␣
↪connections
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```
    # connect nmos source/bulk to vss
    via_M1_M5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=35,
    )
    via_M1_M5.ports["bottom"].orientation = 0
    via_M1_M5_ref = c.add_ref(via_M1_M5)
    via_M1_M5_ref.connect("bottom", output_stage_ref.ports["M1_DS_27"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect nmos source/bulk to vdd
    via_M1_TM1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=35,
        vt1_columns=1,
        vt1_rows=17,
    )
    via_M1_TM1.ports["bottom"].orientation = 90
    via_M1_TM1_ref = c.add_ref(via_M1_TM1)
    via_M1_TM1_ref.connect("bottom", output_stage_ref.ports["M2_DS_28"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    tm1_straight = ihp.cells.straight(
        length=abs(output_stage_ref.ports["M2_DS_28"].center[0] - C3_ref.
↪ports["MC_T_4_1"].center[0]),
        cross_section="topmetal1_routing",
        width=2 # manual meassurement to align with the via
    )
    tm1_straight_ref = c.add_ref(tm1_straight)
    tm1_straight_ref.connect("e1", C3_ref.ports["MC_T_4_1"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    tm1_straight_ref = c.add_ref(tm1_straight)
    tm1_straight_ref.connect("e1", C3_ref.ports["MC_T_4_2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect C2 to bias1 network
    via_M1_TM1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=4,
        vt1_columns=1,
```

```
        vt1_rows=2,
        )
    via_M1_TM1.ports["top"].orientation = 0
    via_M1_TM1.ports["bottom"].orientation = 180
    via_M1_TM1_ref = c.add_ref(via_M1_TM1).rotate(90)
    via_M1_TM1_ref.xmax = C2_ref.xmax - 4.55
    via_M1_TM1_ref.ymax = C3_ref.ymin - 5 # min M5 spacing

    route_C2_bias = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[C2_ref.ports["TR_T_1_6"]],
        ports2=[via_M1_TM1_ref.ports["top"]],
        route_width=max(via_M1_TM1_ref.xsize, via_M1_TM1_ref.ysize),    #␣
↪manual change to fit width
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    route = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_M1_TM1_ref.ports["bottom"]],
        ports2=[XR1_ref.ports["e2"]],
        route_width=0.46,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_M1_TM1.ports["bottom"].orientation = 270
    XR3 = ihp.cells.rhigh(width=0.5, length=2)
    XR3_ref = c.add_ref(XR3).rotate(90)
    XR3_ref.center = via_M1_TM1_ref.center
    XR3_ref.xmin = C3_ref.xmin
    rotute_XR3 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_M1_TM1_ref.ports["bottom"]],
        ports2=[XR3_ref.ports["e1"]],
        route_width=0.46,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
```

```
    )
    via_M1_TM1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=4,
        vn_rows=3,
        vt1_columns=2,
        vt1_rows=1,
        )
    via_M1_TM1.ports["bottom"].orientation = 0
    via_M1_TM1_ref = c.add_ref(via_M1_TM1)
    via_M1_TM1_ref.connect("bottom", XR3_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    route_C2_bias = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[C3_ref.ports["RC_T_5_1"]],
        ports2=[via_M1_TM1_ref.ports["top"]],
        route_width=max(via_M1_TM1_ref.xsize, via_M1_TM1_ref.ysize),    #␣
↪manual change to fit width
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # bb_int connect D1 to network
    via_M1_M3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=6,
        vn_rows=3,
    )
    via_M1_M3.ports["top"].orientation = 90
    via_M1_M3.ports["bottom"].orientation = 270
    via_M1_M3_ref = c.add_ref(via_M1_M3)
    via_M1_M3_ref.connect("bottom", D1_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # bb_int
    routing_bb_int = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M1_gate_M1_top"]],
        ports2=[via_M1_M3_ref.ports["top"]],
        route_width=0.6,
        layer=ihp.tech.LAYER.Metal3drawing,
```

```
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=2,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": 3.65, "dy": -1}, # strange behavior without the dy direction
            {"dy": -6},
            {"dx": -4, "dy": 0},
        ],
        # layer_marker=(41, 0), # Mark the steps with a layer
    )

    # bias2 connect D2 to network
    via_M1_M2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=6,
        vn_rows=3,
    )
    via_M1_M2.ports["top"].orientation = 90
    via_M1_M2.ports["bottom"].orientation = 270
    via_M1_M2_ref = c.add_ref(via_M1_M2)
    via_M1_M2_ref.connect("bottom", D2_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    # bias2 connection from gate of M3 to D2
    routing_bias2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M1_gate_connection_M3"]],
        ports2=[via_M1_M2_ref.ports["top"]],
        route_width=0.5,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": 2.46, "dy": -1}, # strange behavior without the dy direction
            {"dy": -1},
            {"dx": 2.17, "dy": 0},
            {"dy": -2},
        ],
        # layer_marker=(41, 0), # Mark the steps with a layer
    )
```

```
    # bias2 connection from gate of M4 to D2
    routing_bias2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M2_gate_connection_M4"]],
        ports2=[via_M1_M2_ref.ports["top"]],
        route_width=0.5,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": -2.46, "dy": -1}, # strange behavior without the dy direction
            {"dy": -1},
            {"dx": -2.21, "dy": 0},
            {"dy": -2},
        ],
        # layer_marker=(41, 0), # Mark the steps with a layer
    )

    # SBD D1 D2 sub connection
    route_D1_D2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[D1_ref.ports["e3"]],
        ports2=[D2_ref.ports["e1"]],
        route_width=5.7,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_M1_M3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=5,
    )
    via_M1_M3.ports["bottom"].orientation = 0
    via_M1_M3.ports["top"].orientation = 180
    via_M1_M3_ref = c.add_ref(via_M1_M3)
    via_M1_M3_ref.connect("bottom", D2_ref.ports["e3"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_M3 = ihp.cells.straight(
        length=18,
```

```
        cross_section="metal3_routing",
        width=max(via_M1_M3_ref.xsize, via_M1_M3_ref.ysize),
    )
    straight_M3_ref = c.add_ref(straight_M3)
    straight_M3_ref.connect("e1", via_M1_M3_ref.ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    via_M3_M5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal3",
        vn_columns=1,
        vn_rows=5,
    )
    via_M3_M5.ports["bottom"].orientation = 180
    via_M3_M5_ref = c.add_ref(via_M3_M5)
    via_M3_M5_ref.connect("bottom", straight_M3_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

    schottky.ports["e1"].orientation = 90
    # connect D1 and D2 sub connection together
    schottky.ports["e3"].orientation = 90
    route_SBD_D1_D2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[D1_ref.ports["e1"]],
        ports2=[D2_ref.ports["e3"]],
        route_width=0.3,
        layer=ihp.tech.LAYER.Metal1drawing,
        start_straight_length=2.85,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    straight_M3 = ihp.cells.straight(
        length=10,
        cross_section="metal3_routing",
        width=2,
    )
    straight_M3_ref = c.add_ref(straight_M3)
    straight_M3_ref.connect("e1", output_stage_ref.ports["vref"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)
    # add ports to the powerdetector
    c.add_port(name="vout", port=output_stage_ref.ports["vout"])
    c.ports["vout"].orientation = 90
    c.add_port(name="vref", port=straight_M3_ref.ports["e2"])
    c.ports["vref"].center = (c.ports["vref"].center[0], c.ports["vref"].
↪center[1]-0.1) # preventing notch
```

```
    c.ports["vref"].orientation = 90
    c.add_port(name="rfin", port=via_TM1_TM2_ref.ports["top"])
    c.ports["rfin"].orientation = 270
    c.add_port(name="vss", port=C3_ref.ports["LC_B_1_1"])
    c.ports["vss"].orientation = 90
    c.add_port(name="vdd", port=C3_ref.ports["LC_T_5_1"])
    c.ports["vdd"].orientation = 90
    c.pprint_ports()

    c.center = (0, 0)
    c.add_ref(gf.components.rectangle(size=(c.xsize + 2, c.ysize + 2),␣
↪layer=ihp.tech.LAYER.Activnofill, centered=True))
    c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.GatPolynofill, centered=True))
    c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.Metal1nofill, centered=True))
    c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.Metal2nofill, centered=True))
    c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.Metal3nofill, centered=True))
    c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.Metal4nofill, centered=True))
    # c.add_ref(gf.components.rectangle(size=(c.xsize, c.ysize), layer=ihp.tech.
↪LAYER.Metal5nofill, centered=True))
    M5_nofill = c.add_ref(gf.components.rectangle(size=(C1_ref.xsize+40, C1_ref.
↪ysize+20), layer=ihp.tech.LAYER.Metal5nofill)).center = C1_ref.center

    sizex = C2_ref.xsize + C3_ref.xsize
    sizey = C2_ref.ysize
    x = C2_ref.xmin + sizex/2
    y = C2_ref.ymin + sizey/2
    cap_M5_nofill = c.add_ref(gf.components.rectangle(size=(sizex-10,␣
↪sizey-10), layer=ihp.tech.LAYER.Metal5nofill)).center = (x,y)

    return c
```

8 mmWave Frequency-Scalable Six-Port Receiver Design

First, a GDSFactory Component is created as the top-level container for the entire six-port receiver design. All RF devices and signal routing will be added as references within this Component.

[ ]:
```
c = gf.Component("Six-Port-CAC")
```

8.1 Design Parameters and Wavelength Calculations

Define the key design parameters that govern the entire six-port receiver layout.

Process & Six-Port Specifications & Transmission Line Metal Stack:

Parameter	Description	Value
`e_r`	Relative permittivity (from process specs)	4.1
`Z0`	Characteristic impedance	50 Ω
`f`	Operating frequency (e.g. 160 GHz) — scaling this value automatically resizes the entire six-port layout	variable
`signal_cross_section`	Metal layer for transmission line signal conductors	TopMetal2
`ground_cross_section`	Metal layer for transmission line ground conductors	Metal5

From these parameters, the effective dielectric constant, wavelength, and fractional wavelengths (/4 and /8) are calculated. These quantities directly set the physical dimensions of the RF devices such as the branch-line couplers and bandpass filter.

Bandpass Filter Specifications:

Parameter	Description	Value
`order`	Filter order	3
`bandwidth`	Filter bandwidth	1 GHz
`filter_type`	Filter type — `"butter"` (Butterworth) or `"cheby"` (Chebyshev)	`"butter"`
`ripple_dB`	Passband ripple for Chebyshev filter	3 dB

Interconnect Lengths:

Parameter	Description	Value
`connection_length_bpf`	Connection length between the bandpass filter and the rest of the circuit	10 µm
`connection_length_wpd`	Connection length at the Wilkinson power divider ports	0 µm
`connection_length_bpf_wpd`	Connection length between the branch-line couplers and the Wilkinson power divider	3.5/5 · /4
`connection_length_blc`	Connection length at the branch-line coupler ports	0 µm

[ ]:
```
# RF Design Parameters
e_r = 4.1   # relative permittivity (from process specifications)
Z0  = 50    # characteristic impedance in Ω
f   = 160e9 # operating frequency in Hz

# Transmission Line Metal Stack
signal_cross_section = "topmetal2_routing"  # signal conductor layer (TopMetal2)
ground_cross_section = "metal5_routing"     # ground conductor layer (Metal5)

# Effective Dielectric Constant & Wavelength
e_eff = ihp.cells.waveguides._calculate_effective_dielectric_constant(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    e_r=e_r,
)

c0           = scipy.constants.c            # speed of light in vacuum in m/s
wavelength   = c0 / f * 1e6 / sqrt(e_eff)   # guided wavelength in µm
wavelength_4 = wavelength / 4               # quarter-wavelength in µm

# Snap to process grid (nearest nm)
wavelength   = round(wavelength   - wavelength   % ihp.tech.nm, 3)      #   in␣
↪µm
wavelength_4 = round(wavelength_4 - wavelength_4 % ihp.tech.nm, 3)      # /4␣
↪in µm
wavelength_8 = round(wavelength/8 - (wavelength/8) % ihp.tech.nm, 3)    # /8␣
↪in µm

# Bandpass Filter Parameters
order                 = 3        # filter order
bandwidth             = 1e9      # 3 dB bandwidth (~5% of center frequency) in␣
↪Hz
filter_type           = "butter" # filter type: "butter" (Butterworth) or␣
↪"cheby" (Chebyshev)
ripple_dB             = 3        # passband ripple in dB - applies to Chebyshev␣
↪only
connection_length_bpf = 10       # connection length between the bandpass␣
↪filter and the circuit in µm

# Wilkinson Power Divider Parameters
connection_length_wpd     = 0                       # connection length at WPD␣
↪ports in µm
connection_length_bpf_wpd = wavelength_4 * 3.5 / 5  # connection length between␣
↪the WPD and the branch-line couplers in µm

# Branch-Line Coupler Parameters
```

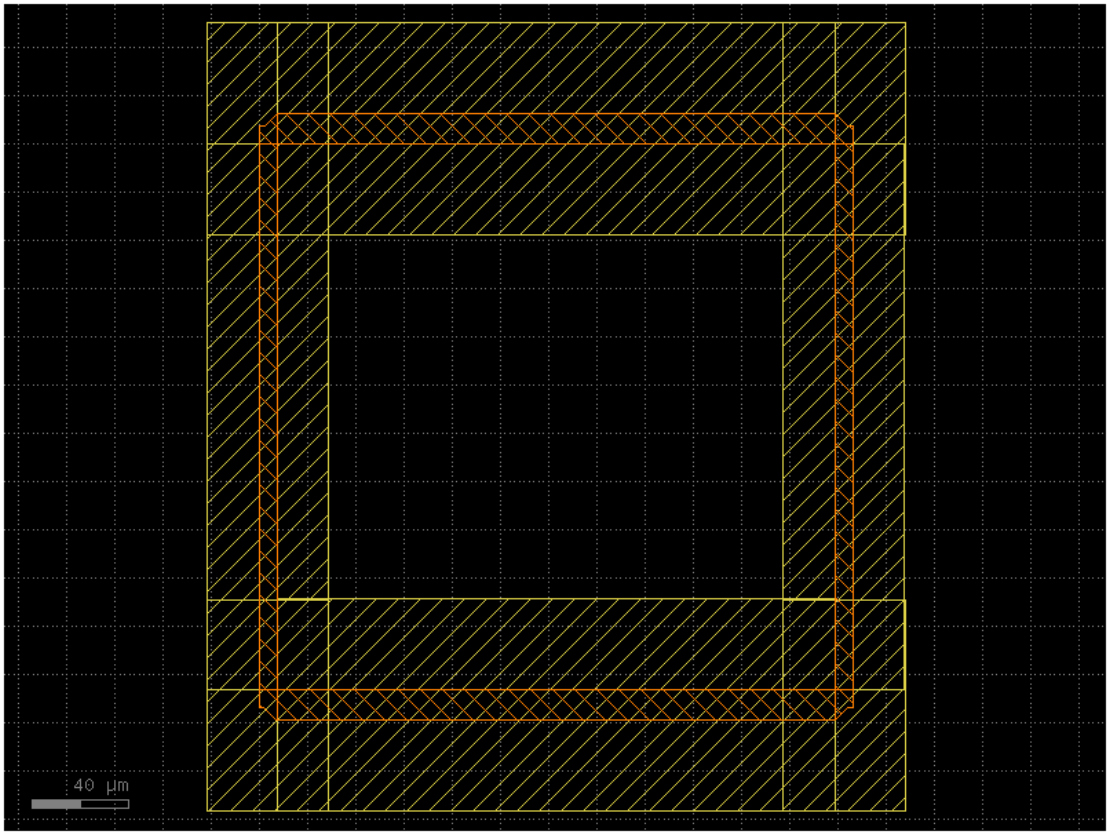
```
connection_length_blc = 0  # connection length at branch-line coupler ports in␣
↪µm
```

8.2 Branch-Line Coupler Design

The first RF device to be created is a branch-line coupler (BLC). The BLC: - is a custom add-on to the IHP-Open-PDK GDSFactory version integrated by us. - is automatically generated from the frequency `f` and relative permittivity `e_r`. - has branch lengths derived from the quarter-wavelength (/4). - has branch widths determined by a formula that yields the desired characteristic impedance. - serves as a 3 dB power divider with the phase relationships required by the six-port architecture. - scales all dimensions automatically with the operating frequency defined above.

[ ]:
```
blc = ihp.cells.branch_line_coupler(
    frequency=f,
    connection_length=connection_length_blc,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
)
blc.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 12.51 um for Z0: 35.35533905932737 ohms
Calculated Z0: 35.3772968531251 ohms for width: 12.51 um
Calculated Z0: 50.03094234385018 ohms for width: 7.2 um
Calculated Z0: 50.03094234385018 ohms for width: 7.2 um
```



The first branch line coupler (BLC_3) is added as a reference to the main component at the center position (0, 0).

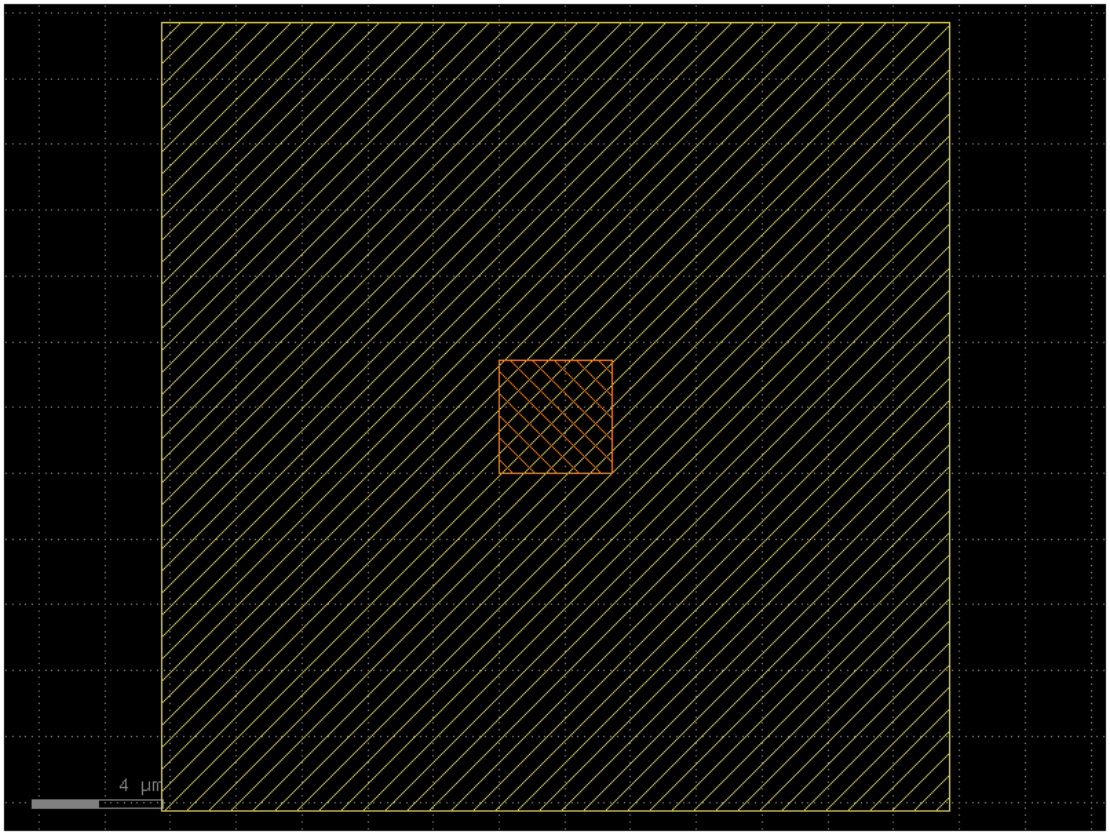
[ ]:
```
blc_3_ref = c.add_ref(blc)
```

8.3 Corner Transition Piece

Instead of connecting directly to BLC ports, this design uses 45-degree diagonal slopes for area optimization. A corner transmission line piece with a 45° angle is created to smoothly transition from the corner port. This impedance-transforming corner connects the first BLC to the adjacent branch line coupler.

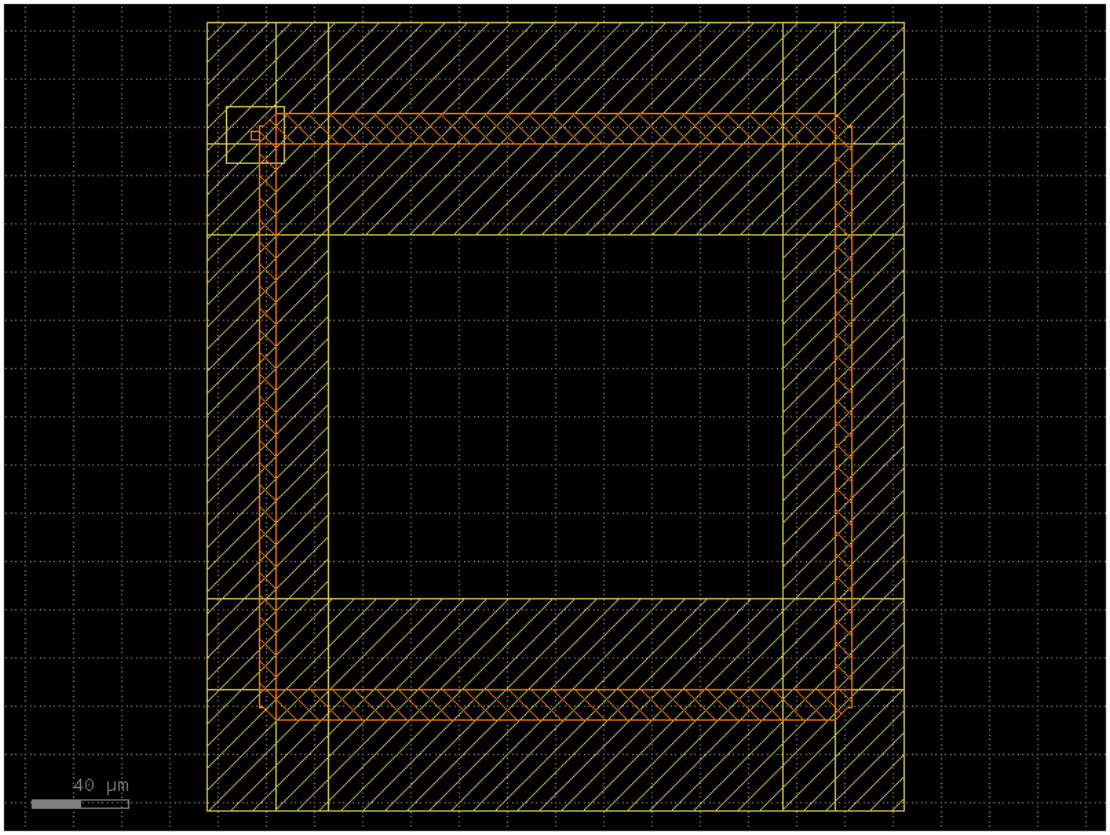
[ ]:
```
corner = ihp.cells.tline_corner(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0*1.314,
)

corner.plot()
corner_top_ref = c.add_ref(corner)
```

```
Calculated width: 3.42 um for Z0: 65.7 ohms
```



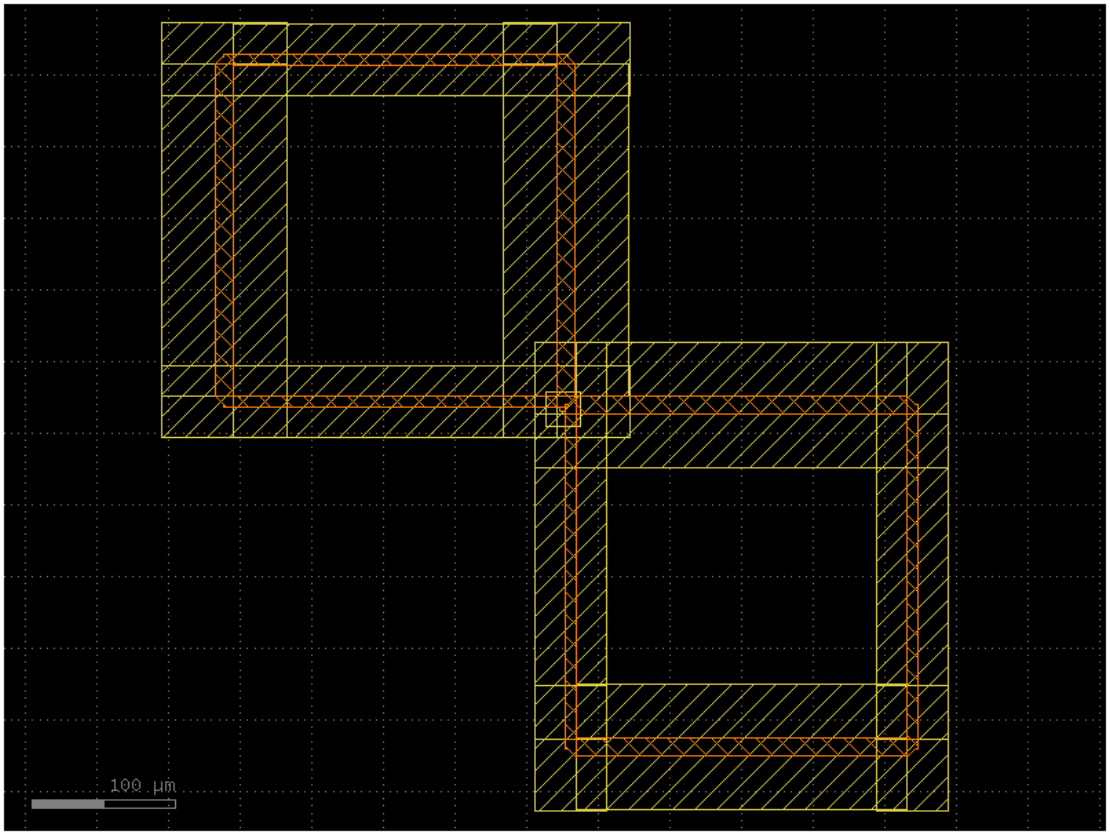
The first BLC (BLC_3) is positioned at the origin (0, 0) and connects the top corner piece to its port (e1) at 45 degrees. To verify the placement, the result is plotted.

[ ]:
```
blc_3_ref.center = (0, 0)
corner_top_ref.connect("e1", blc_3_ref.ports["e1"], allow_width_mismatch=True)

c.plot()
```



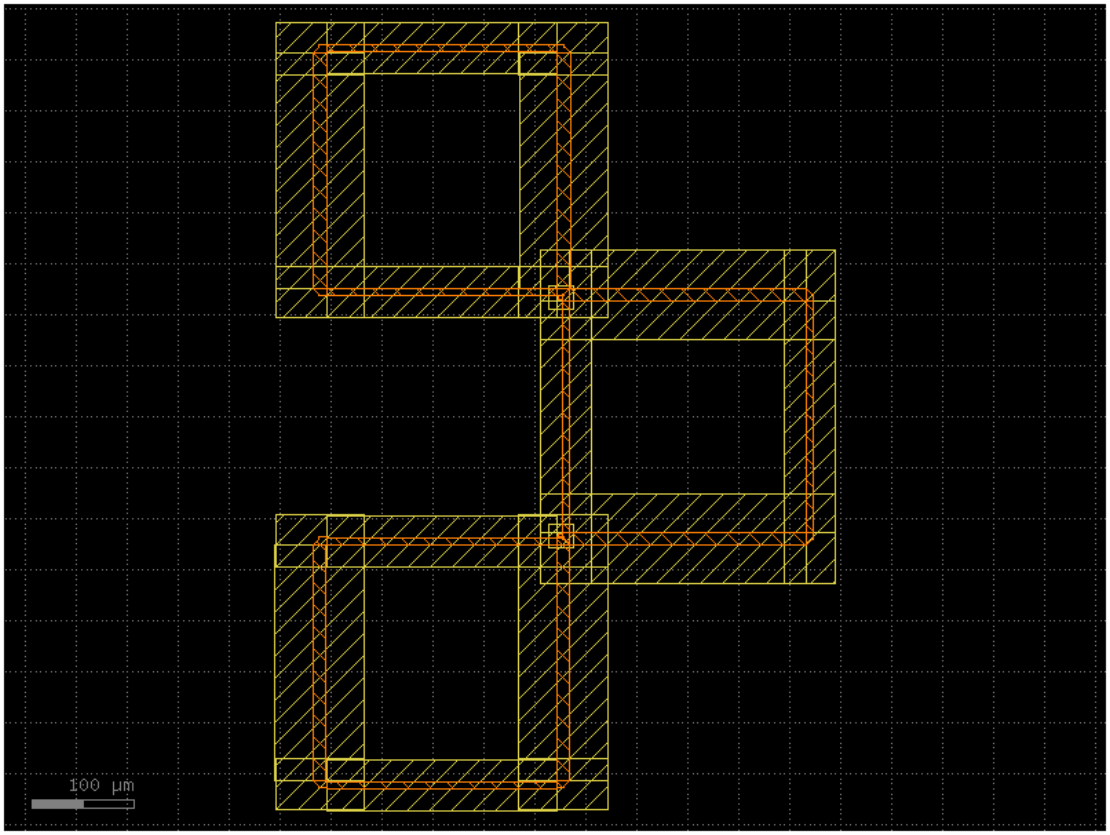
The second branch line coupler (BLC_1) is added at the top by connecting it to the top corner pieces port (e4).

[ ]:
```
blc_1_ref = c.add_ref(blc)
blc_1_ref.connect("e4", corner_top_ref.ports["e4"], allow_width_mismatch=True)

c.plot()
```



A second corner piece is added for the bottom branch, and the third BLC (BLC_2) is placed by connecting it through this corner piece to BLC_3's bottom port (e4).

[ ]:
```
corner_bot_ref = c.add_ref(corner)
corner_bot_ref.connect("e1", blc_3_ref.ports["e4"], allow_width_mismatch=True)
blc_2_ref = c.add_ref(blc)
blc_2_ref.connect("e1", corner_bot_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```



8.4 Wilkinson Power Divider Design

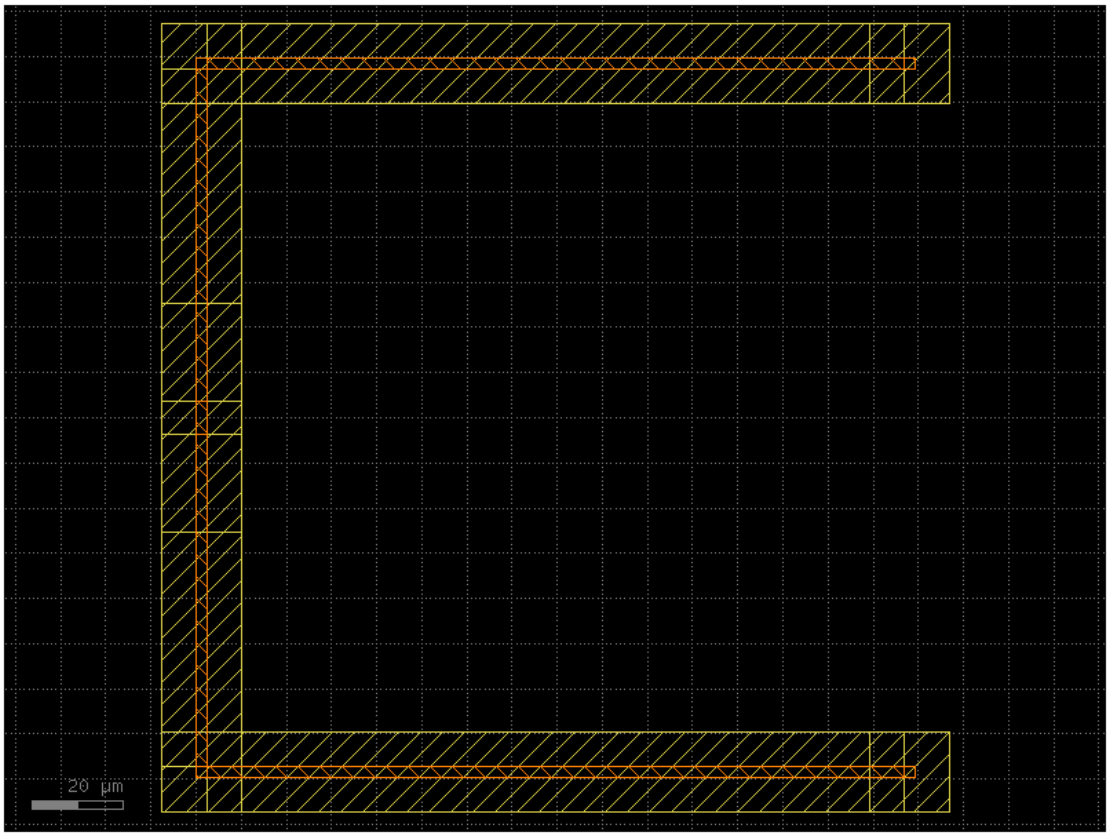
Now, that the three BLCs form a corner structure, the second RF device to be created is a Wilkinson power divider (WPD). The WPD: - is a custom add-on to the IHP-Open-PDK GDSFactory version integrated by us. - is automatically generated from the frequency `f` and relative permittivity `e_r`. - has branch lengths derived from the quarter-wavelength (/4). - has branch widths determined by a formula that yields the desired characteristic impedance. - serves as a 3 dB power divider. Since the WPD is not used for combining, the textbook resistor between the two output ports can be neglected. The target wavelength is anyway so small that the resistor would only have a negligible impact. - scales all dimensions automatically with the operating frequency defined above.

The WPD is placed in the central space between BLC_1 (top) and BLC_2 (bottom) for an optimized area. This U-shaped power divider will split the RF signal from the bandpass filter into two paths, feeding the top and bottom BLC.

[ ]:
```
wpd = ihp.cells.wilkinson_power_divider(
    frequency=f,
    connection_length=connection_length_wpd,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
```

```
    e_r=e_r,
    shape="U",
)

wpd.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 2.5100000000000002 um for Z0: 70.71067811865476 ohms
Calculated Z0: 70.75520001248569 ohms for width: 2.5100000000000002 um
Calculated width: 2.5100000000000002 um for Z0: 70.71067811865476 ohms
Calculated Z0: 70.75520001248569 ohms for width: 2.5100000000000002 um
```



The input port of the WPD is rotated to face left (orientation = 0), then is added as a reference to the main component. This ensures the input transmission line connects properly from the left side.

[ ]:
```
wpd.ports["e1"].orientation = 0
wpd_ref = c.add_ref(wpd)
```

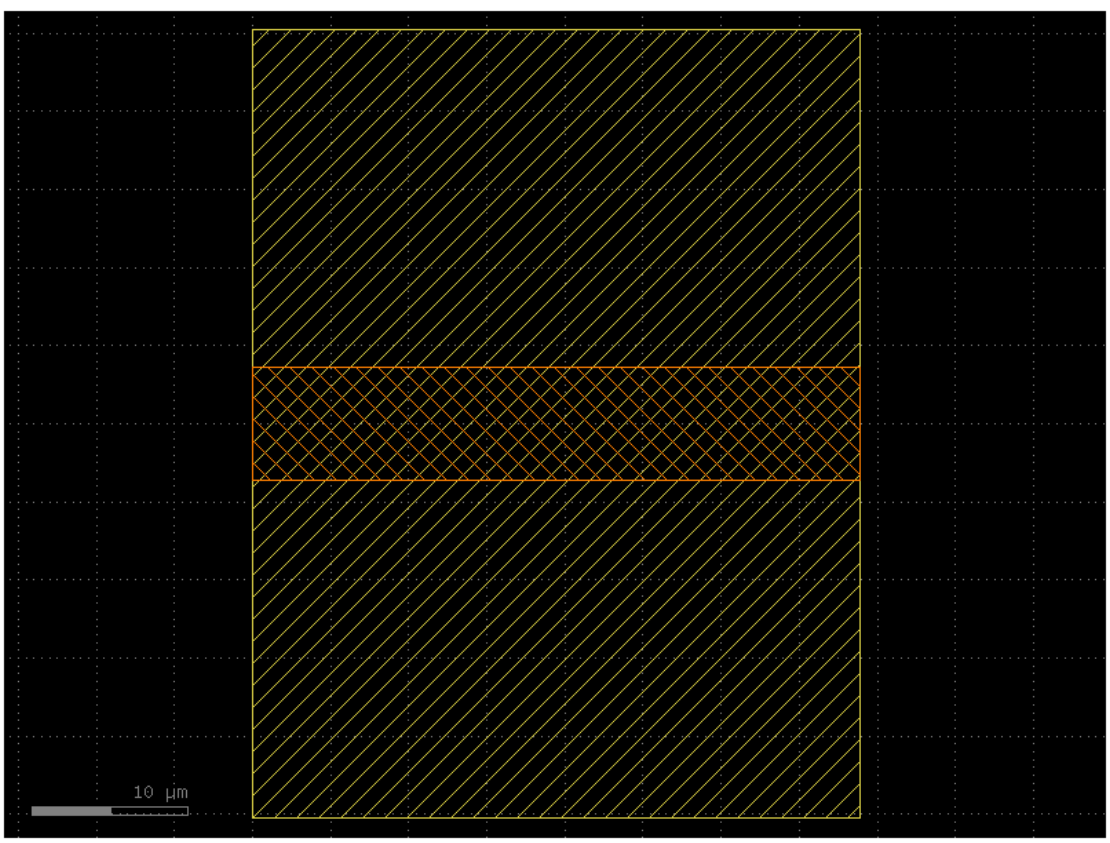
8.5 WPD-to-BLC Connections

The transmission line segments that connect the WPD outputs to the BLC ports are calculated as follows: - Measure the vertical distance between BLC_1 and BLC_2 - Subtract the WPD height - Divide by 2 to get the connection length for each side (top and bottom)

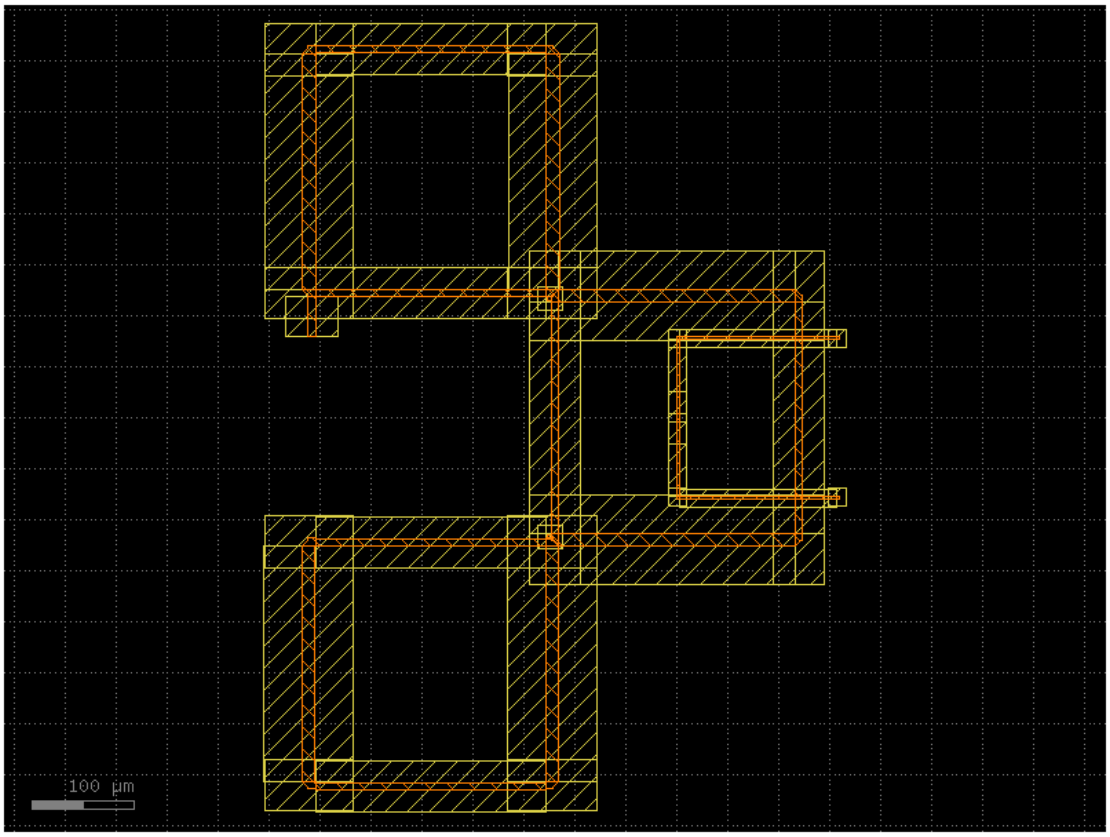
A single connection line segment is created, which is used for both the top and bottom connections to balance the layout.

[ ]:
```
connection_length_wpd_blc_one_leg = blc_1_ref.ports["e1"].center[1] - blc_2_ref.
↪ports["e4"].center[1] - (wpd_ref.ports["e2"].center[1] - wpd_ref.ports["e3"].
↪center[1])

connection_wpd_blc = ihp.cells.tline(
    length=connection_length_wpd_blc_one_leg/2,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
)

connection_wpd_blc.plot()
```

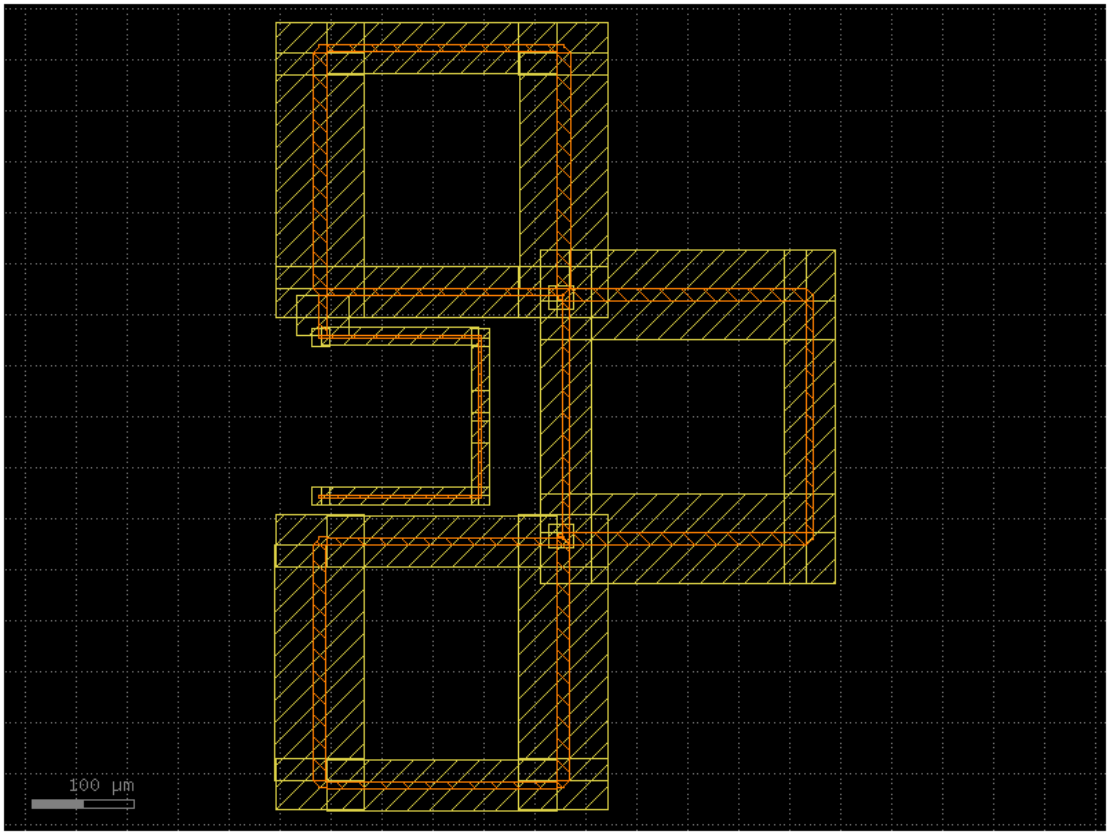
```
Calculated width: 7.2 um for Z0: 50 ohms
```



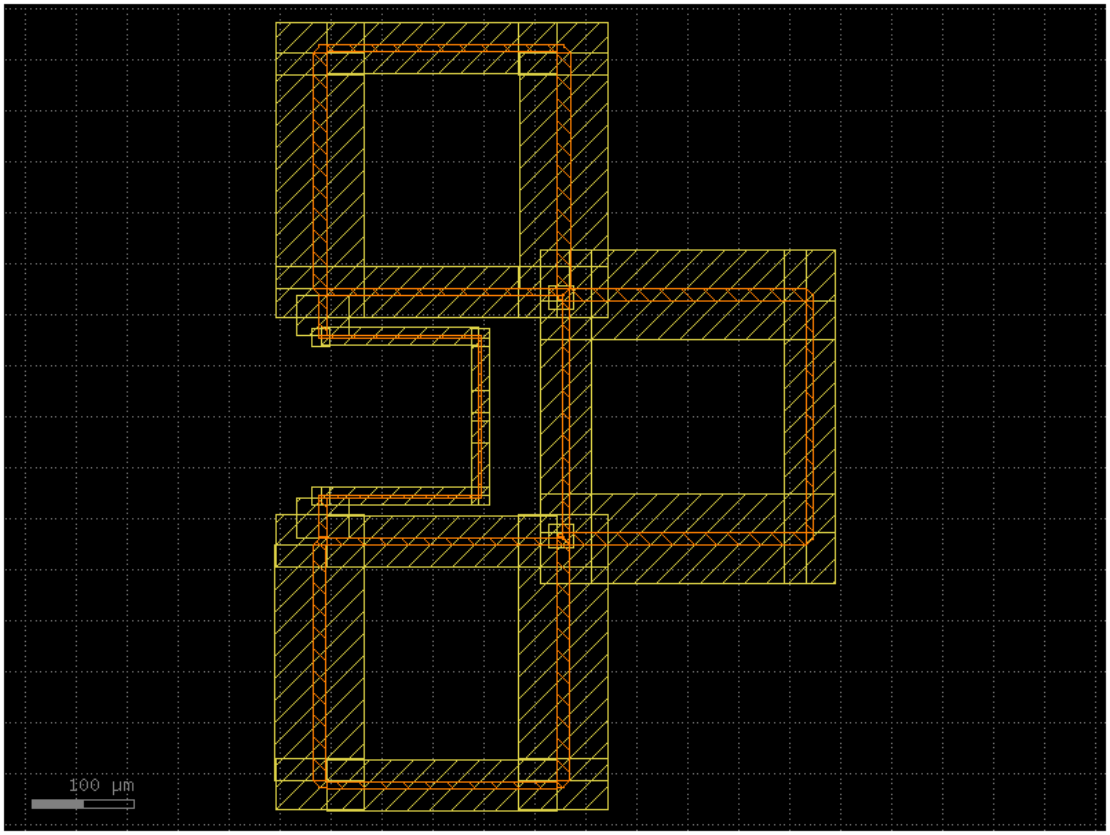
[ ]:
```
connection_wpd_blc_top_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_top_ref.connect("e1", blc_1_ref.ports["e1"])

c.plot()
```



[ ]:
```
wpd_ref.connect("e3", connection_wpd_blc_top_ref.ports["e2"])

c.plot()
```



[ ]:
```
connection_wpd_blc_bot_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_bot_ref.connect("e1", blc_2_ref.ports["e4"])

wpd_ref.connect("e2", connection_wpd_blc_bot_ref.ports["e2"])

c.plot()
```



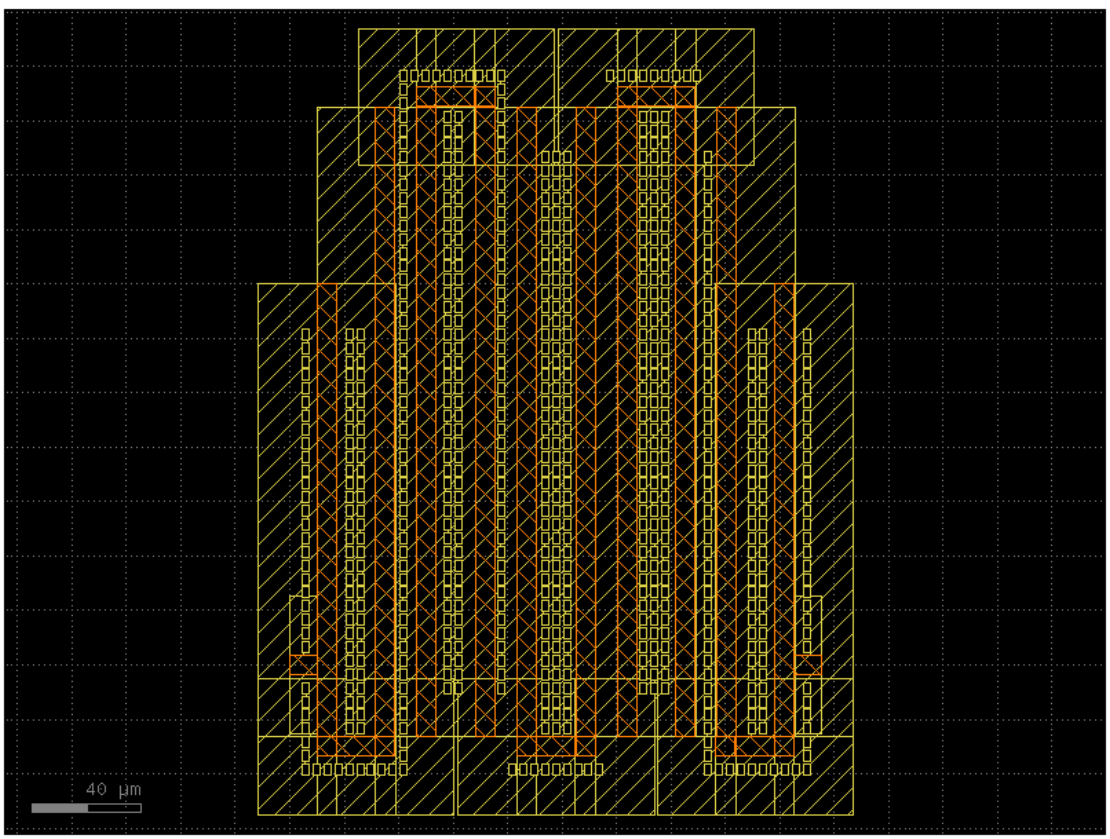
8.6 Add Hairpin Coupled-Line Bandpass Filter

Create the bandpass filter that provides the RF input to the six-port network. Using the previously defined filter parameters (3rd-order Butterworth, 1 GHz bandwidth, 160 GHz center frequency), generate a hairpin coupled-line structure. This filter: - Provides frequency selectivity and gain for the RF input - Uses inter-digital coupling for compact size - Connects to the WPD input through a connection line segment

[ ]:
```
bandpass_filter = ihp.cells.hairpin_coupled_line_bandpass_filter(
    frequency=f,
    bandwidth=bandwidth,
    order=order,
    filter_type=filter_type,
    ripple_dB=ripple_dB,
    connection_length=connection_length_bpf,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
)
```

```
bandpass_filter_ref = c.add_ref(bandpass_filter)

bandpass_filter.plot()
```

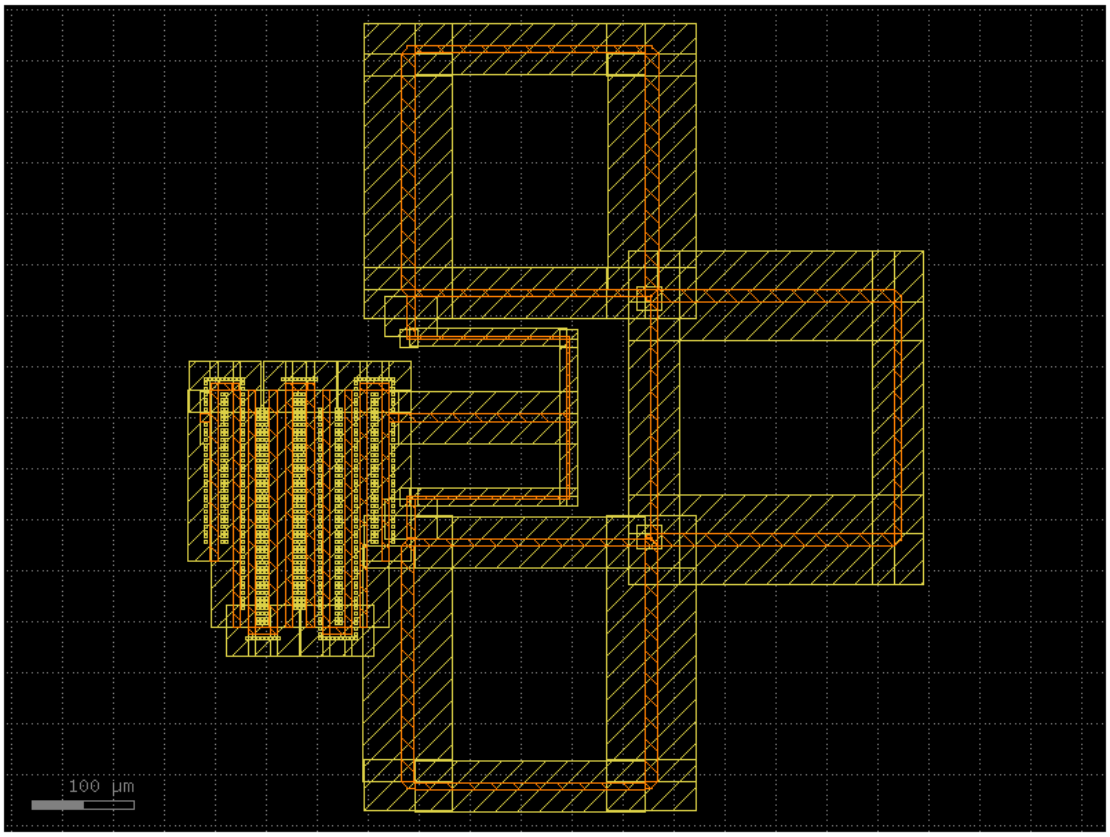
```
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 7.13 um for Z0: 50.24723524478605 ohms
Calculated Z0: 50.26863690347976 ohms for width: 7.13 um
Calculated width: 7.2 um for Z0: 50 ohms
Calculated width: 7.2 um for Z0: 50.0012048292377 ohms
Calculated Z0: 50.03094234385018 ohms for width: 7.2 um
Calculated width: 7.2 um for Z0: 50 ohms
2026-04-10 13:34:31.838 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with
calculating fill regions for Unnamed_27
```



[ ]:
```
connection_bpf_wpd = c.add_ref(ihp.cells.tline(
    length=connection_length_bpf_wpd,
    signal_cross_section=signal_cross_section,
```

```
    ground_cross_section=ground_cross_section,
    Z0=Z0,
))

connection_bpf_wpd.connect("e1", wpd_ref.ports["e1"])
bandpass_filter_ref.connect("e1", connection_bpf_wpd.ports["e2"])

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohms
```

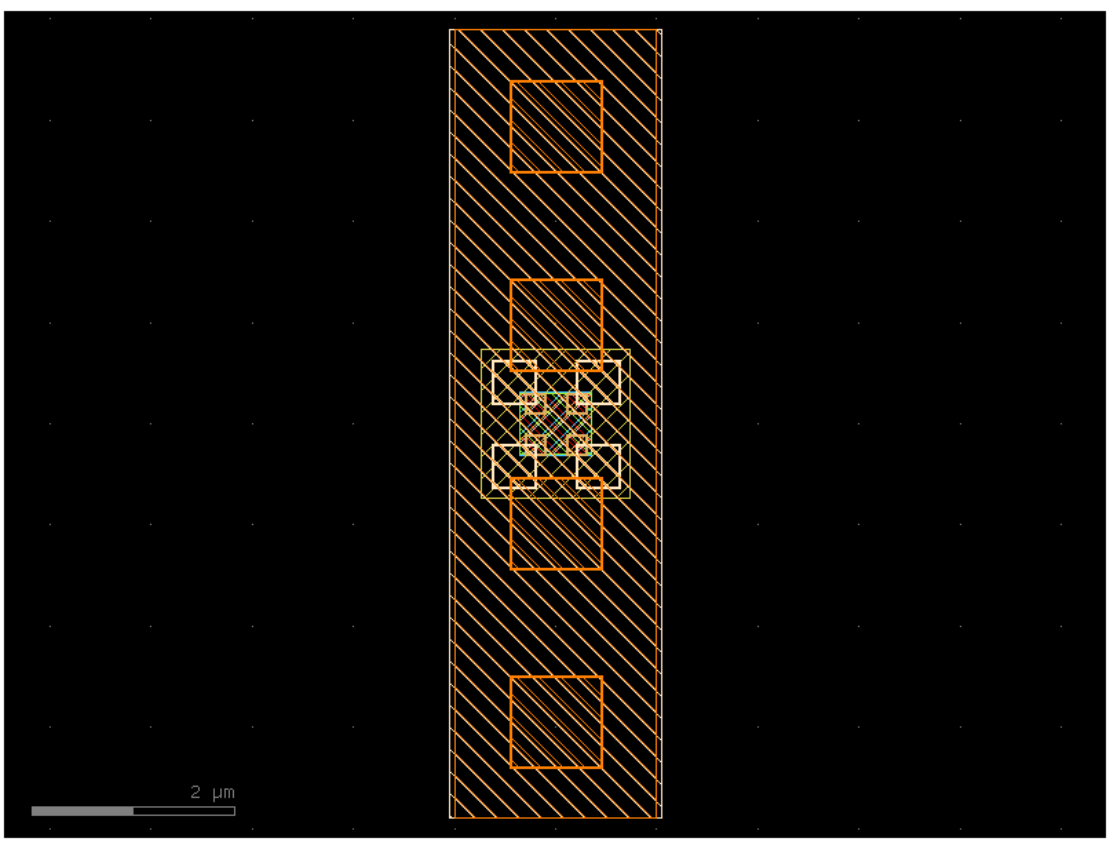


With the RF structures complete, the next step is adding a termination resistor on one port of `blc_3`, which is required for proper six-port operation.

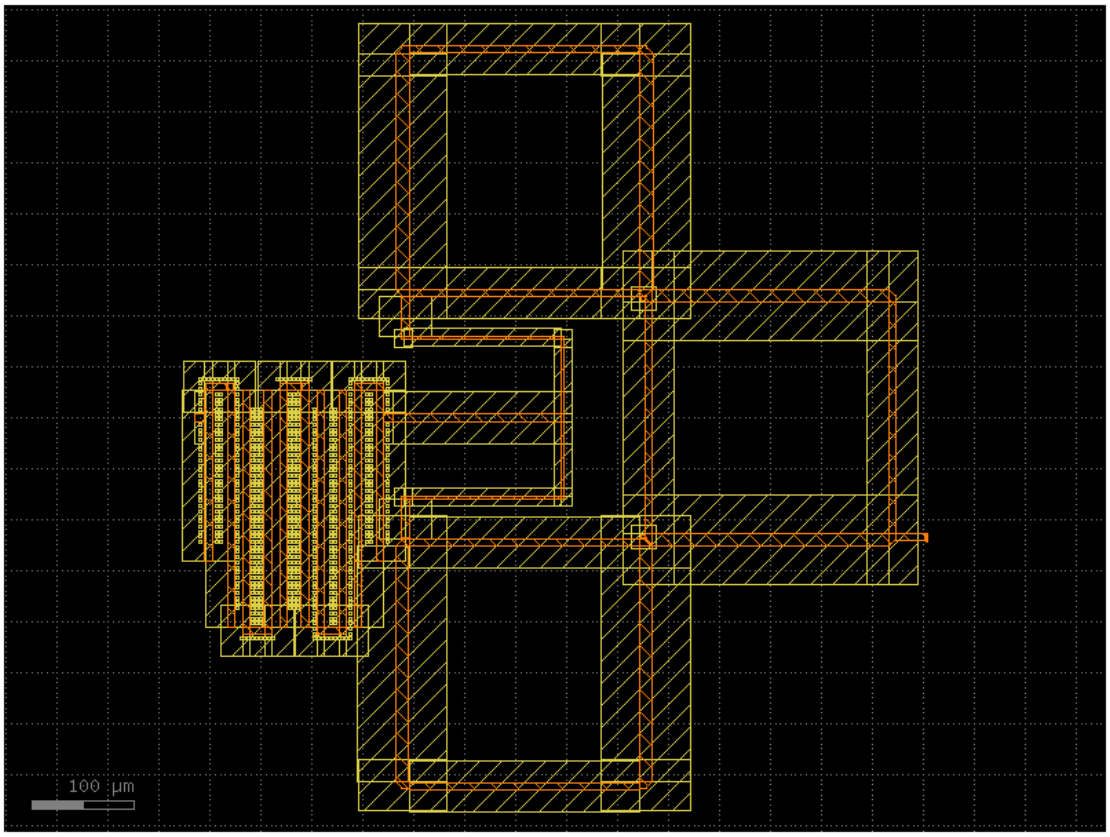
[ ]:
```
connection_blc_R_termination = ihp.cells.straight(
    length=30, # manual meassurement to fit the design
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,␣
 ↪signal_cross_section=signal_cross_section,␣
 ↪ground_cross_section=ground_cross_section),
)
```

```
connection_blc_R_termination_ref = c.add_ref(connection_blc_R_termination)
connection_blc_R_termination_ref.connect("e1", blc_3_ref.ports["e3"])

via_M1_TM2 = ihp.cells.via_stack(
    top_layer="TopMetal2",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
    vt1_columns=2,
    vt1_rows=2,
    vt2_columns=1,
    vt2_rows=4,
)

via_M1_TM2.plot()
via_M1_TM2.ports["top"].orientation = 180
via_M1_TM2.ports["bottom"].orientation = 180
via_M1_TM2_ref = c.add_ref(via_M1_TM2)
via_M1_TM2_ref.connect("top", connection_blc_R_termination_ref.ports["e2"],␣
 ↪allow_width_mismatch=True)
```

```
Calculated width: 7.2 um for Z0: 50 ohms
via_stack PyCell placed successfully and GDS written.
```

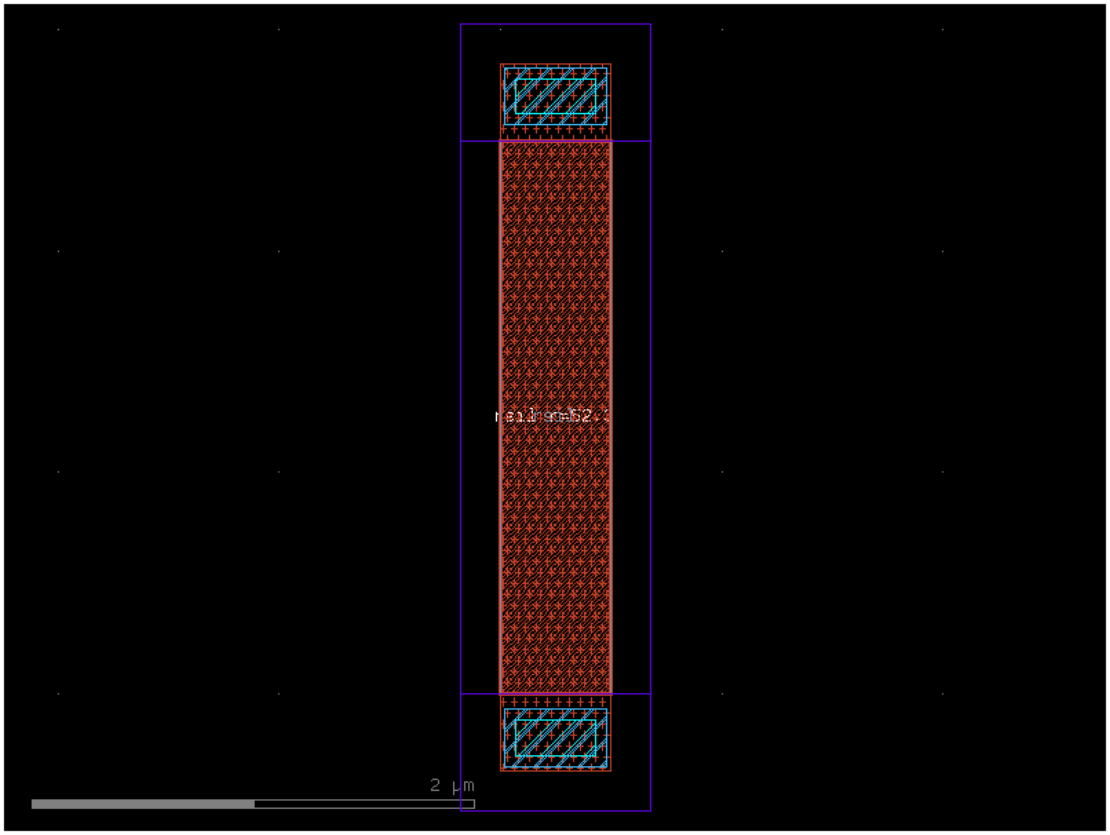


[ ]:
```
c.plot()
```



[ ]:
```
R_termination = ihp.cells.rsil(
    resistance=Z0,
    width=0.5,
    length=2.5,
)
R_termination.plot()
```

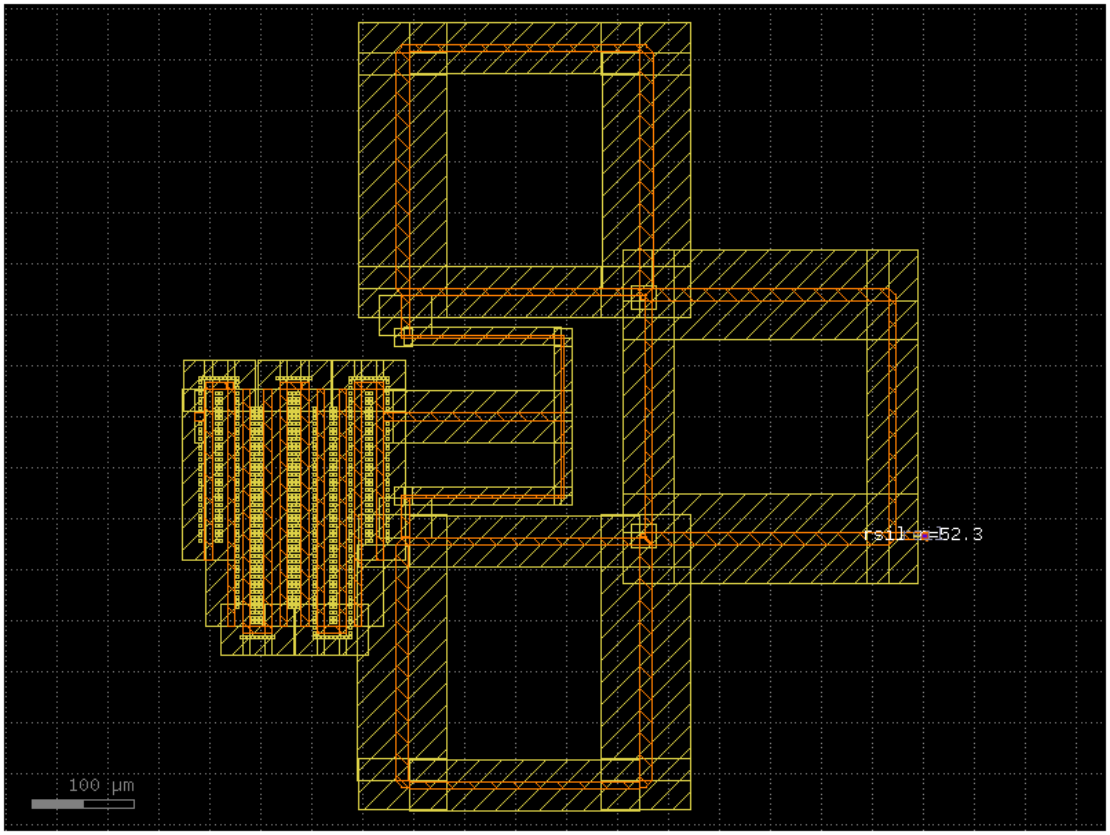
```
rsil PyCell placed successfully and GDS written.
```



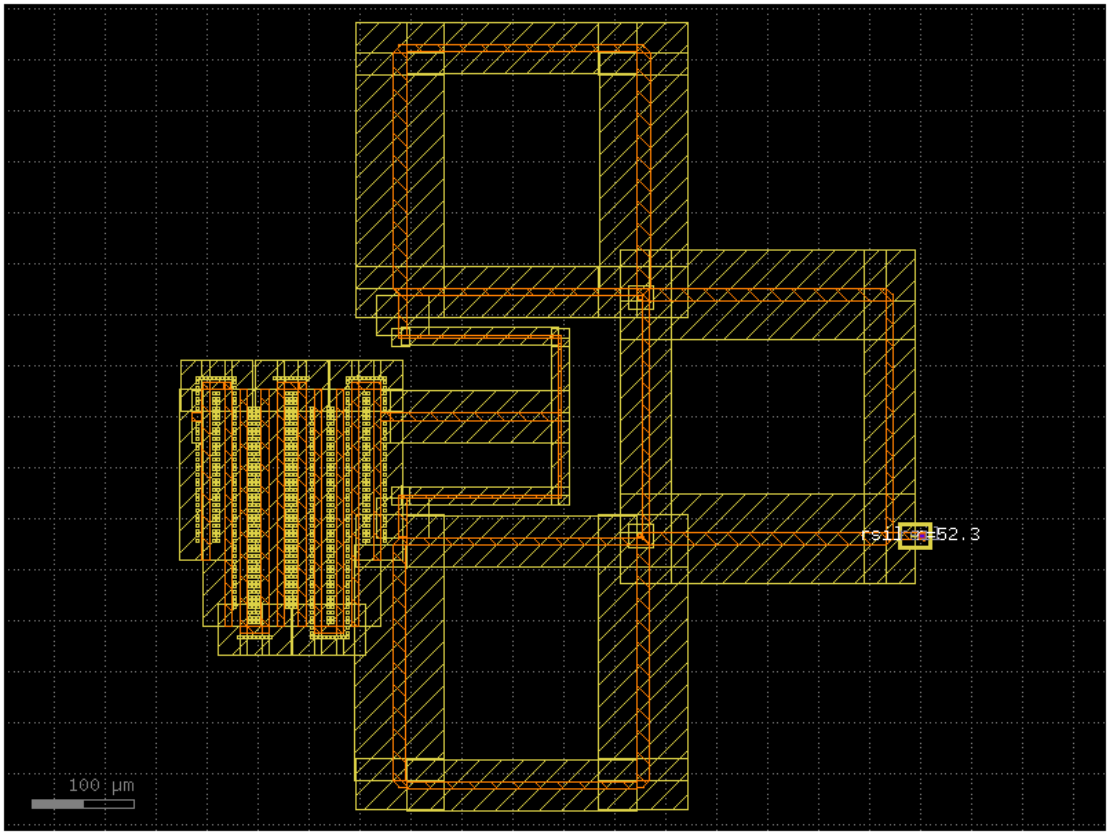
[ ]:
```
R_termination_ref = c.add_ref(R_termination)
R_termination_ref.connect("e1", via_M1_TM2_ref.ports["bottom"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

via_M1_M5 = ihp.cells.via_stack(
    top_layer="Metal5",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
)
via_M1_M5.ports["top"].orientation = 180
via_M1_M5.ports["bottom"].orientation = 0
via_M1_M5_ref = c.add_ref(via_M1_M5)
via_M1_M5_ref.connect("bottom", R_termination_ref.ports["e2"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)

connection_R_termination_vss = ihp.cells.straight(
    length=10, # manual meassurement to fit the design
    cross_section=ground_cross_section,
    width=2, # manual meassurement to fit the design
```

```
)
connection_R_termination_vss_ref = c.add_ref(connection_R_termination_vss)
connection_R_termination_vss_ref.connect("e1", via_M1_M5_ref.ports["top"],␣
↪allow_width_mismatch=True, allow_layer_mismatch=True)


c.plot()
```

via_stack PyCell placed successfully and GDS written.

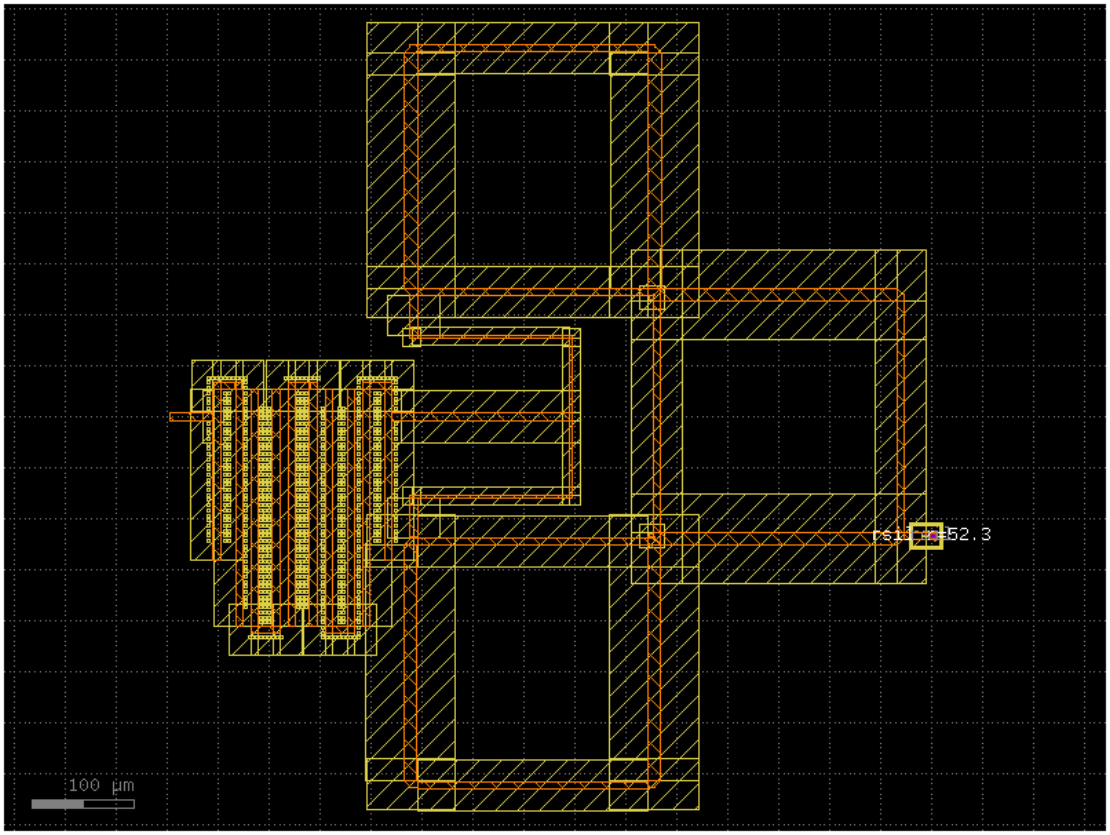


[ ]:
```
# r termination metal5 nofill
c.add_ref(gf.components.rectangle(size=(connection_R_termination_vss_ref.
↪xsize+20, connection_R_termination_vss_ref.ysize+20), layer=ihp.tech.LAYER.
↪Metal5nofill)).center = connection_R_termination_vss_ref.center

c.plot()
```



[ ]:
```
connection_bpd_pad = c.add_ref(ihp.cells.straight(
    length=32,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,␣
 ↪signal_cross_section=signal_cross_section,␣
 ↪ground_cross_section=ground_cross_section),
))
connection_bpd_pad.connect("e1", bandpass_filter_ref.ports["e2"],␣
 ↪allow_width_mismatch=True)
c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohms
```



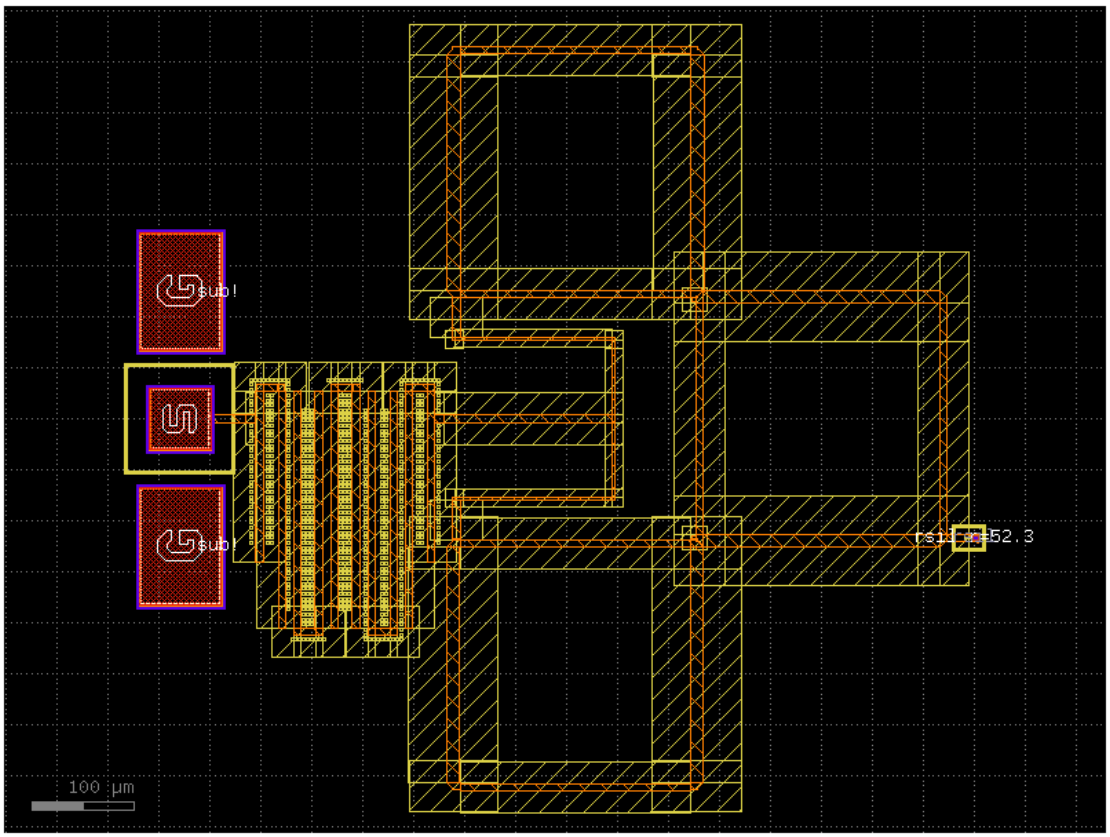
8.7 Add Probe Pad for Local Oscillator (LO)

Add a GSG (Ground-Signal-Ground) bondpad array on the left side for the LO input signal. This pad: - Connects to the bandpass filter input path through a transition line - Provides controlled 50-ohm probing interface - Includes a Metal5 no-fill region to minimize parasitic capacitance

[ ]:
```
probe_left = c.add_ref(ihp.cells.bondpad_array(config="GSG", pitch= 125,␣
↪width_ground=120, width_signal=65, length_signal=65,␣
↪signal_cross_section="topmetal1_routing",␣
↪ground_cross_section="metal1_routing", ground_connection='psub')).rotate(-90)
probe_left.center = bandpass_filter.ports["e2"].center
probe_left.xmax = bandpass_filter.ports["e2"].center[0]-15
probe_left.connect("e1", connection_bpd_pad.ports["e2"],␣
↪allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(105, 105), layer=ihp.tech.LAYER.
↪Metal5nofill)).center = probe_left.center
c.plot()
```

```
bondpad PyCell placed successfully and GDS written.
guardring PyCell placed successfully and GDS written.
```

```
bondpad PyCell placed successfully and GDS written.
```



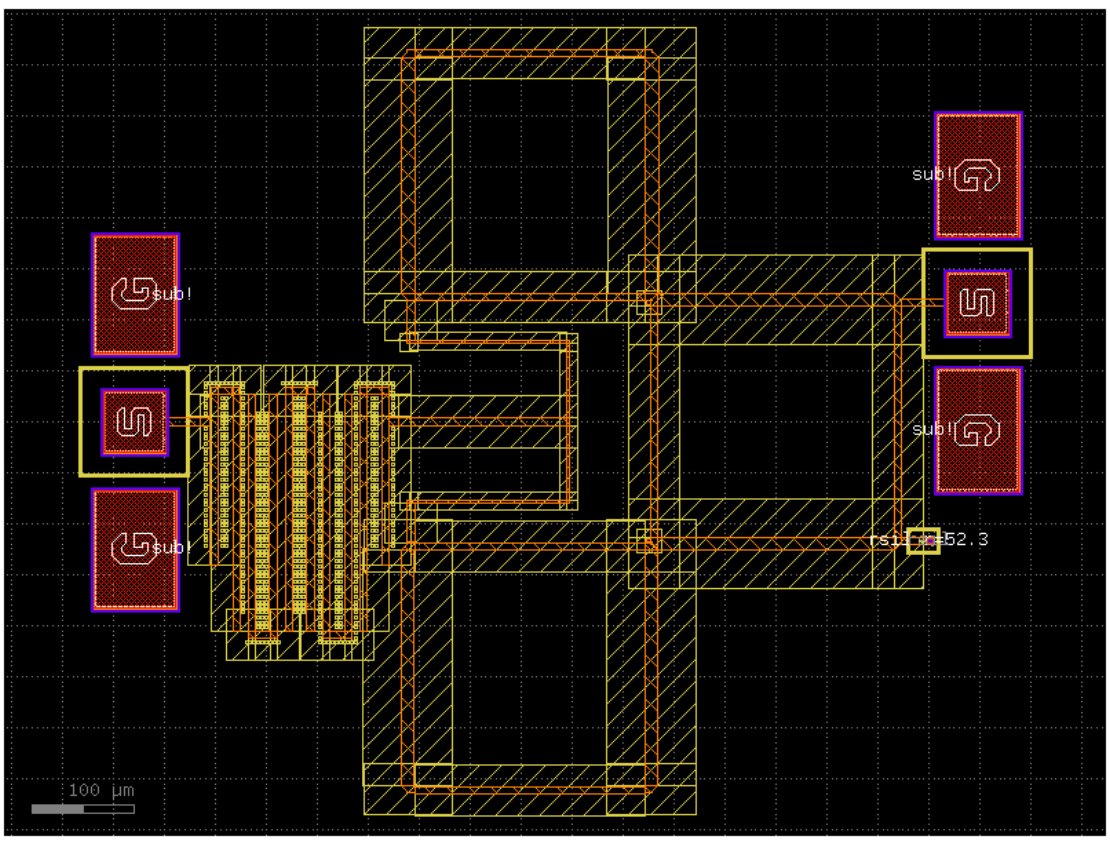
8.8 Add Probe Pad for RF Input to BLC_3

Add a GSG bondpad array on the right side to provide the RF input signal directly to the center BLC_3. This connection: - Adds a transmission line segment from BLC_3 output - Places the pad at the RF input node of the six-port network - Includes Metal5 no-fill region to minimize loading effects

[ ]:
```
connection_blc_pad = c.add_ref(ihp.cells.straight(
    length=42,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,␣
 ↪signal_cross_section=signal_cross_section,␣
 ↪ground_cross_section=ground_cross_section),
))
connection_blc_pad.connect("e1", blc_3_ref.ports["e2"])

# probe pads right
```

```
probe_right = c.add_ref(ihp.cells.bondpad_array(config="GSG", pitch= 125,␣
↪width_ground=125, width_signal=65, length_signal=65,␣
↪signal_cross_section="topmetal1_routing",␣
↪ground_cross_section="metal1_routing", ground_connection='psub')).rotate(-90)
probe_right.connect("e1", connection_blc_pad.ports["e2"],␣
↪allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(105, 105), layer=ihp.tech.LAYER.
↪Metal5nofill)).center = probe_right.center

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohms
bondpad PyCell placed successfully and GDS written.
guardring PyCell placed successfully and GDS written.
```



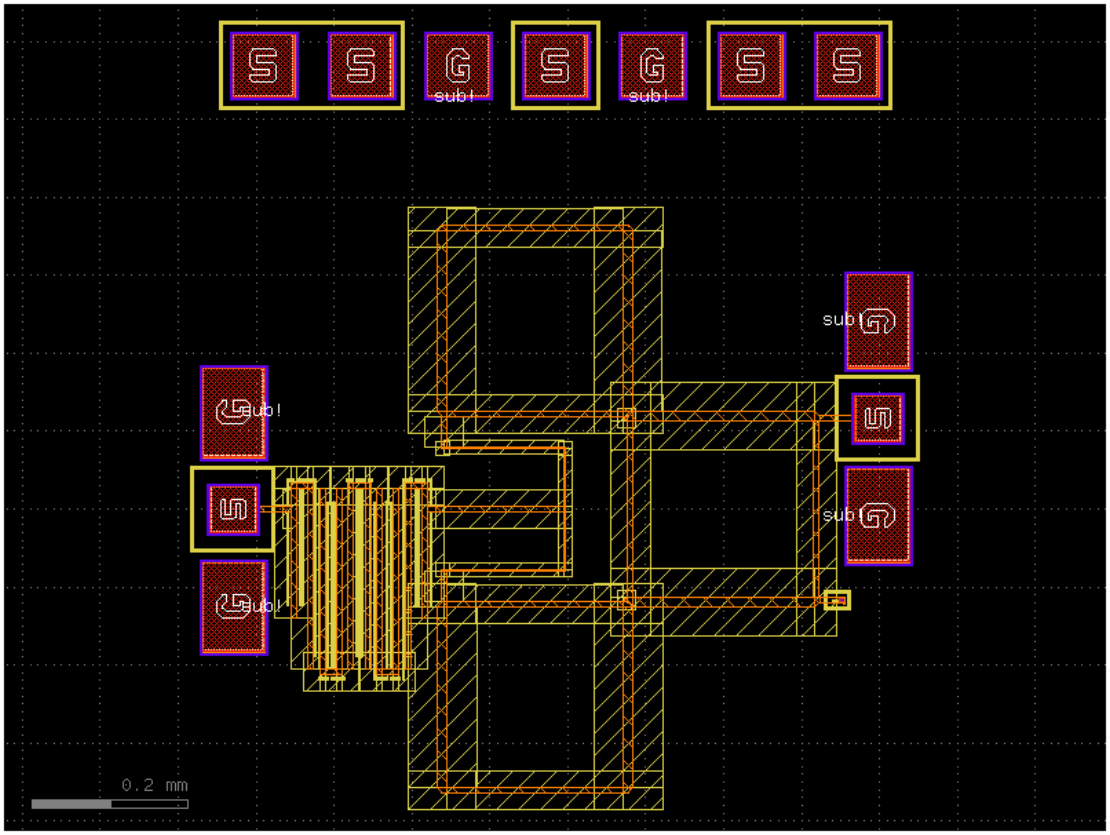
8.9 Add Measurement Pads for Power Detectors

The RF network is now complete, but the outputs feed into power detectors for direct waveform sampling. Add bondpad arrays: - **Top pads**: Seven signal pads (SSGSGSS configuration) for VDD, VSS_PD, VOUT, VREF, VSS, VSS, VSS connections - **Bottom pads**: Seven signal pads

(SSGSGSS configuration, rotated 180°) for the bottom-side detectors - **No-fill regions**: Metal5 rectangles placed around each pad to minimize parasitic capacitance

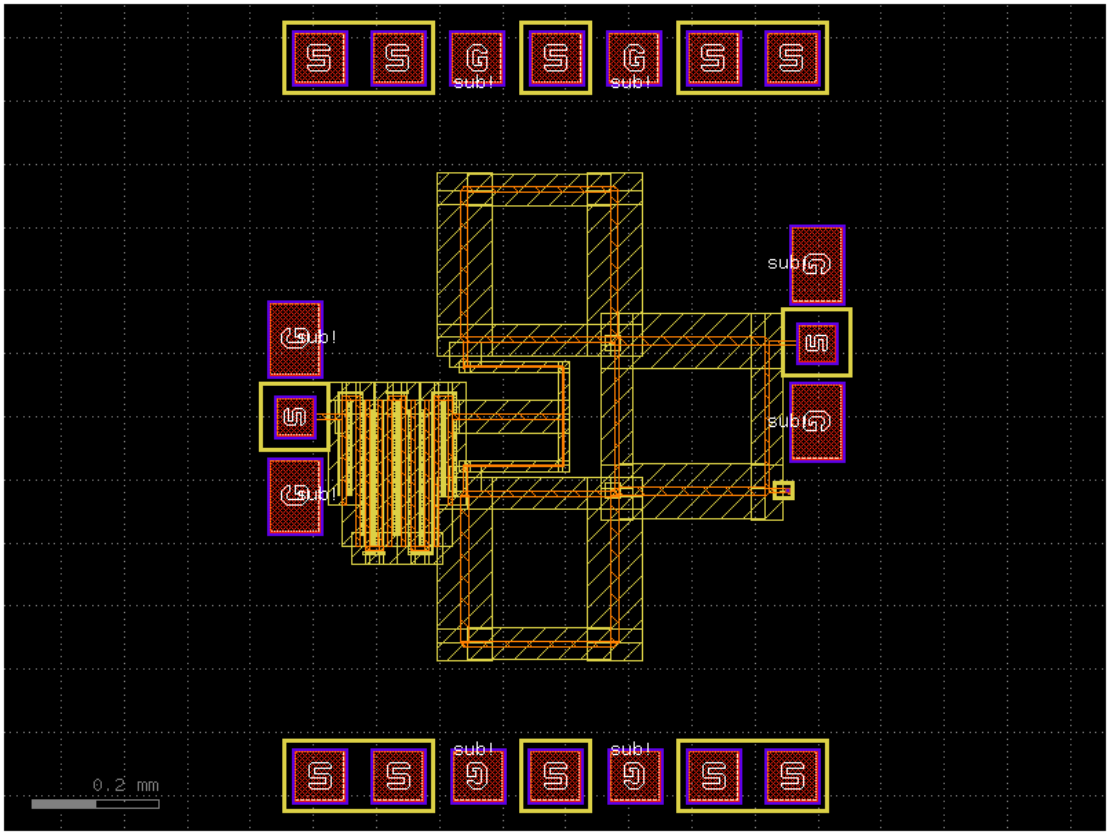
[ ]:
```
# probe pads top
_ = c.center
probe_top = c.add_ref(ihp.cells.bondpad_array(config="SSGSGSS",␣
↪signal_cross_section="topmetal1_routing",␣
↪ground_cross_section="metal1_routing", ground_connection='psub'))
probe_top.center = _
probe_top.ymax = c.ymax + 225

# no fill VDD
c.add_ref(gf.components.rectangle(size=(110, 110), layer=ihp.tech.LAYER.
↪Metal5nofill)).center = probe_top.center

# no fill top
no_fill_top_left = c.add_ref(gf.components.rectangle(size=(85+125+25, 110),␣
↪layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_left.xmin = probe_top.xmin-(110-85)/2
no_fill_top_left.ymin = probe_top.ymin-(110-85)/2

no_fill_top_right = c.add_ref(gf.components.rectangle(size=(85+125+25, 110),␣
↪layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_right.xmax = probe_top.xmax+(110-85)/2
no_fill_top_right.ymin = probe_top.ymin-(110-85)/2

c.plot()
```

```
bondpad PyCell placed successfully and GDS written.
bondpad PyCell placed successfully and GDS written.
guardring PyCell placed successfully and GDS written.
```



[ ]:
```
# probe pads bottom
probe_bottom = c.add_ref(ihp.cells.bondpad_array(config="SSGSGSS",␣
↪signal_cross_section="topmetal1_routing",␣
↪ground_cross_section="metal1_routing", ground_connection='psub')).rotate(180)
probe_bottom.center = _
probe_bottom.ymin = c.ymin - 225

# no fill VDD
c.add_ref(gf.components.rectangle(size=(110, 110), layer=ihp.tech.LAYER.
↪Metal5nofill)).center = probe_bottom.center

# no fill bottom
no_fill_bottom_left = c.add_ref(gf.components.rectangle(size=(85+125+25, 110),␣
↪layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_left.xmin = probe_bottom.xmin-(110-85)/2
no_fill_bottom_left.ymin = probe_bottom.ymin-(110-85)/2

no_fill_bottom_right = c.add_ref(gf.components.rectangle(size=(85+125+25, 110),␣
↪layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_right.xmax = probe_bottom.xmax+(110-85)/2
```

```
no_fill_bottom_right.ymin = probe_bottom.ymin-(110-85)/2

c.plot()
```



8.10 Instantiate Power Detector Module

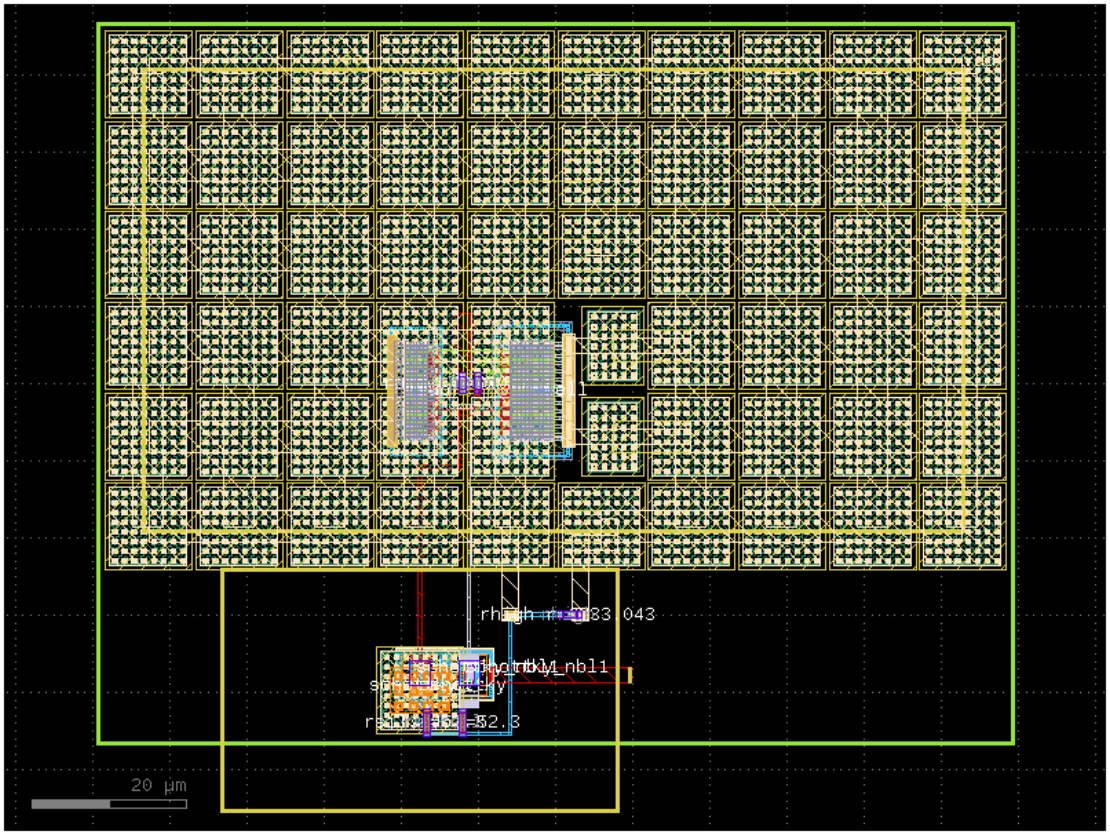
Create an instance of the power detector module (powdet_sbd) that was defined earlier. This detector will be used as the base template for all four detector channels on the chip.

[ ]:
```
pd = powdet_sbd()
pd.plot()
```

```
via_stack PyCell placed successfully and GDS written.
cmim PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
schottky PyCell placed successfully and GDS written.
rsil PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
nmos PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
```

```
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
pmos PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
rppd PyCell placed successfully and GDS written.
cmim PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
rhigh PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
via_stack PyCell placed successfully and GDS written.
```

```
name   width   orientation   layer                      center                    ␣
↪ port_type

vout   0.7     90.0          Metal4drawing (50/0)       (7.37, 40.84)             ␣
↪ electrical
vref   2.0     90.0          Metal3drawing (30/0)       (5.47, 48.68)             ␣
↪ electrical
rfin   7.78    270.0         TopMetal2drawing (134/0)   (0.0, 0.0)                ␣
↪ electrical
vss    11.2    90.0          Metal5drawing (67/0)       (70.32000000000001, 79.8)␣
↪ electrical
vdd    10.08   90.0          TopMetal1drawing (126/0)   (23.44, 79.8)             ␣
↪ electrical
```



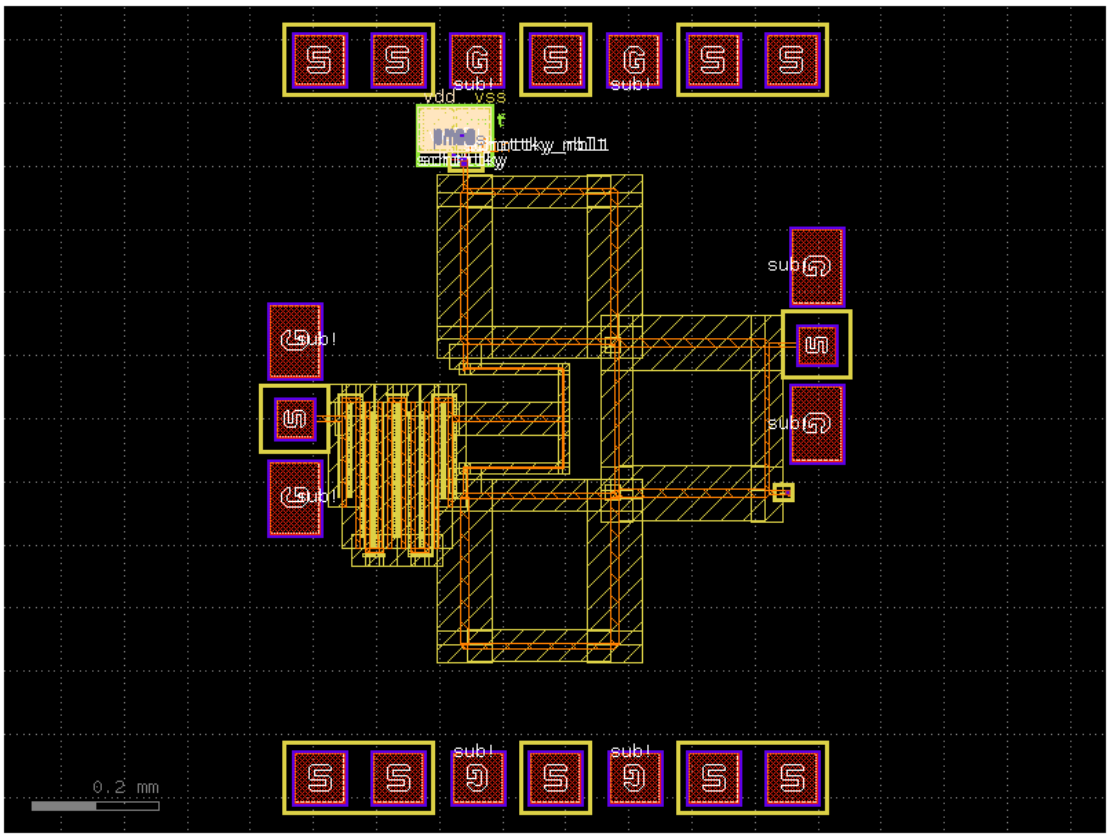
8.11 Place and Connect First Power Detector (PD1)

Instantiate PD1 using a mirrored detector variant and place it by connectivity: - Create a short RF interconnect line (`connection_rfin`) - Connect that line to `blc_1_ref` port `e2` - Attach PD1 `rfin` to the interconnect output

This anchors PD1 location from the RF topology and keeps placement robust to geometry changes.

[ ]:
```
pd1_ref = c.add_ref(pd).mirror_x()
connection_rfin = ihp.cells.straight(
    length=wavelength/22,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,␣
 ↪signal_cross_section=signal_cross_section,␣
 ↪ground_cross_section=ground_cross_section),
)
connection_rfin_ref = c.add_ref(connection_rfin)
connection_rfin_ref.connect("e1", blc_1_ref.ports["e2"])
pd1_ref.connect("rfin", connection_rfin_ref.ports["e2"],␣
 ↪allow_width_mismatch=True)
c.plot()
```

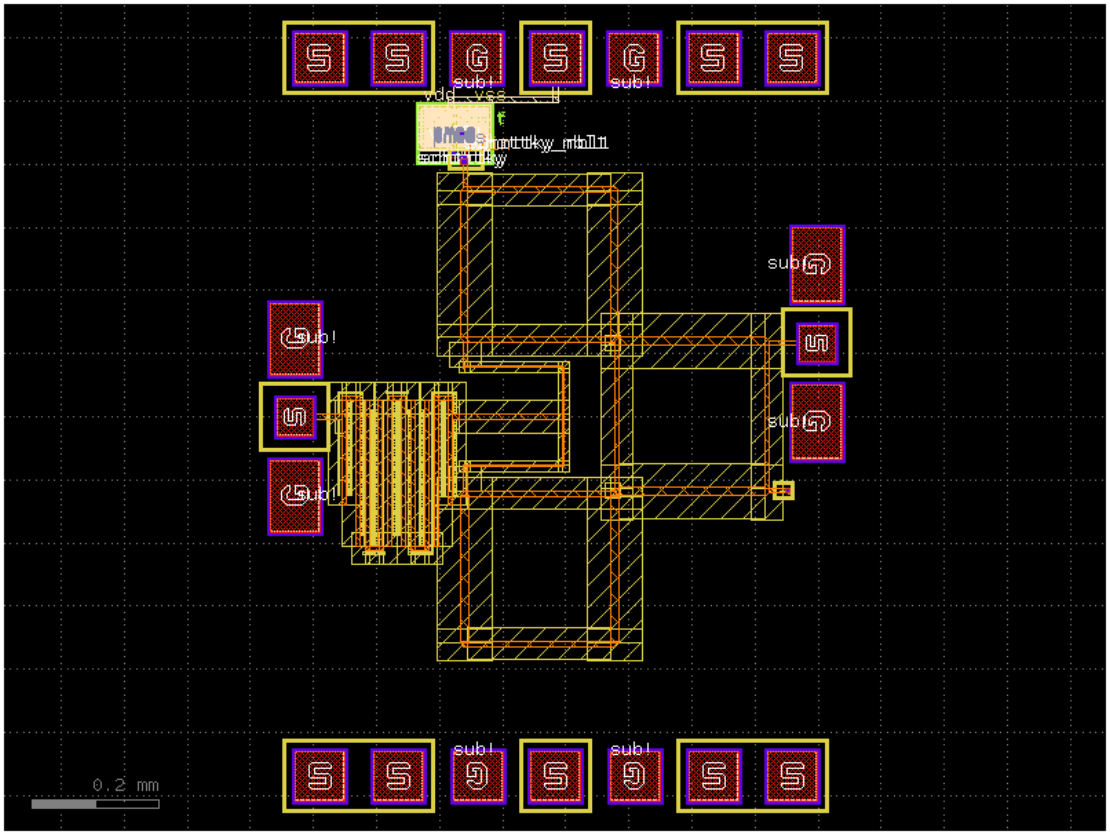
```
Calculated width: 7.2 um for Z0: 50 ohms
```



PD1 is now topology-placed via RF port connections. Next, route its supply and output nets to the top probe pads.

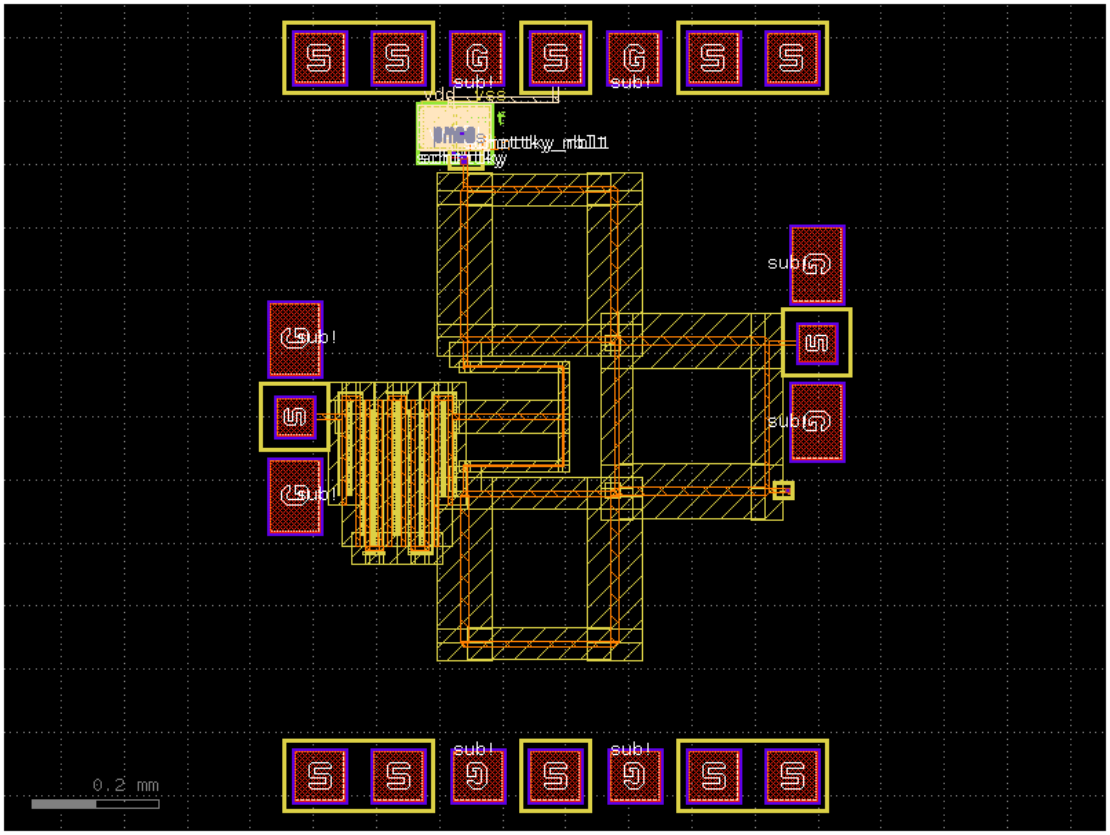
This first step connects PD1 `vdd` using the built-in electrical bundle router.

[ ]:
```
route_pd1_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



Connect PD1 `vss` to the top probe pad using a Metal5 electrical route with a straight lead-in for clean routing.

[ ]:
```
route_pd1_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vss"]],
    ports2=[probe_top.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```

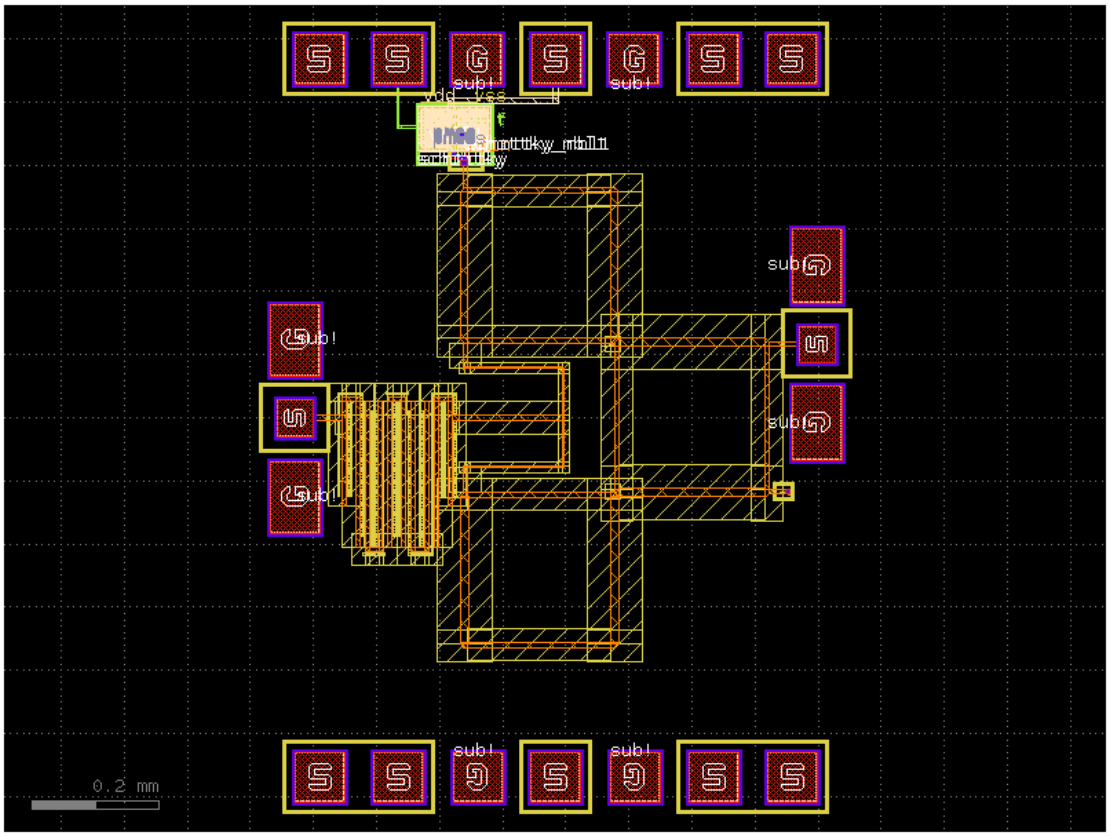


Connect PD1 `vout` to the top probe pad through a Metal4 route using a TopMetal1-to-Metal4 via stack.

[ ]:
```
via_M4_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal4",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
    vt1_rows=1,
)
via_M4_TM1.ports["top"].orientation = 90
via_M4_TM1.ports["bottom"].orientation = 270
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_top.ports["e2"], allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd1_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
```

```
    route_width=4,     # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```

via_stack PyCell placed successfully and GDS written.

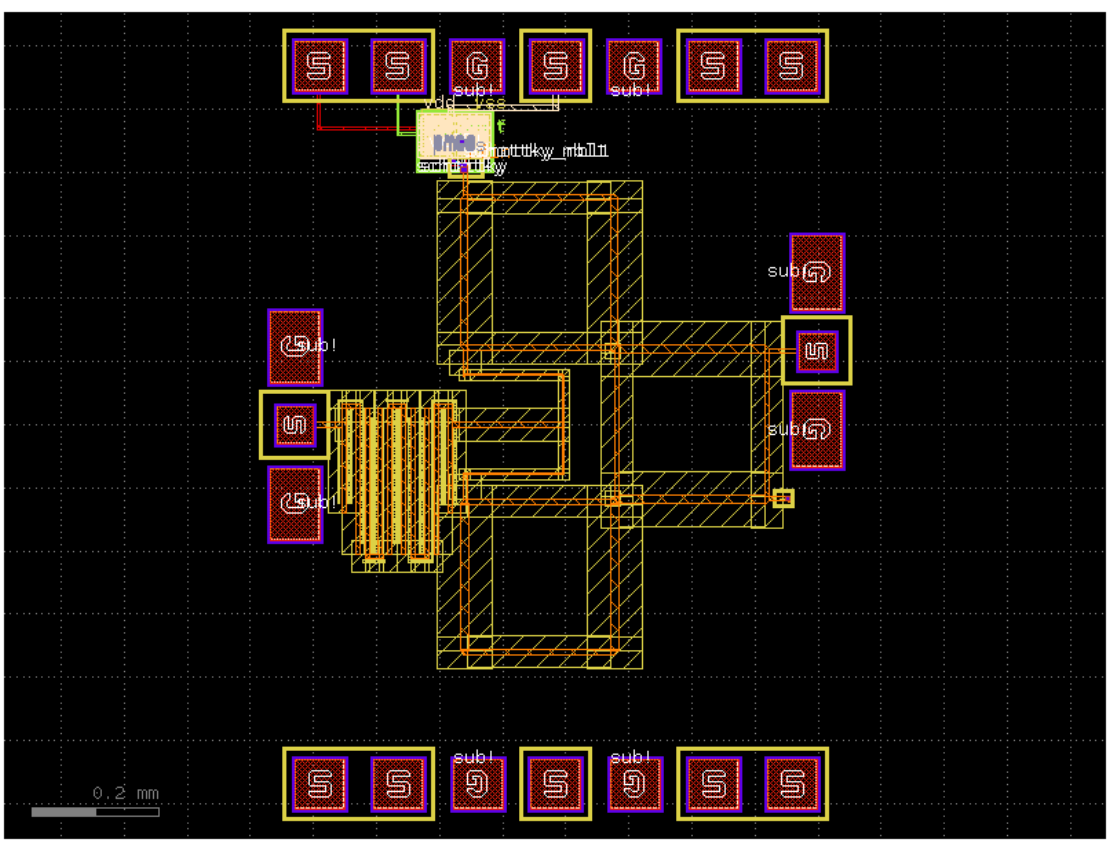


Connect PD1 `vref` to the top probe pad through a Metal3 route using a TopMetal1-to-Metal3 via stack.

[ ]:
```
via_M3_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal3",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
```

```
    vt1_rows=1,
)
via_M3_TM1.ports["top"].orientation = 90
via_M3_TM1.ports["bottom"].orientation = 270
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e1"], allow_width_mismatch=True,␣
 ↪allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
route_pd1_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```

via_stack PyCell placed successfully and GDS written.



PD1 RF and bias routing is complete. Next, instantiate and route PD2 using the complementary orientation and adjacent BLC branch.

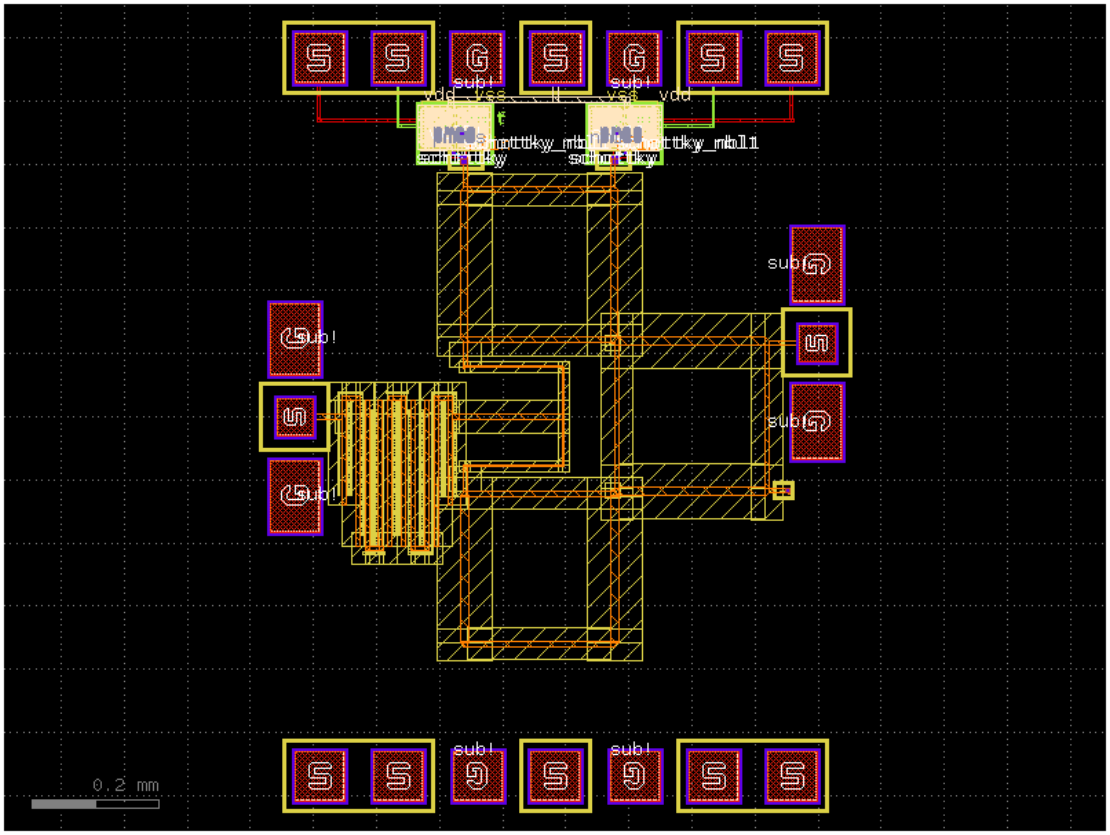
8.12 Place and Connect Second Power Detector (PD2)

Instantiate PD2 and place it by RF connectivity rather than fixed coordinates: - Connect an RF interconnect from `blc_1_ref` port `e3` to PD2 `rfin` - Route `vdd` and `vss` to top probe pads - Route `vout` and `vref` through Metal4/Metal3 paths using via stacks to top pads

This keeps PD2 aligned with the surrounding network as dimensions scale.

[ ]:
```
pd2_ref = c.add_ref(pd)
connection_rfin_ref = c.add_ref(connection_rfin)
connection_rfin_ref.connect("e1", blc_1_ref.ports["e3"])
pd2_ref.connect("rfin", connection_rfin_ref.ports["e2"],␣
 ↪allow_width_mismatch=True)

# vdd connection of pd2 to probe top
route_pd2_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd2 to probe top
route_pd2_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vss"]],
    ports2=[probe_top.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd2 to probe top
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
```

```
via_M4_TM1_ref.connect("top", probe_top.ports["e6"], allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd2_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd2 to probe top
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e7"], allow_width_mismatch=True,␣
↪allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
route_pd2_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c.plot()
```



8.13 Place and Connect Third Power Detector (PD3)

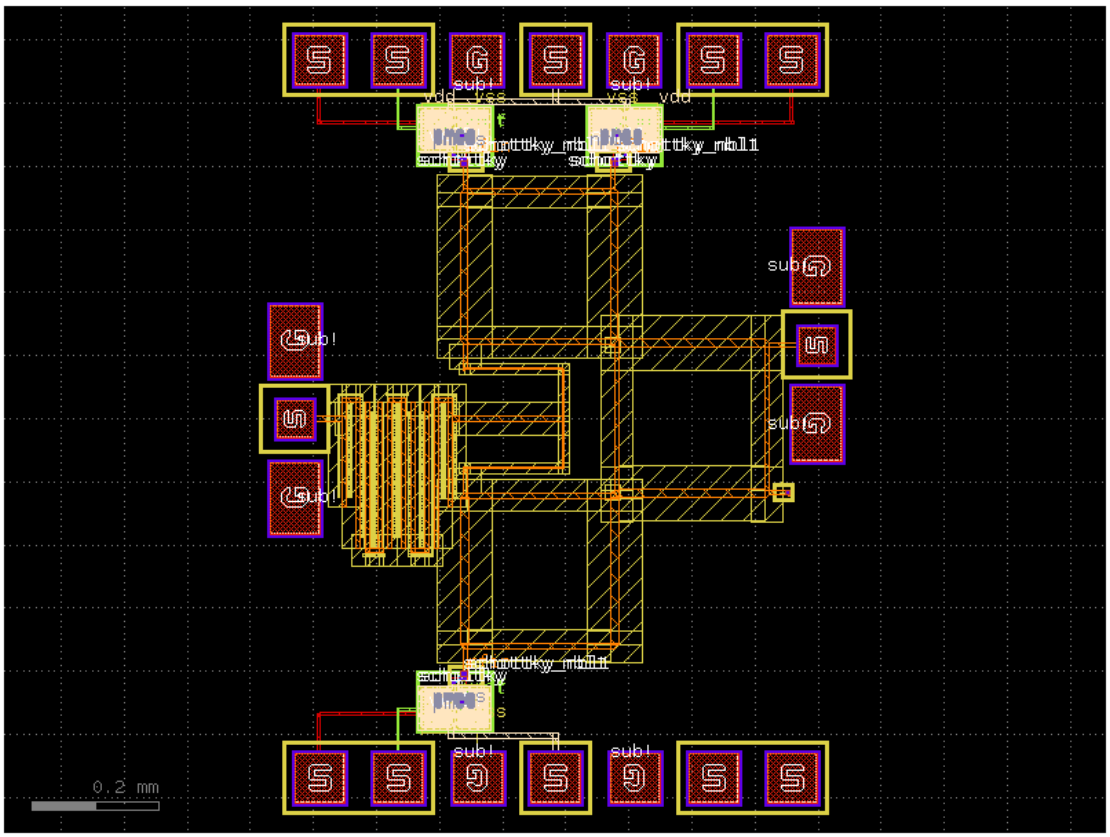
Create PD3 with both mirror operations (`mirror_y().mirror_x()`), then place it through net connectivity: - Drive `rfin` from `blc_2_ref` port `e3` through the RF interconnect - Route `vdd`, `vss`, `vout`, and `vref` to bottom probe pads - Use via-assisted transitions for `vout` (Metal4) and `vref` (Metal3)

PD3 placement is derived from mirror transforms plus port connections, so it adapts automatically with layout changes.

[ ]:
```
pd3_ref = c.add_ref(pd).mirror_y().mirror_x()
connection_rfin_ref = c.add_ref(connection_rfin)
connection_rfin_ref.connect("e1", blc_2_ref.ports["e3"])
pd3_ref.connect("rfin", connection_rfin_ref.ports["e2"],␣
↪allow_width_mismatch=True)

# vdd connection of pd3 to probe bottom
route_pd3_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
```

```
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd3 to probe bottom
route_pd3_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd3 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e6"],␣
 ↪allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))

route_pd3_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd3 to probe bottom
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e7"],␣
 ↪allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd3_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vref"]],
```

```
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,     # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



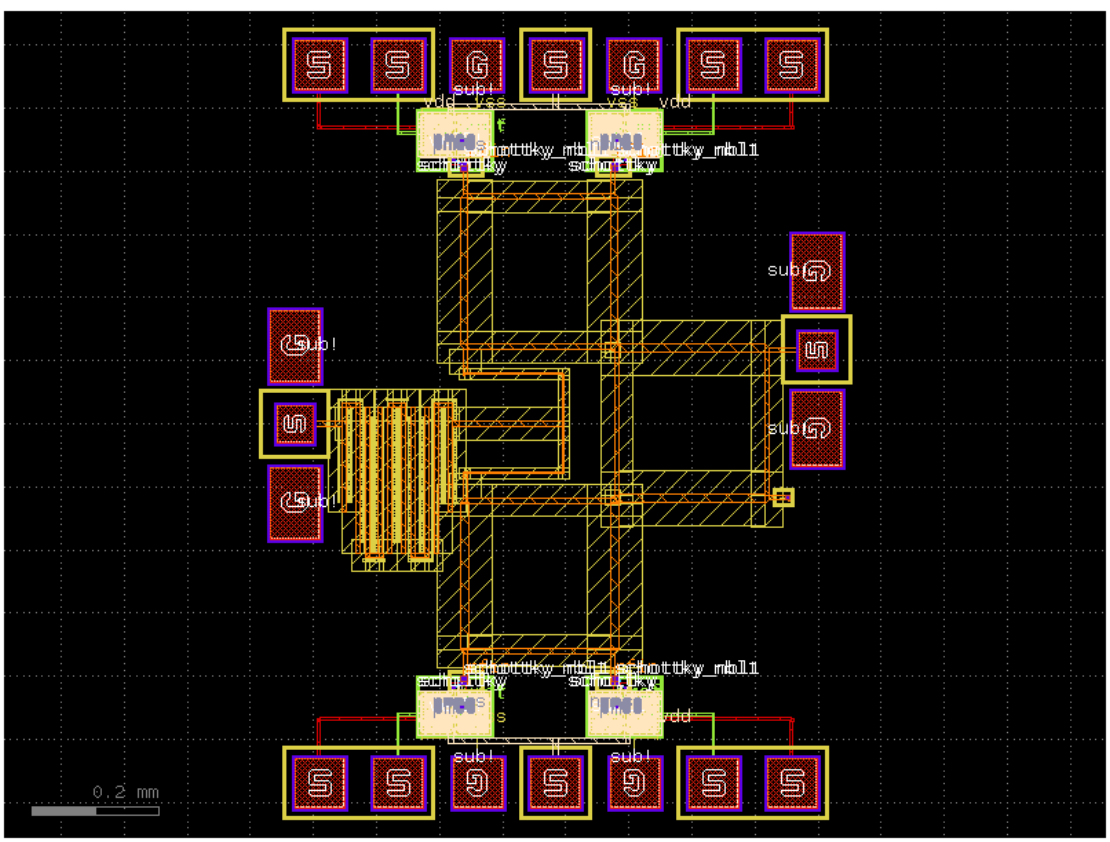
8.14 Place and Connect Fourth Power Detector (PD4)

Create PD4 with a single Y-axis mirror (`mirror_y()`), then place it by connecting ports: - Connect `rfin` from `blc_2_ref` port `e2` through the RF interconnect - Route `vdd` and `vss` to bottom probe pads - Route `vout` and `vref` through via-assisted Metal4/Metal3 paths to bottom pads

Like the other detectors, PD4 location is topology-driven and scales with the surrounding layout geometry.

[ ]:
```
pd4_ref = c.add_ref(pd).mirror_y()
connection_rfin_ref = c.add_ref(connection_rfin)
connection_rfin_ref.connect("e1", blc_2_ref.ports["e2"])
pd4_ref.connect("rfin", connection_rfin_ref.ports["e2"],␣
 ↪allow_width_mismatch=True)

# vdd connection of pd4 to probe bottom
route_pd4_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd4 to probe bottom
route_pd4_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd4 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e2"],␣
 ↪allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))
route_pd4_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
```

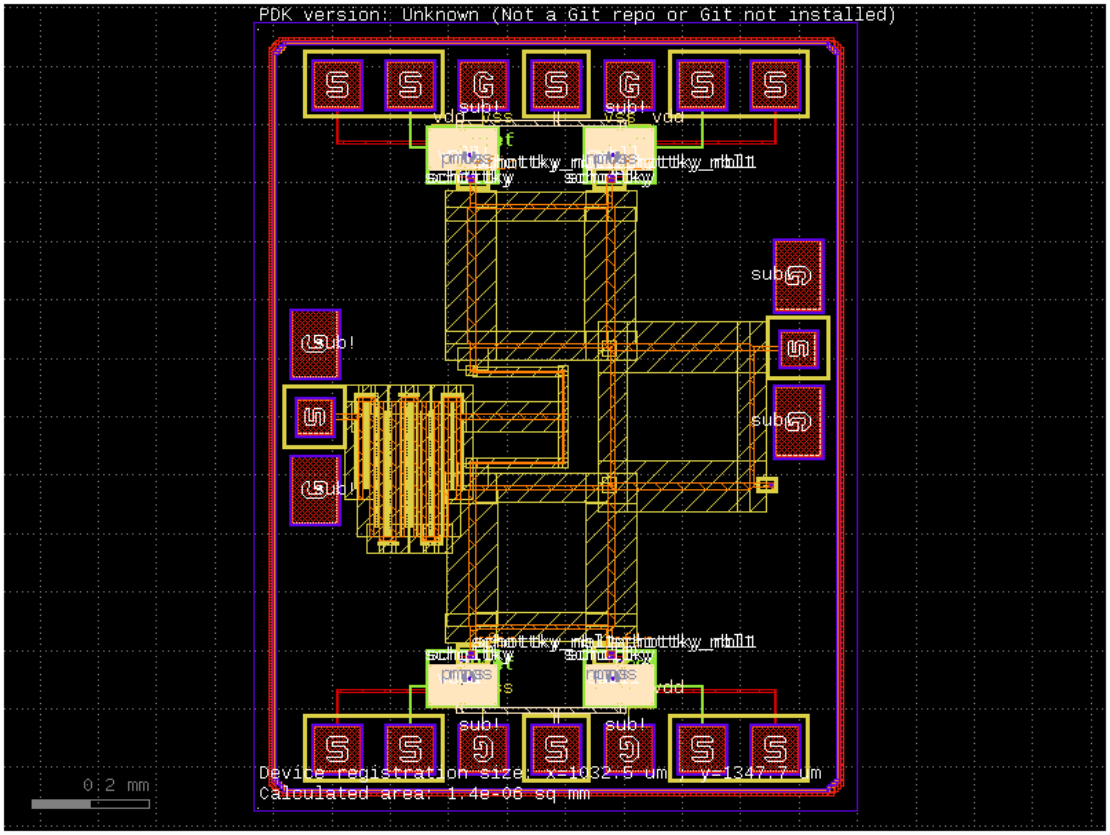
```
    separation=0,
)
# vref connection of pd4 to probe bottom
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e1"],␣
 ↪allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd4_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# rfin connection of pd4 to blc
c.plot()
```



[ ]:
```
c.add_ref(ihp.cells.sealring(width=c.xsize + 50, height=c.ysize + 50)).center =␣
↪c.center

c.plot()
```

```
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
sealring PyCell placed successfully and GDS written.
```

```
fatal: not a git repository (or any of the parent directories): .git
```



8.15 Define Metal Fill Patterns

Define helper functions for creating metal fill cells: - **`fill_cell()`**: Generic rectangular fill cell (9×9µm) on specified layer - **`fill_gat_active()`**: Small fill polygon (1.1×0.5µm) for gate and active region filling - **`fill_ground()`**: Metal5 ground plane fill cell - **`slit_ground()`**: Metal5 slit pattern for ground distribution network

These fill cells are used by gdsfactory's `fill()` function to uniformly distribute metal and maintain electrical properties.

[ ]:
```
@gf.cell
def fill_cell(layer = ihp.tech.LAYER.Metal5slit, size=(3,3)) -> gf.Component:
    c = gf.Component()
    c << gf.c.rectangle(layer=layer, size=size)
    return c

@gf.cell
def fill_gat_active(size=(3,3), active_extension=(0.18,0.18)) -> gf.Component:
    c = gf.Component()
    gat_ref = c.add_ref(gf.c.rectangle(layer=ihp.tech.LAYER.GatPolyfiller,␣
 ↪size=size))
```

```
    activ_ref = c.add_ref(gf.c.rectangle(layer=ihp.tech.LAYER.Activfiller,␣
↪size=(size[0] + 2*active_extension[0], size[1] + 2*active_extension[1])))
    activ_ref.center = gat_ref.center
    return c

@gf.cell
def fill_ground() -> gf.Component:
    c = gf.Component()
    c.add_polygon(layer=ihp.tech.LAYER.Metal5drawing, points=[
        # (0,0),  # used for L
        # (5,0),  # used for L
        # (5,1),  # used for L
        # (1,1),  # used for L
        # (1,5),  # used for L
        # (0,5),  # used for L
        (0,0),
        (1,0),
        (1,1),
        (0,1)
        ])
    return c

@gf.cell
def slit_ground() -> gf.Component:
    c = gf.Component()
    c.add_polygon(layer=ihp.tech.LAYER.Metal5slit, points=[
        (1.1,1.1),
        (4,1.1),
        (4,4.1),
        (1.1,4.1)])
    return c
```

8.16 Fill All Metal Layers with Design Rules Compliance

Apply hierarchical metal fill patterns across all metal layers (GatePoly, Active, Metal1-5, TopMetal1) to: - Maintain CMP (Chemical Mechanical Polishing) uniformity - Ensure design rule compliance with minimum metal density requirements - Avoid isolated structures that degrade process performance - Fill ground planes with optimal slit patterns for RF performance

Each layer uses appropriate fill cells, spacing, and exclusion zones around signal lines and sensitive structures.

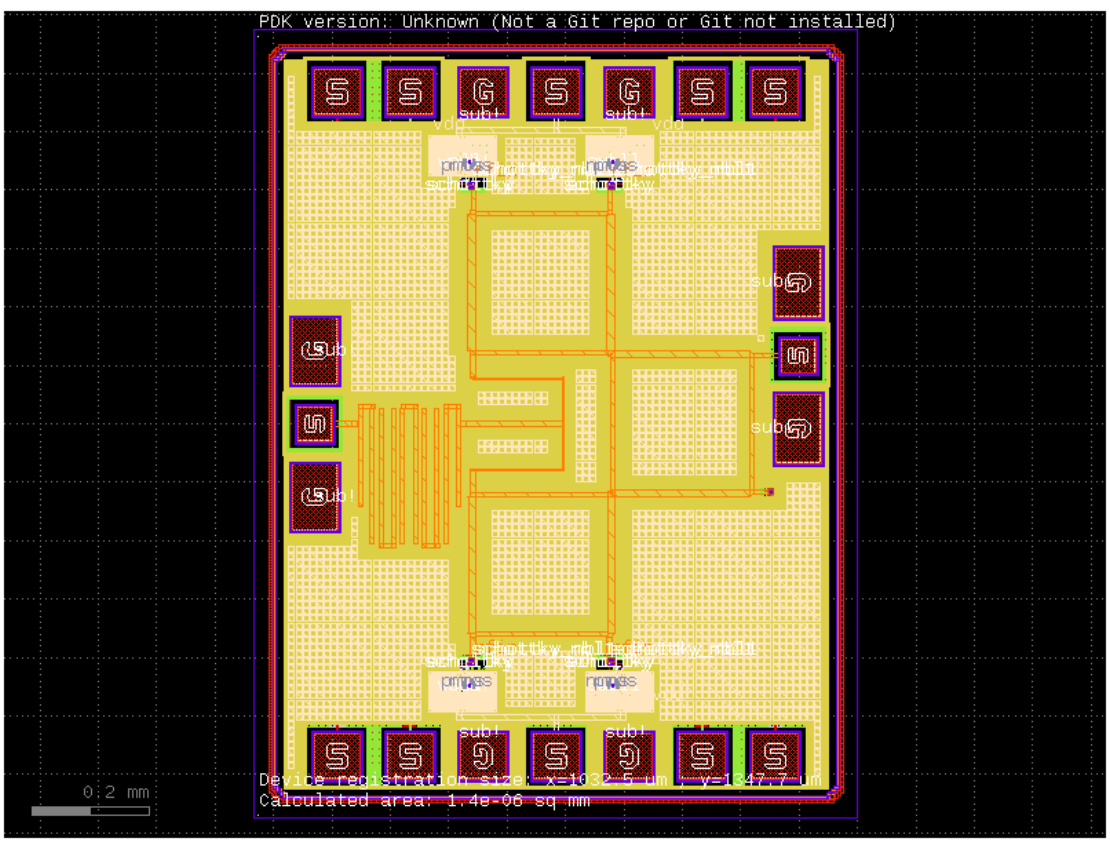
[ ]:
```
gatpoly_activ_fill_space = 1
c.fill(
    fill_cell=fill_gat_active( size=(3, 3)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[
        (ihp.tech.LAYER.Activdrawing, 1.1),
```

```
        (ihp.tech.LAYER.pSDdrawing, 1.1),
        (ihp.tech.LAYER.nSDblock, 1.1),
        (ihp.tech.LAYER.SalBlockdrawing, 1.1),
        (ihp.tech.LAYER.NWelldrawing, 1.1),
        (ihp.tech.LAYER.nBuLaydrawing, 1.1),
        (ihp.tech.LAYER.PWellblock, 1.5),
        (ihp.tech.LAYER.GatPolydrawing, 2),
        (ihp.tech.LAYER.Contdrawing, 1.2),
        (ihp.tech.LAYER.Passivdrawing, 12),
        (ihp.tech.LAYER.GatPolynofill, 1),
        (ihp.tech.LAYER.Activnofill, 1),
        ],
    x_space=gatpoly_activ_fill_space,
    y_space=gatpoly_activ_fill_space,
)
# metal1 fill
metal_fill_space = 1
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal1filler),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal1drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal1nofill, 1),
        ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal2 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal2filler),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal2drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal2nofill, 1),
        ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal3 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal3filler),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal3drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
```

```
        (ihp.tech.LAYER.Metal3nofill, 1),
        ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal4 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal4filler),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal4drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal4nofill, 1),
        ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
c.fill(fill_cell=fill_ground(),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[(ihp.tech.LAYER.Passivdrawing, 0),
    (ihp.tech.LAYER.Metal5nofill, 0)],
    x_space=0,
    y_space=0,
)
c.fill(fill_cell=slit_ground(),
    fill_layers=[(ihp.tech.LAYER.Metal5drawing, -2)],
    exclude_layers=[(ihp.tech.LAYER.TopMetal2drawing, 5), (ihp.tech.LAYER.
 ↪MIMdrawing, 1), (ihp.tech.LAYER.Metal5slit, 1)],
    x_space=1.1,
    y_space=1.2,
)
c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal1filler, size=(9, 9)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal1drawing, 10), (ihp.tech.LAYER.
 ↪TopMetal2drawing, 20)],
        x_space=3,
        y_space=3,
    )
c.plot()
```

```
2026-04-10 13:34:44.081 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:44.181 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
```

```
2026-04-10 13:34:44.270 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:44.342 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:44.414 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:44.459 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:46.286 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
2026-04-10 13:34:46.355 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with

calculating fill regions for Six-Port-CAC
```

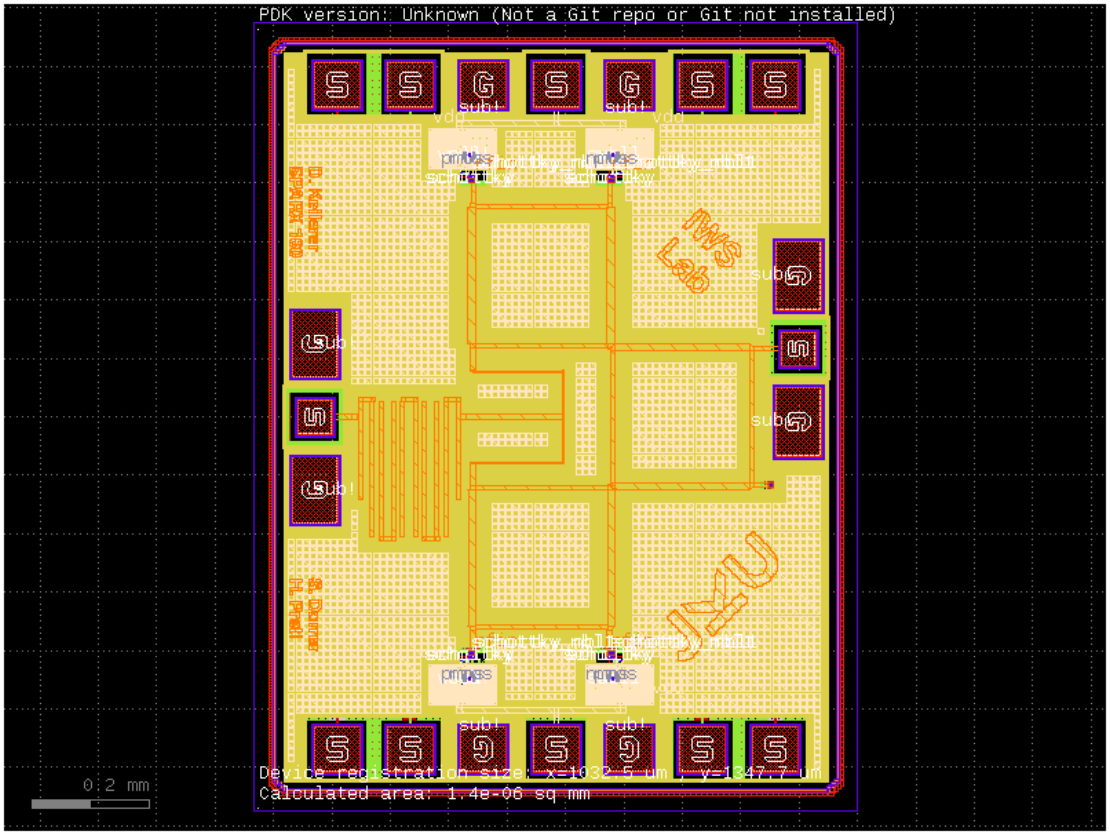


8.17 Convert Logos and Add Decorative Elements

Import and place decorative logos for the chip: - **JKU Logo**: Johannes Kepler University branding on TopMetal2 - **D. Kellerer**: Designer name annotation (rotated -90°) - **IWS Logo**: Institute branding
- **Supervisors**: Acknowledgment text

Convert PNG image files to GDS format with specified pixel size (2µm), scaling (0.15-0.15), and layer assignment. Place logos in optimized positions around the chip perimeter.

[ ]:
```
logo_dir = Path("assets")


logo_gds_path = logo_dir / "jku_logo_m5.gds"

kellerer_gds_path = logo_dir / "D_Kellerer.gds"

iws_gds_path = logo_dir / "IWS.gds"

helper_gds_path = logo_dir / "supervisors.gds"

tm1_layer = ihp.tech.LAYER.TopMetal2drawing
tm1_foreground = f"{tm1_layer[0]}/{tm1_layer[1]}"


logo_jku = c.add_ref(gf.import_gds(str(logo_gds_path), cellname="JKU_LOGO"))

logo_jku.move((-50,-420))

kellerer = c.add_ref(gf.import_gds(str(kellerer_gds_path), cellname="Name_D"))

iws = c.add_ref(gf.import_gds(str(iws_gds_path), cellname="Logo_IWS"))


kellerer.rotate(-90)

kellerer.move((-690, 430))

iws.move((-120,130))

helper = c.add_ref(gf.import_gds(str(helper_gds_path), cellname="helper")).
 ↪rotate(-90)
helper.xmin = kellerer.xmin
helper.ymax = kellerer.ymin - 550

c.plot()
```

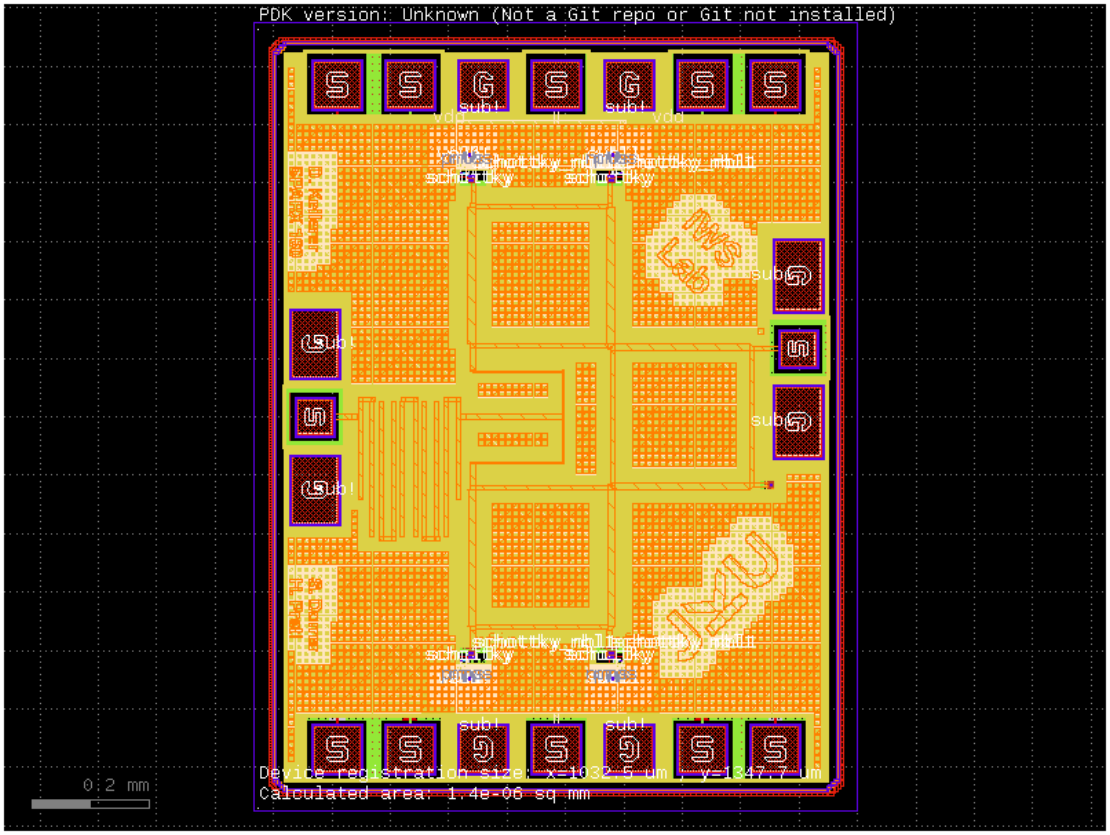


8.18 Final TopMetal2 Fill and Chip Visualization

Apply TopMetal2 fill pattern to complete metal density requirements. Use 9×9µm fill cells with 3µm spacing, excluding the 20µm perimeter around TopMetal2 drawing layers. Plot the final complete chip design to visualize the finished layout with all components, routing, logos, and metal fill in place.

[ ]:
```
c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal2filler, size=(9, 9)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal2drawing, 20)],
        x_space=3,
        y_space=3,
    )
c.plot()
```

```
2026-04-10 13:34:47.485 | INFO     |
kfactory.utils.fill:fill_tiled:278 - done with
calculating fill regions for Six-Port-CAC
```



[ ]:
```
! rm -rf IHP/
! rm -rf pdk/
```